

Methods for solving linear systems

Reminders on norms and scalar products of vectors. The application $\| \cdot \| : \mathbb{R}^n \rightarrow \mathbb{R}$ is a **norm** if

$$1 - \|\underline{v}\| \geq 0, \quad \forall \underline{v} \in \mathbb{R}^n \quad \|\underline{v}\| = 0 \text{ if and only if } \underline{v} = 0;$$

$$2 - \|\alpha \underline{v}\| = |\alpha| \|\underline{v}\| \quad \forall \alpha \in \mathbb{R}, \quad \forall \underline{v} \in \mathbb{R}^n;$$

$$3 - \|\underline{v} + \underline{w}\| \leq \|\underline{v}\| + \|\underline{w}\|, \quad \forall \underline{v}, \underline{w} \in \mathbb{R}^n.$$

Examples of norms of vectors:

$$\|\underline{v}\|_2^2 = \sum_{i=1}^n (v_i)^2 \quad \text{Euclidean norm}$$

$$\|\underline{v}\|_\infty = \max_{1 \leq i \leq n} |v_i| \quad \text{max norm}$$

$$\|\underline{v}\|_1 = \sum_{i=1}^n |v_i| \quad \text{1-norm}$$

Being in finite dimension, they are all equivalent, with the equivalence constants depending on the dimension n . Ex: $\|\underline{v}\|_\infty \leq \|\underline{v}\|_1 \leq n \|\underline{v}\|_\infty$.

A **scalar product** is an application $(\cdot, \cdot) : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ that verifies:

- 1 – linearity: $(\alpha \underline{v} + \beta \underline{w}, \underline{z}) = \alpha(\underline{v}, \underline{z}) + \beta(\underline{w}, \underline{z}) \quad \forall \alpha, \beta \in \mathbb{R}, \quad \forall \underline{v}, \underline{w}, \underline{z} \in \mathbb{R}^n$;
- 2 – $(\underline{v}, \underline{w}) = (\underline{w}, \underline{v}) \quad \forall \underline{v}, \underline{w} \in \mathbb{R}^n$;
- 3 – $(\underline{v}, \underline{v}) > 0 \quad \forall \underline{v} \neq \underline{0}$ (that is, $(\underline{v}, \underline{v}) \geq 0$, $(\underline{v}, \underline{v}) = 0$ iff $\underline{v} = \underline{0}$).

To a scalar product we can associate a norm defined as

$$\|\underline{v}\|^2 = (\underline{v}, \underline{v}).$$

Example: $(\underline{v}, \underline{w}) = \sum_{i=1}^n v_i w_i, \quad \implies \quad (\underline{v}, \underline{v}) = \sum_{i=1}^n v_i v_i = \|\underline{v}\|_2^2.$

(in this case we can write $(\underline{v}, \underline{w}) = \underline{v} \cdot \underline{w}$ or $\underline{v}^T \underline{w}$ for “column” vectors)

Theorem 1 (Cauchy-Schwarz inequality)

Given a scalar product $(\cdot, \cdot)_*$ and associated norm $\|\cdot\|_*$, the following inequality holds:

$$|(\underline{v}, \underline{w})_*| \leq \|\underline{v}\|_* \|\underline{w}\|_* \quad \forall \underline{v}, \underline{w} \in \mathbb{R}^n$$

Proof.

For $t \in \mathbb{R}$, let $t\underline{v} + \underline{w} \in \mathbb{R}^n$. Clearly, $\|t\underline{v} + \underline{w}\|_* \geq 0$. Hence:

$$\|t\underline{v} + \underline{w}\|_*^2 = t^2 \|\underline{v}\|_*^2 + 2t(\underline{v}, \underline{w})_* + \|\underline{w}\|_*^2 \geq 0$$

The last expression is a non-negative convex parabola in t (No real roots, or 2 coincident). Then the discriminant is non-positive

$$(\underline{v}, \underline{w})_*^2 - \|\underline{v}\|_*^2 \|\underline{w}\|_*^2 \leq 0$$

and the proof is concluded. □

Reminders on matrices $A \in \mathbb{R}^{n \times n}$

- A is symmetric if $A = A^T$. The eigenvalues of a symmetric matrix are real.
- A symmetric matrix A is positive definite if

$$(A\underline{x}, \underline{x})_2 > 0 \quad \forall \underline{x} \in \mathbb{R}^n, \quad \underline{x} \neq \underline{0}, \quad (A\underline{x}, \underline{x})_2 = 0 \text{ iff } \underline{x} = \underline{0}$$

The eigenvalues of a positive definite matrix are positive.

- if A is non singular, $A^T A$ is symmetric and positive definite

Proof of the last statment:

- $A^T A$ is always symmetric; indeed $(A^T A)^T = A^T (A^T)^T = A^T A$.

To prove that it is also positive definite we have to show that

$(A^T A\underline{x}, \underline{x})_2 > 0 \quad \forall \underline{x} \in \mathbb{R}^n, \quad \underline{x} \neq \underline{0}, \quad (A^T A\underline{x}, \underline{x})_2 = 0 \text{ iff } \underline{x} = \underline{0}$. We have:

$$(A^T A\underline{x}, \underline{x})_2 = (A\underline{x}, A\underline{x})_2 = \|A\underline{x}\|_2^2 \geq 0, \text{ and } \|A\underline{x}\|_2^2 = 0 \text{ iff } A\underline{x} = \underline{0}$$

If A is non-singular (i.e., $\det(A) \neq 0$), the system $A\underline{x} = \underline{0}$ has only the solution $\underline{x} = \underline{0}$, and this ends the proof.

Norms of matrices

Norms of matrices are applications from $\mathbb{R}^{m \times n}$ to $\mathbb{R}$ satisfying the same properties as for vectors. Among the various norms of matrices we will consider the norms associated to norms of vectors, called natural norms, defined as:

$$|||A||| = \sup_{\underline{v} \neq 0} \frac{\|A\underline{v}\|}{\|\underline{v}\|}$$

It can be checked that this is indeed a norm, that moreover verifies:

$$\|A\underline{v}\| \leq |||A||| \|\underline{v}\|, \quad |||AB||| \leq |||A||| |||B|||.$$

Examples of natural norms (of square $n \times n$ matrices)

$$\underline{v} \in \mathbb{R}; \quad \|\underline{v}\|_{\infty} \longrightarrow \|A\|_{\infty} = \max_{i=1, \dots, n} \sum_{j=1}^n |a_{ij}|,$$

$$\underline{v} \in \mathbb{R}; \quad \|\underline{v}\|_1 \longrightarrow \|A\|_1 = \max_{j=1, \dots, n} \sum_{i=1}^n |a_{ij}|,$$

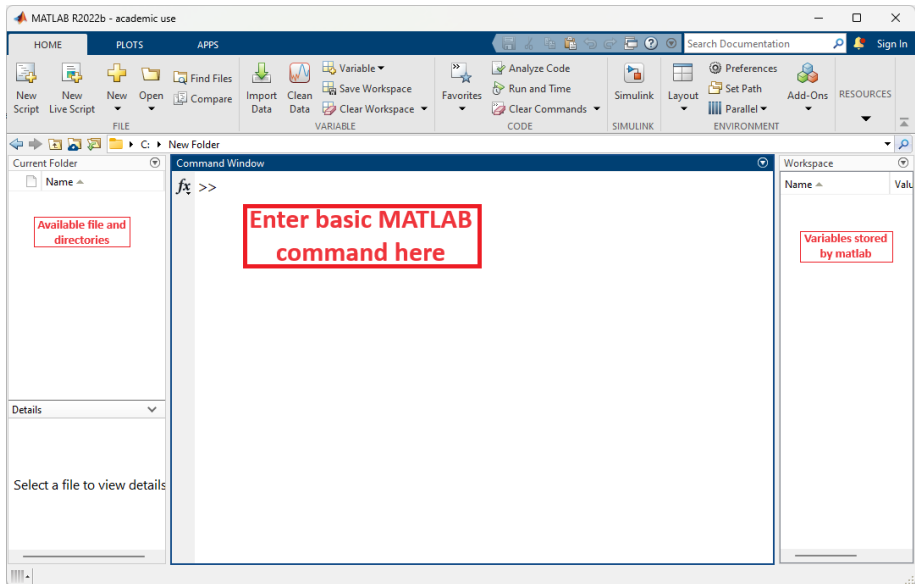
$$\underline{v} \in \mathbb{R}; \quad \|\underline{v}\|_2 \longrightarrow \|A\|_2 = \sqrt{|\lambda_{\max}(A^T A)|}.$$

If A is symmetric, $\|A\|_{\infty} = \|A\|_1$, and $\|A\|_2 = |\lambda_{\max \text{ in abs val}}(A)|$.

Indeed, if $A = A^T$, then $\max_i \lambda_i(A^T A) = \max_i \lambda_i(A^2) = (\max_i \lambda_i(A))^2$.

The norm $\|A\|_2$ is the *spectral norm*, since it depends on the spectrum of A .

Introduction on MATLAB



Command Window

The command window accepts variables declaration, expressions, function calling and from it we can run scripts.

- Writing in the Command Window:

```
>> a = 1.54
```

create a variable `a` with value `1.54`, and it is stored in MATLAB Workspace

- Adding `;` at the end of a command prevent MATLAB from printing the result of the command in the Command Window
- Variable name can be max 31 character (letter or number) and the first must not be a number. Moreover names are case-sensitive
- if we write an expression without specifying a variable name, MATLAB will use the default variable `ans`

Common commands

- **clc**: deletes all the contents of the Command Window, but maintains the variable stored in the Workspace.
- **clear**: deletes all the variable in the Workspace.
- **clear var**: deletes the variable **var** in the Workspace.
- **help** followed by a command (or a MATLAB function), prints in the Command Window a small guide on the command.
- If a and b are numbers (or numeric variables), then
 - `>> a + b` is the sum
 - `>> a - b` is the difference
 - `>> a * b` is the product
 - `>> a/b` is a divided by b
 - `>> a\b` is b divided by a
 - `>> a^b` is a raised to the power of b.

- Round brackets are used to specify the order of operations in composite expressions, for example the command:

```
>> a = (1 + 3)/(2^(1 + 2));
```

stores in the Workspace the variable `a` with value 0.5, without showing the results in the Command Window.

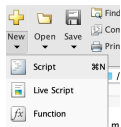
- Other useful math functions available in MATLAB are: `sin cos tan asin acos atan exp log log10 abs sqrt sign ...`. For a guide on how they work, use:

```
>> help sin, or >> help cos, ...
```

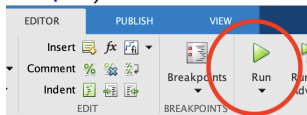
Scripts

A script is a text file (with extension `.m`, for example `Script1.m`) containing a sequence of instructions, one per line.

To create a script, use the button 'New'



To execute a script, either press 'Run' or use the file name (for example `Script1`) without an extension in the Command Window



Executing a script is equivalent to typing the commands in the Command Window

- it has not arguments
- It does not return the results
- All variables in the Workspace can be read/written

example of script: *Script1.m*

```
a=1
```

```
b=a+1
```

```
a=a*b
```

Executed with:

```
>> Script1
```


Functions

In MATLAB, it is possible to create new functions. You just need to create a file with the .m extension and a filename matching the desired function name. The first line of the file must start with the reserved word **function** followed by the output arguments, the function name and the input arguments. In the following lines, you should write the code for the function, and finally, the last line should contain the reserved word **end**.

example of function: *fun1.m*

```
function [x,y] = fun1(a,b)
x=(a+b)/2;
y=sqrt(a^2+b^2);
end
```

a and *b* are the input arguments, while *x* and *y* are the output arguments.

Variables used in a function are local and therefore independent of those in the calling environment.

To add comments to MATLAB code (scripts or functions), use the percent symbol `%` . Comment lines can appear anywhere in a code file, and you can append comments to the end of a line of code. **example of a function with comments:** *fun1.m*

```
function [x,y] = fun1(a,b)
x=(a+b)/2; % Arithmetic mean
% Second part of the code
y=sqrt(a^2+b^2);% Length of the vector (a,b)
end
```

Matrices and vectors

To define a matrix, you write the coefficients between '`[]`',
the coefficients are separated by '`,`' or by space '',
the rows are separated by '`;`'

Example: `A=[1,4;2,5;3,6];` defines a variable that contains the matrix

$$A = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

Accessing the Coefficients

To access the coefficients, you use row and column indices enclosed in parentheses `()`, where indices start from 1.

Continuation of the example:

$$A = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

`A(1,2)`

returns 4

`A(3,2)`

returns 6

`A(1,3)`

Error!

`A(4,1)`

Error!

`A(0,1)`

Error!

The coefficients can be modified

`A(3,2)=9`

A becomes $\begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 9 \end{pmatrix}$

Vectors are Matrices

To define a vector, you use a matrix of size $n \times 1$.

Example:

`v=[1;2;3];`

defines a variable containing

$$v = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

Note that there are also row vectors: matrices of size $1 \times n$

`w=[1,2,3]`

defines a variable containing

$$w = (1 \quad 2 \quad 3)$$

Accessing the Coefficients

$$v = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad w = (1 \ 2 \ 3)$$

<code>v(1)</code>	returns 1
<code>v(4)</code>	Error!
<code>v(0)</code>	Error!
same for <code>w</code>	

Row vectors are $1 \times n$ matrices, and column vectors are $n \times 1$ matrices.

<code>v(2,1)</code>	returns 2
<code>v(1,2)</code>	Error!
<code>w(2,1)</code>	Error!
<code>w(1,2)</code>	returns 2

Matrices as Vectors

You can use a single index with matrices as well!

$$A = \begin{pmatrix} a_{1,1} & \cdots & a_{1,m} \\ \vdots & & \vdots \\ a_{n,1} & \cdots & a_{n,m} \end{pmatrix}$$

$A(k)$ is $A(s,t)$ for (s,t) that satisfies $(t-1)n + (s-1) = k$, which means it's the k -th coefficient when going:

- from top to bottom
- from left to right

$A(:)$ is the column vector $[A(1) \dots A(n*m)]'$.

You can modify A by assigning a vector to $A(:)$.

Example: If A is a matrix obtained with $A=\text{zeros}(2,2);$,

$$A(:)=[1 \ 2 \ 3 \ 4]';$$

is equivalent to $A=[1 \ 3; 2 \ 4];$

Scalars are Matrices

A scalar is a 1×1 matrix.

and you can access the single coefficient: if `a=3;`, then `a(1,1)` is 3.

Almost everything in MATLAB is a matrix!

Basic Operations

Knowing the dimensions:

`size(A)` returns a vector containing the number of rows and the number of columns

```
A=[1,4;2,5;3,6]
```

```
size(A)                                returns [3,2]
```

For a single dimension (e.g., number of rows), you can use `size(A,1)`

Transpose

An apostrophe `'` to the right of a matrix transposes it.

```
v=[1 2 3; 4 5 6]';
```

defines the matrix

$$v = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

Note: `[1 2 3]'` is a column vector.

To Verify

Verify that by defining a matrix

$$A = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix},$$

A(1)	returns 1
A(2)	returns 2
A(3)	returns 3
A(4)	returns 4
A(5)	returns 5
A(6)	returns 6
A(7)	Error!

...Continuation

Defining a matrix and then modifying it:

Use the functions 'zeros' or 'ones'

or with arguments for rows and columns

```
A=zeros(3,2)
```

A is a 3×2 matrix with all zeros

```
A=ones(3,2)
```

A is a 3×2 matrix with all ones

or using a vector for dimensions

```
sz=[3,2];
```

```
A=zeros(sz)
```

A is a 3×2 matrix with all zeros

```
A=ones(sz)
```

A is a 3×2 matrix with all ones

Blocks

The instructions in a function can be included in blocks.

In MATLAB, a block starts with a keyword such as

- `if`
- `for`
- `while`
- ...

and ends with the keyword `end`.

You can nest blocks inside another block.

Unlike Python, indentation does not delimit blocks!

```
if ...end
```

```
if a==1
```

```
    b=9; % executes between "if" and "end" only if "a" is 1  
end
```

```
if ...else ...end
```

```
if a==1
```

```
    b=9; % executes between "if" and "else" if "a" is 1
```

```
else
```

```
    b=8; % executes between "else" and "end" if "a" is not 1
```

```
end
```

Syntax for Multiple if

The following three codes are equivalent:

Multiple if:

```
if a==1
    disp('one');
else
    if a==2
        disp('two');
    else
        if a==3
            disp('three');
        else
            disp('big')
        end
    end
end
end
```

if...elseif...:

```
if a==1
    disp('one');
elseif a==2
    disp('two');
elseif a==3
    disp('three');
else
    disp('big')
end
```

switch...case...:

```
switch a
    case 1
        disp('one');
    case 2
        disp('two');
    case 3
        disp('three');
    otherwise
        disp('big')
end
```

For knowing more about the function 'disp', use `>> help disp`

Conditional Syntax

Comparisons: Take two scalars a and b and return a logical value between true and false

$a==b$ true if they are equal

$a\sim b$ true if they are not equal

$a<b$ true if a is strictly less than b

$a\leq b$ true if a is less than or equal to b

$a>b, a\geq b$ as for less than

Logical Operations:

$a|b$ a or b (true if at least one is true)

$a\&b$ a and b (true if both are true)

$\sim a$ true if a is false and vice versa

For numeric values, " $a=\text{true}$ " is equivalent to " $a \neq 0$ ".


```
for ...end
```

```
b=0;  
for i=3:10  
    b=b*b+i;  
end
```

Executes between `for` and `end` first with $i = 3$, then with $i = 4$ up to $i = 10$.

```
while ...end
```

```
We cannot always use for. a=1;  
b=0;  
while a<10  
    a=a+1;  
    b=b*b+1;  
end
```

Executes between "while" and "end" repeatedly. Before each iteration, it checks if $a < 10$, and if $a \geq 10$, it proceeds to the next instruction after the block.

`while true` starts an infinite loop.

Use *ctrl-c* to stop an infinite loop.

continue and break

```
a=1;
b=0;
while a<10 % continue jumps here
    a=a+1;
    if b>10
        continue; % or break;
    end
    b=b*b+1;
end
% break jumps here
```

continue goes to the condition check of the while loop.
break jumps to the first instruction after the while loop.

These can also be used with for loops.

Limitations

If a while loop contains another while loop, then continue and break affect the **innermost** while loop that encloses them.

```
i=0;
j=0;
a=1;
while i<10
    i=i+1;
    while j<5
        j=j+1;
        if i+j<10
            continue; % or break;
        end % if
    end % inner while
    a=a*(i+j);
end % outer while
```

To affect the outer while, you need to repeat the control block (if i+j<10 continue; end).

Solving linear systems

The problem: given $\underline{b} \in \mathbb{R}^n$, and $A \in \mathbb{R}^n \times \mathbb{R}^n$, we look for $\underline{x} \in \mathbb{R}^n$ solution of

$$A\underline{x} = \underline{b} \quad (1)$$

Problem (1) has a unique solution *if and only if* the matrix A is non-singular (or invertible), i.e., $\exists A^{-1}$ such that $A^{-1}A = AA^{-1} = I$; necessary and sufficient condition for A being invertible is that $\det(A) \neq 0$. Then the solution $\underline{x}$ is formally given by $\underline{x} = A^{-1}\underline{b}$.

Beware: never invert a matrix unless really necessary, due to the costs, as we shall see later on. (Solving a system with a general full matrix is also expensive, but not nearly as expensive as matrix inversion).

Some example of linear systems

The simplest systems to deal with are diagonal systems:

$$D\underline{x} = \underline{b}$$

$$D = \begin{bmatrix} d_{11} & 0 & \cdots & 0 \\ 0 & d_{22} & \cdots & 0 \\ 0 & \cdots & \ddots & 0 \\ 0 & \cdots & & d_{nn} \end{bmatrix} \longrightarrow x_i = \frac{b_i}{d_{ii}} \quad i = 1, 2, \dots, n$$

The cost in terms of number of operations is negligible, given by n division.

Lower Triangular matrices

Triangular matrices are also easy to handle. If $A = L$ is lower triangular, the system can be solved “forward”:

$$L = \begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{bmatrix} \longrightarrow \begin{cases} x_1 = \frac{b_1}{l_{11}} \\ x_2 = \frac{b_2 - l_{21}x_1}{l_{22}} \\ \vdots \\ x_n = \frac{b_n - \sum_{j=1}^{n-1} l_{nj}x_j}{l_{nn}} \end{cases}$$

Counting the operations: for x_1 we have 1 product and 0 sums; for x_2 2 products and 1 sum, ..., and for x_n n products and $n - 1$ sums, that is, $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$ products, plus $1 + 2 + \cdots + n - 1 = \frac{(n-1)n}{2}$ sums, for a **total number of operations** $= n^2$.

Forward Substitution

What follows is an algorithm, for solving $L\underline{x} = \underline{b}$, with L lower triangular.

forward substitution

Input: $L \in \mathbb{R}^{n \times n}$, lower t., and $b \in \mathbb{R}^n$

for $i = 1, \dots, n$

for $j = 1, \dots, i - 1$

$$b_i = b_i - l_{i,j}b_j$$

end

$$b_i = \frac{b_i}{l_{ii}}$$

end

the solution is in $\underline{b}$, i.e., $\underline{x} \leftarrow \underline{b}$

Homework

Write the above algorithm in MATLAB

A special case of lower triangular matrix (useful later...)

Assume L has only 1 on the diagonal, e.g., $l_{ii} = 1$:

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ l_{n1} & l_{n2} & \cdots & 1 \end{bmatrix} \rightarrow \begin{cases} x_1 = b_1 \\ x_2 = b_2 - l_{21}x_1 \\ \vdots \\ x_n = b_n - \sum_{j=1}^{n-1} l_{nj}x_j \end{cases}$$

Then the following two algorithms, for solving $L\underline{x} = \underline{b}$ are equivalent:

forward substitution as above

Input: $L \in \mathbb{R}^{n \times n}$, lower t., and $b \in \mathbb{R}^n$

for $i = 1, \dots, n$

for $j = 1, \dots, i - 1$

$b_i = b_i - l_{i,j}b_j$

end

end

the solution is in $\underline{b}$, i.e., $\underline{x} \leftarrow \underline{b}$

equivalent to forward substitution

Input: $L \in \mathbb{R}^{n \times n}$, lower t., and $b \in \mathbb{R}^n$

for $k = 1, \dots, n - 1$

for $i = k + 1, \dots, n$

$b_i = b_i - l_{i,k}b_k$

end

end

the solution is in $\underline{b}$, i.e., $\underline{x} \leftarrow \underline{b}$

Upper Triangular matrices

Upper triangular systems (if $A = U$ is upper triangular) are also easy to deal with, and can be solved “backward” with the same costs as lower triangular systems:

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & & \vdots \\ 0 & & \cdots & u_{nn} \end{bmatrix} \longrightarrow \begin{cases} x_n = \frac{b_n}{u_{nn}} \\ x_{n-1} = \frac{b_{n-1} - u_{n-1n}x_n}{u_{n-1n-1}} \\ \vdots \\ x_1 = \frac{b_1 - \sum_{j=2}^n u_{1j}x_j}{u_{11}} \end{cases}$$

Backward Substitution

What follows is an algorithm, for solving $U\underline{x} = \underline{b}$, with U upper triangular.

backward substitution

Input: $L \in \mathbb{R}^{n \times n}$, upper t., and
 $b \in \mathbb{R}^n$

for $i = n, \dots, 1$

for $j = i + 1, \dots, n$

$$b_i = b_i - u_{i,j}b_j$$

end

$$b_i = \frac{b_i}{u_{ii}}$$

end

the solution is in $\underline{b}$, i.e., $\underline{x} \leftarrow \underline{b}$

Homework

Write the above algorithm in MATLAB

Costs for solving triangular systems

We saw that for solving the system we perform n^2 operations.

To have an idea of the time necessary to solve a triangular system, suppose that the number of equations is $n = 100.000$, and the computer performance is a TERAFL0P = 10^{12} FLOPS (floating-point operations per second); the time in seconds is given by

$$t = \frac{\#operations}{\#flops} = \frac{10^{10}ops}{10^{12}flops} = \frac{1}{100}sec.$$

(pretty quick)

The next target of High Performance Computing is EXAFLOP
= 10^{18} FLOPS

Numerical methods

There are two classes of methods for solving the linear system

$$A\underline{x} = \underline{b} \quad (1)$$

Direct methods: they give the exact solution (up to computer precision) in a finite number of operations.

Iterative methods: starting from an initial guess $\underline{x}^{(0)}$ they construct a sequence $\{\underline{x}^{(k)}\}$ such that

$$\underline{x} = \lim_{k \rightarrow \infty} \underline{x}^{(k)}.$$

Direct methods

The most important direct method is **GEM** (Gaussian elimination method).

With GEM, the original system (1) is transformed into an equivalent system (i.e., having the same solution)

$$U\underline{x} = \underline{\tilde{b}} \quad (*)$$

with U upper triangular matrix. This is done in $n - 1$ steps where, at each step, one column is eliminated, that is, all the coefficients of that column below the diagonal are transformed into zeros. The cost for computing U is of the order of $\frac{2}{3}n^3$, see later... the cost for computing $\tilde{b}$ is of the order of n^2 ; the cost for solving the upper triangular system is n^2 , so that **the total cost is $\sim \frac{2}{3}n^3 + 2n^2 \sim \frac{2}{3}n^3$.**

GEM algorithm

$$A^{(1)} := A, \quad b^{(1)} := b$$

$$A^{(1)}x = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} \\ a_{21}^{(1)} & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1}^{(1)} & a_{n2}^{(1)} & \cdots & a_{nn}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ \vdots \\ b_n^{(1)} \end{bmatrix} = b^{(1)}$$

- We eliminate the first column:

$$A^{(2)}x = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(2)} \\ \vdots \\ b_n^{(2)} \end{bmatrix} = b^{(2)}$$

$$l_{i1} = a_{i1}^{(1)} / a_{11}^{(1)}, \quad a_{ij}^{(2)} = a_{ij}^{(1)} - l_{i1} a_{1j}^{(1)}, \quad b_i^{(2)} = b_i^{(1)} - l_{i1} b_1^{(1)}$$

GEM algorithm

- We go on eliminating columns in the same way. At the k -th step:

$$A^{(k)}x = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & \cdots & \cdots & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & & & & a_{2n}^{(2)} \\ \vdots & & \ddots & & & \vdots \\ 0 & \cdots & 0 & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ \vdots & & \vdots & \vdots & & \vdots \\ 0 & \cdots & 0 & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(2)} \\ \vdots \\ b_k^{(k)} \\ \vdots \\ b_n^{(k)} \end{bmatrix} = b^{(k)}$$

- At the last step $A^{(n)}x = b^{(n)}$ with U upper triangular,

$$\implies U := A^{(n)}, \tilde{b} := b^{(n)}$$

GEM pseudocode

Input: $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$

for $k = 1, \dots, n - 1$

for $i = k + 1, \dots, n$

$$l_{i,k} = a_{i,k} / a_{k,k}$$

$$a_{i,k:n} = a_{i,k:n} - l_{i,k} a_{k,k:n}$$

$$b_i = b_i - l_{i,k} b_k$$

end

end

define $U = A$, $\tilde{b} = b$, then solve $Ux = \tilde{b}$ with back substitution

remark: we do not need to define U and $\tilde{b}$, it is just to be consistent with the notation of the previous slides

Homework

Write the above algorithm in MATLAB

Computational cost of GEM

Let us start by analysing the inner loop:

for $i = k + 1, \dots, n$

$$l_{i,k} = a_{i,k} / a_{k,k}$$

1 FLOP

$$a_{i,k:n} = a_{i,k:n} - l_{i,k} a_{k,k:n}$$

$2(n - k + 1) \sim 2(n - k)$ FLOPs

$$b_i = b_i - l_{i,k} b_k$$

2 FLOPs

end

Inner loop cost $\sim 2(n - k)^2$ FLOPs

Let us focus on the outer loop:

- For $k = 1$, the inner loop costs $\sim 2(n - 1)(n - 1) \sim 2(n - 1)^2$ FLOPs
- For $k = 2$, the inner loop costs $\sim 2(n - 2)(n - 2) \sim 2(n - 2)^2$ FLOPs
- ...
- For $k = n - 1$, the inner loop costs $\sim 2 \sim 2 \cdot (1)^2$ FLOPs

Finally, recalling that:

$$\sum_{t=1}^T t^2 = \frac{T \cdot (T + 1) \cdot (2T + 1)}{6},$$

the cost of GEM is:

$$\sim 2 \cdot \sum_{s=1}^{n-1} s^2 = 2 \frac{(n-1) \cdot n \cdot (2(n-1) + 1)}{6} \sim \frac{2}{3} n^3 \text{ FLOPs}$$

If we also include the cost for solving the resulting upper triangular system

$U\underline{x} = \underline{\tilde{b}}$, the total cost is $\sim \frac{2}{3} n^3 + n^2 \sim \frac{2}{3} n^3 \text{ FLOPs}$.

LU factorization

LU factorisation, consists in looking for two matrices L lower triangular, and U upper triangular, both non-singular, such that

$$LU = A \quad (1)$$

If we find these matrices, the original system $A\underline{x} = \underline{b}$ splits into two triangular systems easy to solve:

$$A\underline{x} = \underline{b} \rightarrow L(U\underline{x}) = \underline{b} \rightarrow \begin{cases} L\underline{y} = \underline{b} & \text{solved forward} \\ U\underline{x} = \underline{y} & \text{solved backward} \end{cases}$$

Mathematically equivalent to GEM: matrix U is the same, $\tilde{b} = L^{-1}b$.

LU: unicity of the factors

The factorization $LU = A$ is unique?

$$\underbrace{\begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{bmatrix}}_L \underbrace{\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & & \vdots \\ 0 & & \cdots & u_{nn} \end{bmatrix}}_U = \underbrace{\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}}_A$$

The unknowns are the coefficients l_{ij} of L , which are

$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$, and the coefficients u_{ij} of U , also $\frac{n(n+1)}{2}$, for a total of $n^2 + n$ unknowns.

We only have n^2 equations (as many as the number of coefficients of A), so we need to fix n unknowns. Usually, the diagonal coefficients of L are set equal to 1: $l_{ii} = 1$. If you do so...

How to compute L and U

Let us introduce Atomic Lower Triangular matrix $L^{(k)}$ defined as

$$L^{(k)} = \begin{bmatrix} 1 & & & & & 0 \\ & \ddots & & & & \\ & & 1 & & & \\ & & -l_{k+1,k} & \ddots & & \\ & & \vdots & & \ddots & \\ 0 & & \underbrace{-l_{n,k}}_{\text{k-th column}} & & & 1 \end{bmatrix}$$

The basic version of GEM can be rewritten in terms of matrix multiplications as:

$$A^{(1)} = A, \quad A^{(2)} = L^{(1)}A, \quad \dots, \quad A^{(n)} = L^{(n-1)}L^{(n-2)} \dots L^{(1)}A = \textcolor{red}{U},$$

where $\textcolor{red}{U}$ is upper triangular.

We observe that

$$L^{(k)-1} = \begin{bmatrix} 1 & & & & 0 \\ & \ddots & & & \\ & & 1 & & \\ & & l_{k+1,k} & \ddots & \\ & & \vdots & & \\ 0 & & \underbrace{l_{n,k}}_{\text{k-th column}} & & 1 \end{bmatrix}, \text{ is still lower triangular}$$

Let us define L as:

$$L := \left(L^{(n-1)} \dots L^{(1)} \right)^{-1} = L^{(1)-1} \dots L^{(n)-1}, \text{ that is still lower triangular}$$

and contains the coefficients $l_{i,k}$ computed in GEM:

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ l_{2,1} & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ l_{n,1} & l_{n,2} & \cdots & 1 \end{bmatrix}$$

Recalling that $L^{(n-1)} L^{(n-2)} \dots L^{(1)} A = U$, we obtain $A = LU$.

GEM vs LU (pseudocodes)

GEM

Input: $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$

for $k = 1, \dots, n-1$

for $i = k+1, \dots, n$

$$l_{i,k} = a_{i,k} / a_{k,k}$$

$$a_{i,k:n} = a_{i,k:n} - l_{i,k} a_{k,k:n}$$

$$b_i = b_i - l_{i,k} b_k$$

end

end

set $U = A$, then solve $Ux = b$ with back substitution

- we can store the $l_{i,k}$ in a matrix:

$$L = \begin{bmatrix} ? & ? & \cdots & ? \\ l_{2,1} & ? & \cdots & ? \\ \vdots & \vdots & & \vdots \\ l_{n,1} & l_{n,2} & \cdots & ? \end{bmatrix}$$

that can be completed as a lower triangular matrix

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ l_{2,1} & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ l_{n,1} & l_{n,2} & \cdots & 1 \end{bmatrix}$$

- the loops on the left replace $\underline{b}$ by $L^{-1}\underline{b}$ (see the “equivalent forward substitution” algorithms): $\underline{b} \leftarrow L^{-1}\underline{b}$
- similarly $A \leftarrow L^{-1}A$, which is called U and is an upper triangular matrix:

$$L^{-1}A = U \Rightarrow A = LU$$

GEM vs LU (pseudocodes)

GEM

Input: $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$

for $k = 1, \dots, n - 1$

for $i = k + 1, \dots, n$

$$l_{i,k} = a_{i,k} / a_{k,k}$$

$$a_{i,k:n} = a_{i,k:n} - l_{i,k} a_{k,k:n}$$

$$b_i = b_i - l_{i,k} b_k$$

end

end

set $U = A$, then solve $Ux = b$ with backward substitution

LU

Input: $A \in \mathbb{R}^{n \times n}$

$$L = I_n \in \mathbb{R}^{n \times n}$$

for $k = 1, \dots, n - 1$

for $i = k + 1, \dots, n$

$$l_{i,k} = a_{i,k} / a_{k,k}$$

$$a_{i,k:n} = a_{i,k:n} - l_{i,k} a_{k,k:n}$$

end

end

Set $U = A$ and output: U and L

Homework

Write the LU algorithm in MATLAB

Permutation matrices

$P \in \mathbb{R}^{n \times n}$ is a permutation matrix if it has only one entry 1 on each row and each column, while the remaining entries are all 0. P produces permutation of rows when multiplying on the left and of columns when multiplying on the right. For example:

$$P = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad A = \begin{bmatrix} \text{---}r_1\text{---} \\ \text{---}r_2\text{---} \\ \text{---}r_3\text{---} \\ \text{---}r_4\text{---} \end{bmatrix} = \begin{bmatrix} | & | & | & | \\ c_1 & c_2 & c_3 & c_4 \\ | & | & | & | \end{bmatrix}$$

$$PA = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \text{---}r_1\text{---} \\ \text{---}r_2\text{---} \\ \text{---}r_3\text{---} \\ \text{---}r_4\text{---} \end{bmatrix} = \begin{bmatrix} \text{---}r_3\text{---} \\ \text{---}r_1\text{---} \\ \text{---}r_2\text{---} \\ \text{---}r_4\text{---} \end{bmatrix}$$

$$AP = \begin{bmatrix} | & | & | & | \\ c_1 & c_2 & c_3 & c_4 \\ | & | & | & | \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} | & | & | & | \\ c_2 & c_3 & c_1 & c_4 \\ | & | & | & | \end{bmatrix}$$

Remark 1

The product of permutation matrices is still a permutation matrix

GEM: possible troubles and remedy

The condition $\det(A) \neq 0$ is not sufficient to guarantee that the elimination procedure will be successful. For example $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

To avoid this the remedy is the “**pivoting**” algorithm:

- **first step**: before eliminating the first column, look for the coefficient of the column biggest in absolute value, the so-called “pivot”; if r is the row where the pivot is found, exchange the first and the r^{th} row.
- **second step**: before eliminating the second column, look for the coefficient of the column biggest in absolute value, starting from the second row; if r is the row where the pivot is found, exchange the second and the r^{th} row.
- $\vdots$
- **step j** : before eliminating the column j , look for the pivot in this column, from the diagonal coefficient down to the last row. If the pivot is found in the row r , exchange the rows j and r .

This is the pivoting procedure on the rows, which amounts to multiply at the left the matrix A by a permutation matrix P . (An analogous procedure can be applied on the columns, or globally).

Lemma 1

Let $A \in \mathbb{R}^{n \times n}$ be a non singular matrix. Then, at each step of GEM, the “pivot” is not null.

Remark 2

If A is non singular, Lemma 1 ensures that GEM with pivoting can be successfully completed.

The pivoting procedure corresponds then to solve, instead of the original system $A\underline{x} = \underline{b}$, the system

$$P\underline{A}\underline{x} = P\underline{b} \quad (2)$$

GEM pseudocode with pivoting

Input as before: $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$

for $k = 1, \dots, n - 1$

 select $j \geq k$ that maximise $|a_{jk}|$

$a_{j,k:n} \longleftrightarrow a_{k,k:n}$

$b_j \longleftrightarrow b_k$

for $i = k + 1, \dots, n$

$l_{i,k} = a_{i,k} / a_{k,k}$

$a_{i,k:n} = a_{i,k:n} - l_{i,k} a_{k,k:n}$

$b_i = b_i - l_{i,k} b_k$

end

end

define $U = A$, $\tilde{b} = b$, then solve $Ux = \tilde{b}$ with backward substitution

remark: we do not need to define U and $\tilde{b}$, it is just to be consistent with the notation of the previous slides

LU-continued

GEM and LU have the same computational cost: $\frac{2}{3}n^3$

In GEM, the coefficients l_{ik} are discarded after application to the right-hand side b , while in the LU factorisation they are stored in the matrix L .

If we have to solve a single linear system, GEM is preferable (less memory storage).

If we have to solve many systems with the same matrix and different right-hand sides LU is preferable (the heavy cost is paid only once).

Pivoting is applied also to the LU factorization to ensure that the factorisation is succesful

$$PA = LU \implies PA\underline{x} = P\underline{b} \rightarrow L(U\underline{x}) = P\underline{b} \rightarrow \begin{cases} L\underline{y} = P\underline{b} \\ U\underline{x} = \underline{y} \end{cases}$$

The Matlab function that computes L and U is $lu(.,.)$.

How to compute L , U and P

GEM with pivoting can be rewritten in terms of matrix multiplications as:

$$\begin{aligned} A^{(1)} &= A, & A^{(2)} &= L^{(1)} P^{(1)} A, & \dots, \\ A^{(n)} &= L^{(n-1)} P^{(n-1)} L^{(n-2)} \dots L^{(1)} P^{(1)} A = U, \end{aligned}$$

where U is upper triangular.

Given two generic matrices M, N , it holds $MN \neq NM$, but if $P^{(j)}$ is a permutation matrix that switches i -th and j -th rows, then, for $i \geq j > k$:

$$P^{(j)} L^{(k)} = \tilde{L}^{(k)} P^{(j)},$$

where $\tilde{L}^{(k)}$ is obtained from $L^{(k)}$ by switching $L_{i,k}^{(k)}$ and $L_{j,k}^{(k)}$.

$\tilde{L}^{(k)}$ is still atomic lower triangular, thus $\tilde{L}^{(k)-1}$ is obtained from $\tilde{L}^{(k)}$ by changing signs of the off-diagonal coefficients.

$$\begin{aligned}
U &= L^{(n-1)} P^{(n-1)} L^{(n-2)} P^{(n-2)} L^{(n-3)} \dots L^{(1)} P^{(1)} A \\
&= L^{(n-1)} \tilde{L}^{(n-2)} P^{(n-1)} P^{(n-2)} L^{(n-3)} P^{(n-3)} \dots L^{(1)} P^{(1)} A \\
&= L^{(n-1)} \tilde{L}^{(n-2)} \tilde{\tilde{L}}^{(n-3)} P^{(n-1)} P^{(n-2)} P^{(n-3)} \dots L^{(1)} P^{(1)} A \\
&= \left(L^{(n-1)} \tilde{L}^{(n-2)} \tilde{\tilde{L}}^{(n-3)} \tilde{\tilde{\tilde{L}}}^{(n-4)} \dots \right) P^{(n-1)} P^{(n-2)} P^{(n-3)} \dots P^{(1)} A
\end{aligned}$$

Defining

$$\begin{aligned}
L &= \left(L^{(n-1)} \tilde{L}^{(n-2)} \tilde{\tilde{L}}^{(n-3)} \tilde{\tilde{\tilde{L}}}^{(n-4)} \dots \right)^{-1} \\
P &= P^{(n-1)} P^{(n-2)} P^{(n-3)} \dots P^{(1)}
\end{aligned}$$

we get

$$LU = PA.$$

LU with pivoting (pseudocode)

Input as before: $A \in \mathbb{R}^{n \times n}$

$L = I_n \in \mathbb{R}^{n \times n}$

$P = I_n \in \mathbb{R}^{n \times n}$

for $k = 1, \dots, n - 1$

 select $j \geq k$ that maximise $|a_{jk}|$

$a_{j,k:n} \longleftrightarrow a_{k,k:n}$

$p_{j,:} \longleftrightarrow p_{k,:}$

if $k \geq 2$

$l_{j,1:k-1} \longleftrightarrow l_{k,1:k-1}$

end

for $i = k + 1, \dots, n$

$l_{i,k} = a_{i,k} / a_{k,k}$

$a_{i,k:n} = a_{i,k:n} - l_{i,k} a_{k,k:n}$

end

end

define $U = A$ and output: U , L and P

LU versus GEM

If one needs to compute the inverse of a matrix, LU is the cheapest way. Indeed, recalling the definition, the inverse of a matrix A is the matrix A^{-1} solution of

$$AA^{-1} = I$$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & & \vdots \\ \underbrace{c_{n1}}_{\underline{c}^{(1)}} & \underbrace{c_{n2}}_{\underline{c}^{(2)}} & \cdots & \underbrace{c_{nn}}_{\underline{c}^{(n)}} \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ \underbrace{0}_{\underline{e}^1} & \underbrace{0}_{\underline{e}^2} & \cdots & \underbrace{1}_{\underline{e}^n} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & & \vdots \\ \underbrace{c_{n1}}_{\underline{c}^{(1)}} & \underbrace{c_{n2}}_{\underline{c}^{(2)}} & \cdots & \underbrace{c_{nn}}_{\underline{c}^{(n)}} \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ \underbrace{0}_{\underline{e}^1} & \underbrace{0}_{\underline{e}^2} & \cdots & \underbrace{1}_{\underline{e}^n} \end{bmatrix}$$

Hence, each column $\underline{c}^{(i)}$ of A^{-1} is the solution of

$$A\underline{c}^{(i)} = \underline{e}^{(i)}, \quad i = 1, 2, \dots, n$$

with $\underline{e}^{(i)} = (0, 0, \dots, 1, \dots, 0)$. The factorisation can be done once and for all at the cost of $O(2n^3/3)$ operations; for each column we have to solve 2 triangular systems ($2n^2$ operations) so that the total cost is of the order of $\frac{2}{3}n^3 + n \times 2n^2 = \frac{8}{3}n^3$.

In case of pivoting, we solve $PA\underline{c}^{(i)} = P\underline{e}^{(i)} = P_{:,i}$, $i = 1, 2, \dots, n$.

Computation of the determinant

We can use the LU factorisation to compute the determinant of a matrix. Indeed, if $A = LU$, thanks to the Binet theorem we have

$$\det(A) = \det(L)\det(U) = \prod_{i=1}^n l_{ii} \prod_{i=1}^n u_{ii} = \prod_{i=1}^n u_{ii}$$

Thus the cost to compute the determinant is the same of the LU factorisation.

In the case of pivoting, $PA = LU$ and then

$$\det(A) = \frac{\det(L)\det(U)}{\det(P)} = \frac{\det(U)}{\det(P)}$$

It turns out that $\det(P) = (-1)^\delta$ where $\delta = \#$ of row exchanges in the LU factorisation.

Matlab function: `det(·)`

Cholesky factorization

If A is symmetric ($A = A^T$) and positive definite (positive eigenvalues) a variant of LU is due to Cholesky: there exists a non-singular lower triangular matrix L such that

$$LL^T = A$$

Costs: approximately $\sim \frac{n^3}{3}$ (half the cost of LU, using the symmetry of A).

Matlab function: `chol(..)`

Iterative methods

Consider the linear system

$$A\underline{x} = \underline{b}$$

Iterative methods start from an initial guess $\underline{x}^{(0)}$ and construct a sequence of approximate solutions $\{\underline{x}^{(k)}\}$ such that

$$\underline{x} = \lim_{k \rightarrow \infty} \underline{x}^{(k)}.$$

Splitting methods

The matrix A is split as

$$A = M - N$$

Splitting methods go like

$$\underline{x}^{(0)} \text{ given} \quad \text{solve} \quad M\underline{x}^{(k)} = \underline{b} + N\underline{x}^{(k-1)} \quad k = 1, 2, \dots \quad (1)$$

With iterative methods we give up the idea of computing the exact solution, but we want low operational costs. In particular:

- the system (1) must be much easier to deal with than the original system $Ax = b$, that is, the matrix M must be as simple as possible, and of course non-singular;
- the sequence $\{\underline{x}^{(k)}\}$ must converge to $\underline{x}$ for any initial guess $\underline{x}^{(0)}$;
- the convergence must be fast.

Different choices for M give rise to different iterative methods.

Jacobi method

take $M = \text{diag}(A)$ (and hence $N = M - A$), applicable if $a_{ii} \neq 0 \forall i$. At each iteration k we have to solve a diagonal system

$$\begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ 0 & \cdots & \ddots & 0 \\ 0 & \cdots & & a_{nn} \end{pmatrix} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} - \begin{pmatrix} 0 & a_{12} & \cdots & a_{1n} \\ a_{21} & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & 0 \end{pmatrix} \begin{pmatrix} x_1^{(k-1)} \\ x_2^{(k-1)} \\ \vdots \\ x_n^{(k-1)} \end{pmatrix}$$

Thus we obtain

$$x_i^{(k)} = \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k-1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k-1)} \right) / a_{ii} \quad i = 1, \dots, n$$

The number of operations for each component is $\sim 2n$, so that the **cost** for one Jacobi iteration is $\sim 2n^2$.

Gauss-Seidel method

take $M = \text{tril}(A)$, applicable if $a_{ii} \neq 0 \ \forall i$. At each iteration k we have to solve a lower triangular system

$$\begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ a_{21} & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & \cdots & & a_{nn} \end{pmatrix} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \\ \vdots \\ x_n^{(k)} \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} - \begin{pmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix} \begin{pmatrix} x_1^{(k-1)} \\ x_2^{(k-1)} \\ \vdots \\ x_n^{(k-1)} \end{pmatrix}$$

Thus we obtain

$$x_i^{(k)} = \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k)} - \sum_{j=i+1}^n a_{ij} x_j^{(k-1)} \right) / a_{ii} \quad i = 1, \dots, n$$

The difference with respect to Jacobi method is in the first sum of the formula, where the updated $x_j^{(k)}$ are used instead of the old $x_j^{(k-1)}$. The number of operations is exactly the same: for each component is $\sim 2n$, so that the **cost for one Gauss-Seidel iteration is $\sim 2n^2$** .

Convergence analysis for splitting methods

In all cases we want convergence for any initial guess $\underline{x}^{(0)}$. With paper and pencil we study the error at each iteration.

Let $e^{(k)} = \underline{x} - \underline{x}^{(k)}$ be the error at the k^{th} iteration.

Since $\underline{x}$ and $\underline{x}^{(k)}$ are solutions of

$$M\underline{x} = \underline{b} + N\underline{x}, \quad M\underline{x}^{(k)} = \underline{b} + N\underline{x}^{(k-1)},$$

by subtracting we get

$$M(\underline{x} - \underline{x}^{(k)}) = N(\underline{x} - \underline{x}^{(k-1)}) \implies e^{(k)} = \underbrace{M^{-1}N}_B e^{(k-1)}$$

where $B = M^{-1}N$ is the iteration matrix.

$$e^{(k)} = B e^{(k-1)} \quad k = 1, 2, \dots, \implies e^{(k)} = B^k e^{(0)}.$$

If we want $\lim_{k \rightarrow \infty} e^{(k)} = 0$ we need $\lim_{k \rightarrow \infty} B^k = 0$.

Convergent matrices

A matrix $B \in \mathbb{R}^{n \times n}$ is convergent if

$$\lim_{k \rightarrow \infty} B^k = 0,$$

where 0 is the matrix identically zero. Then:

Lemma 1

Let $B \in \mathbb{R}^{n \times n}$. We have

$$\lim_{k \rightarrow \infty} B^k = 0 \iff \max_i |\lambda_i(B)| < 1.$$

The proof is not trivial for a generic B .

A useful property of natural norm of matrices

Lemma 2

Let $|||A|||$ be any natural norm of matrix. Then

$$\max_i |\lambda_i(A)| \leq |||A||| \quad \forall A \in \mathbb{R}^{n \times n}.$$

Proof.

Let λ be an eigenvalue of A , and let $\underline{v} \neq 0$ an eigenvector associated to λ , that is $A\underline{v} = \lambda\underline{v}$. From the properties of the norms we immediately have

$$|\lambda| ||\underline{v}|| = ||\lambda\underline{v}|| = ||A\underline{v}|| \leq |||A||| ||\underline{v}||,$$

then $|\lambda| ||\underline{v}|| \leq |||A||| ||\underline{v}||$, and then $|\lambda| \leq |||A|||$. □

The quantity $\max_i |\lambda_i(A)|$ is called the spectral radius of A , and denoted as $\rho(A)$.

The matrix $||| \cdot |||_\infty$ norm

Lemma 3

Given $B \in \mathbb{R}^{n \times n}$, the natural norm $|||B|||_\infty := \sup_{v \in \mathbb{R}^n} \frac{\|Bv\|_\infty}{\|v\|_\infty}$ can be rewritten as

$$|||B|||_\infty = \max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}|$$

Proof.

Let us start by proving that $|||B|||_\infty \leq \max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}|$. It holds:

$$\begin{aligned} \|Bv\|_\infty &= \max_{i=1,\dots,n} |(Bv)_i| = \max_{i=1,\dots,n} \left| \sum_{j=1}^n B_{i,j} v_j \right| \leq \max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j} v_j| \\ &\leq \max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}| |v_j| \leq \|v\|_\infty \max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}| \end{aligned}$$

continue ...

Proof.

Therefore, for all $v \in \mathbb{R}^n$, it holds

$$\frac{\|Bv\|_\infty}{\|v\|_\infty} \leq \max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}|$$

and finally

$$\|B\|_\infty = \sup_{v \in \mathbb{R}^n} \frac{\|Bv\|_\infty}{\|v\|_\infty} \leq \max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}|$$

It remains to prove that $\max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}| \leq \|B\|_\infty$. Let $\hat{i}$ be the row index that realizes the maximum and let us define $w \in \mathbb{R}^n$ as $w_j = \text{sign}(B_{\hat{i},j})$. We observe that $\|w\|_\infty = 1$.

continue ...

Proof.

It holds

$$\begin{aligned}\max_{i=1,\dots,n} \sum_{j=1}^n |B_{i,j}| &= \sum_{j=1}^n |B_{\hat{i},j}| = \sum_{j=1}^n B_{\hat{i},j} w_j = \left| \sum_{j=1}^n B_{\hat{i},j} w_j \right| \\ &\leq \max_{i=1,\dots,n} \left| \sum_{j=1}^n B_{i,j} w_j \right| = \|Bw\|_{\infty} = \frac{\|Bw\|_{\infty}}{\|w\|_{\infty}} \\ &\leq \sup_{v \in \mathbb{R}^n} \frac{\|Bv\|_{\infty}}{\|v\|_{\infty}} = \|B\|_{\infty}\end{aligned}$$



Classes of matrices for which we have convergence results

Lemma 4

If A is diagonally dominant, i.e.,

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad \forall i = 1, 2, \dots, n$$

both Jacobi and Gauss-Seidel converge.

Proof.

We shall prove the Lemma only for Jacobi method. The iteration matrix B_J is given by

$$B_J = \begin{bmatrix} 0 & -\frac{a_{12}}{a_{11}} & \cdots & -\frac{a_{1n}}{a_{11}} \\ -\frac{a_{21}}{a_{22}} & 0 & \cdots & -\frac{a_{2n}}{a_{22}} \\ \vdots & \vdots & \ddots & \vdots \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & \cdots & 0 \end{bmatrix}$$

Since A is diagonally dominant, $\|B_J\|_\infty = \max_i \sum_{j \neq i} \left| \frac{a_{ij}}{a_{ii}} \right| < 1$, and we deduce (from Lemma 1) that $\max_i |\lambda_i(B_J)| < 1$. Then B_J is convergent and Jacobi method converges. □

Lemma 5

If A is symmetric and positive definite Gauss-Seidel converges. Jacobi might or might not converge.

Stopping criteria

As usual with iterative procedures, we need some stopping criteria to decide when to stop. Given a tolerance τ for example $\sim 10^{-3}$, or 10^{-4})

- **test on the iterates:** at each iteration check if

$$\frac{\|\underline{x}^{(k)} - \underline{x}^{(k-1)}\|}{\|\underline{x}^{(k-1)}\|} \leq \tau$$

for some norm of vectors;

- **test on the residual:** when the test on the iterates is satisfied, check if

$$\frac{\|\underline{r}^{(k)}\|}{\|\underline{b}\|} \leq \tau \quad (\underline{r}^{(k)} := \underline{b} - A\underline{x}^{(k)} \text{ is the residual})$$

When both are satisfied, stop and take $\underline{x}^{(k)}$ as solution.

Pseudocode for splitting methods

$$\begin{aligned}M\underline{x}^{(k)} &= \underline{b} + N\underline{x}^{(k-1)} = \underline{b} + (M - A)\underline{x}^{(k-1)} \\&= \underline{b} - A\underline{x}^{(k-1)} + M\underline{x}^{(k-1)} \\&\implies \underline{x}^{(k)} = \underline{x}^{(k-1)} + M^{-1}\underline{r}^{(k-1)}\end{aligned}$$

M is usually referred as a *preconditioner*.

Splitting iterative method

Input: $A \in \mathbb{R}^{n \times n}$, $\underline{b} \in \mathbb{R}^n$, $\underline{x}^{(0)} \in \mathbb{R}^n$, $tol \in \mathbb{R}^+$, $maxiter \in \mathbb{N}$

Choose $M \in \mathbb{R}^{n \times n}$ and set $\underline{r}^{(0)} = \underline{b} - A\underline{x}^{(0)}$

for $k = 1, 2, \dots, maxiter$:

 Solve $M\underline{p}^{(k-1)} = \underline{r}^{(k-1)}$

$\underline{x}^{(k)} = \underline{x}^{(k-1)} + \underline{p}^{(k-1)}$

$\underline{r}^{(k)} = \underline{b} - A\underline{x}^{(k)}$

 If Stopping criteria are satisfied exit the loop

end

Output: $\underline{x}^{(k)}$

Error analysis

Unfortunately, the fact that the residual is small does not guarantee that the error $\underline{x} - \underline{x}^{(k)}$ is small.

$$\underline{r}^{(k)} := \underline{b} - A\underline{x}^{(k)} = A\underline{x} - A\underline{x}^{(k)} \longrightarrow \underline{x} - \underline{x}^{(k)} = A^{-1}\underline{r}^{(k)}.$$

Taking the norm in both sides we have

$$\begin{aligned}\|\underline{x} - \underline{x}^{(k)}\| &= \|A^{-1}\underline{r}^{(k)}\| \leq \|A^{-1}\| \|\underline{r}^{(k)}\| \\ &\leq \|A^{-1}\| \frac{\|\underline{r}^{(k)}\|}{\|\underline{b}\|} \|A\underline{x}\| \leq \|A^{-1}\| \|A\| \|\underline{x}\| \frac{\|\underline{r}^{(k)}\|}{\|\underline{b}\|}.\end{aligned}$$

Then we obtain

$$\frac{\|\underline{x} - \underline{x}^{(k)}\|}{\|\underline{x}\|} \leq \|A^{-1}\| \|A\| \frac{\|\underline{r}^{(k)}\|}{\|\underline{b}\|}.$$

If the number $\kappa(A) := \|A^{-1}\| \|A\|$ is big there is no control on the error, no matter how small the residual is. $\kappa(A)$ is called “condition number of A ”, and if $\kappa(A) \gg 1$ the matrix is said to be ill-conditioned.

Concept of conditioning

When dealing with ill-conditioned matrices, any numerical method (direct or iterative) might produce unsatisfactory results.

Roughly speaking, a problem is *well-conditioned* if “small” perturbations on the data determine “small” perturbations on the results.

To clarify the concept of *conditioning* of a problem, let us consider a generic problem: find u solution of

$$(P) \qquad F(u, d) = 0,$$

where d are the data, and F is the law relating u to d .

Concept of conditioning

More precisely, let u be the solution of the problem (P) $F(u, d) = 0$ corresponding to data d , and let δd be a perturbation on the data. Denote by δu the corresponding perturbation on the solution u . Then, instead of solving (P) we are solving

$$(\tilde{P}) \quad F(u + \delta u, d + \delta d) = 0.$$

Problem (P) is **well-posed** (or, the solution of the problem depends continuously on the data) if

$$\forall \varepsilon > 0, \quad \exists \delta > 0 \text{ such that if } \|\delta d\| \leq \delta, \text{ then } \|\delta u\| \leq \varepsilon$$

for some norm.

If a problem is well-posed, its (relative) **condition number** is the smallest constant $\kappa > 0$ that satisfies

$$\frac{\|\delta u\|}{\|u\|} \leq \kappa \frac{\|\delta d\|}{\|d\|}$$

Example

Back to linear systems, let us make a conditioning analysis in a simple case. Assume that the possible errors are only on the right-hand side (and not on the matrix). Let $\delta \underline{b}$ be the errors on $\underline{b}$, small (in percentage), that is, for ε small,

$$\frac{\|\delta \underline{b}\|}{\|\underline{b}\|} \leq \varepsilon$$

The solution of the system with matrix A and right-hand side $\tilde{\underline{b}} = \underline{b} + \delta \underline{b}$ will be $\tilde{\underline{x}} = \underline{x} + \delta \underline{x}$:

$$A\underline{x} = \underline{b} \rightarrow A(\underline{x} + \delta \underline{x}) = \underline{b} + \delta \underline{b},$$

subtracting

$$A\delta \underline{x} = \delta \underline{b}, \quad \text{giving} \quad \delta \underline{x} = A^{-1}\delta \underline{b}.$$

Proceeding as we did before we have

$$\begin{aligned} \|\delta \underline{x}\| &= \|A^{-1}\delta \underline{b}\| \leq \|A^{-1}\| \|\delta \underline{b}\| = \|A^{-1}\| \frac{\|\delta \underline{b}\|}{\|\underline{b}\|} \|\underline{b}\| \\ &= \|A^{-1}\| \frac{\|\delta \underline{b}\|}{\|\underline{b}\|} \|A\underline{x}\| \leq \|A^{-1}\| \frac{\|\delta \underline{b}\|}{\|\underline{b}\|} \|A\| \|\underline{x}\| \end{aligned}$$

We found

$$\begin{aligned}\|\delta \underline{x}\| &\leq \|A^{-1}\| \frac{\|\delta \underline{b}\|}{\|\underline{b}\|} \|A\| \|\underline{x}\| \\ \Rightarrow \frac{\|\delta \underline{x}\|}{\|\underline{x}\|} &\leq \|A^{-1}\| \|A\| \varepsilon = \kappa(A) \varepsilon\end{aligned}$$

A simple example to understand how a big condition number might affect the results.

$$\begin{pmatrix} 10^6 & 10^{-12} \\ 0 & 10^{-6} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 10^6 \\ 10^{-6} \end{pmatrix}$$

Exact solution $x_2 = 1, x_1 \simeq 1$; $\kappa_\infty(A) \simeq 10^{12}$. Now perturb only the first component of the right-hand side by 10^{-6} , and then only the second component by 10^{-6} . In both cases $\|\delta \underline{b}\|_\infty / \|\underline{b}\|_\infty \leq 10^{-12}$. What happens to the solution?

An approach for symmetric positive definite matrices

We now assume that the system matrix is symmetric and positive definite (SPD), and discuss a different iterative approach.

Recall the problem we want to solve: given $\underline{b} \in \mathbb{R}^n$, and $A \in \mathbb{R}^n \times \mathbb{R}^n$, we look for $\underline{x}^* \in \mathbb{R}^n$ solution of

$$A\underline{x}^* = \underline{b} \quad (1)$$

Since A is SPD, we can define a scalar product associated with A : $(A\underline{x}, \underline{y}) = \underline{y}^T A\underline{x}$. If A is also positive definite, then

$$(A\underline{x}, \underline{x}) > 0 \quad \forall \underline{x} \neq \underline{0}.$$

Then we can introduce the functional $F : \mathbb{R}^n \rightarrow \mathbb{R}$ defined as:

$$F(\underline{v}) := \frac{1}{2}(A\underline{v}, \underline{v}) - (\underline{b}, \underline{v}) \quad \forall \underline{v} \in \mathbb{R}^n \quad (2)$$

Theorem 1

If $A \in \mathbb{R}^n \times \mathbb{R}^n$ is SPD, problem (1) has a unique solution, and is equivalent to the following minimum problem for the functional defined in (2):

$$\begin{cases} \text{find } \underline{u} \in \mathbb{R}^n \text{ such that} \\ F(\underline{u}) \leq F(\underline{v}) \quad \forall \underline{v} \in \mathbb{R}^n \end{cases} \quad (3)$$

(that is, (3) has a unique solution $\underline{u} \in \mathbb{R}^n$, and $\underline{u} \equiv \underline{x}^$).*

Proof.

Since A is positive definite, problem (1) has a unique solution ($\det(A) \neq 0$). Now, F is a quadratic functional (hence, differentiable), and

$$\underline{\nabla} F(\underline{v}) = \begin{bmatrix} \frac{\partial F}{\partial v_1} \\ \frac{\partial F}{\partial v_2} \\ \vdots \\ \frac{\partial F}{\partial v_n} \end{bmatrix} = A\underline{v} - \underline{b} \quad H(F) = A \quad (H(F) = \text{Hessian matrix})$$

Since A is positive definite, the matrix $H(F)$ has positive eigenvalues (and real because A is symmetric). Hence, F is strictly convex, that is, it has a unique minimum. Let $\underline{u} \in \mathbb{R}^n$ be the point of minimum. As such, it verifies

$$\underline{\nabla} F(\underline{u}) = \underline{0} \quad \longrightarrow \quad A\underline{u} - \underline{b} = \underline{0}.$$

Since the solution of (1) is unique, $\underline{u} \equiv \underline{x}^*$.



Descent Methods

Given the equivalence between the linear system (1) and the minimum problem (3), we look for $\underline{x}^*$ as minimum point for $F(\underline{x})$.

Starting from an initial guess $\underline{x}^{(0)}$ (any), we want to construct a sequence $\underline{x}^{(k)}$ converging to $\underline{x}^*$ in the following way:

$\underline{x}^{(0)}$ given. Then, for $k = 1, 2, \dots$ set $\underline{x}^{(k+1)} = \underline{x}^{(k)} + \alpha_k \underline{p}^{(k)}$

- $\underline{p}^{(k)}$ are directions of descent,
- α_k are numbers that tell us how much to descent along $\underline{p}^{(k)}$.

They have to be chosen to guarantee descent, that is, to guarantee that

$$F(\underline{x}^{(k+1)}) < F(\underline{x}^{(k)}) \quad \forall k.$$

Descent methods

The optimal value of α_k can be computed by imposing

$$\frac{\partial}{\partial \alpha} F(x^{(k)} + \alpha p^{(k)}) = 0$$

which guarantees maximum descent of F along the descent direction $p^{(k)}$. Indeed,

$$\begin{aligned} F(x^{(k)} + \alpha p^{(k)}) &= \frac{1}{2} \left(A(x^{(k)} + \alpha p^{(k)}), x^{(k)} + \alpha p^{(k)} \right) - \left(b, x^{(k)} + \alpha p^{(k)} \right) \\ &= \frac{\alpha^2}{2} \left(A p^{(k)}, p^{(k)} \right) + \alpha \left(A x^{(k)} - b, p^{(k)} \right) + \left(\frac{1}{2} A x^{(k)} - b, x^{(k)} \right) \end{aligned}$$

With respect to the variable α , this function is an U-shaped parabola (it has a unique minimum).

$$\frac{\partial}{\partial \alpha} F(x^{(k)} + \alpha p^{(k)}) = \alpha \left(A p^{(k)}, p^{(k)} \right) + \left(A x^{(k)} - b, p^{(k)} \right) = 0$$

$$\alpha_k = \text{optimal } \alpha = \frac{(\underline{b} - A \underline{x}^{(k)}, \underline{p}^{(k)})}{(A \underline{p}^{(k)}, \underline{p}^{(k)})} = \frac{(\underline{r}^{(k)}, \underline{p}^{(k)})}{(A \underline{p}^{(k)}, \underline{p}^{(k)})}$$

Steepest Descent Method (or Gradient Method)

From the Taylor expansion (for $\|v\| \ll 1$)

$$F(x + v) = F(x) + v^T \nabla F(x) + O(\|v\|^2)$$

Thus $F(x + v) \leq F(x)$ if $v = -\alpha \nabla F(\underline{x})$ ($\alpha > 0$, small enough). This motivates the choice

$$\underline{p}^{(k)} = -\underline{\nabla} F(\underline{x}^{(k)}) = b - A\underline{x}^{(k)} = \underline{r}^{(k)}$$

Pseudocode for Steepest Descent Method

Steepest Descent Method

Input: $A \in \mathbb{R}^{n \times n}$ SPD, $\underline{b} \in \mathbb{R}^n$, $\underline{x}^{(0)} \in \mathbb{R}^n$, $tol \in \mathbb{R}^+$, $maxiter \in \mathbb{N}$

$$\underline{r}^{(0)} = \underline{b} - A\underline{x}^{(0)}$$

for $k = 1, 2, \dots, maxiter$:

$$\underline{y} = A\underline{r}^{(k-1)}$$

$$\alpha_{k-1} = (\underline{r}^{(k-1)}, \underline{r}^{(k-1)}) / (\underline{y}, \underline{r}^{(k-1)})$$

$$\underline{x}^{(k)} = \underline{x}^{(k-1)} + \alpha_{k-1}\underline{r}^{(k-1)}$$

$$\underline{r}^{(k)} = \underline{b} - A\underline{x}^{(k)} = \underline{r}^{(k-1)} - \alpha_{k-1}\underline{y}$$

If Stopping criteria are satisfied exit the loop

end

Output: $\underline{x}^{(k)}$

Like for all iterative methods, the dominant computational cost is given by the matrix-vector product with A . Hence the cost is roughly $2n^2$ FLOPs per iteration.

Convergence of Steepest Descent Method

Theorem 2

*The Steepest Descent method converges for all initial guess $\underline{x}^{(0)}$.
Moreover it holds:*

$$\left\| \underline{x} - \underline{x}^{(k)} \right\|_A \leq \left(\frac{\kappa_2(A) - 1}{\kappa_2(A) + 1} \right)^k \left\| \underline{x} - \underline{x}^{(0)} \right\|_A$$

where $\|v\|_A = \sqrt{v^T A v}$ is the A -norm.

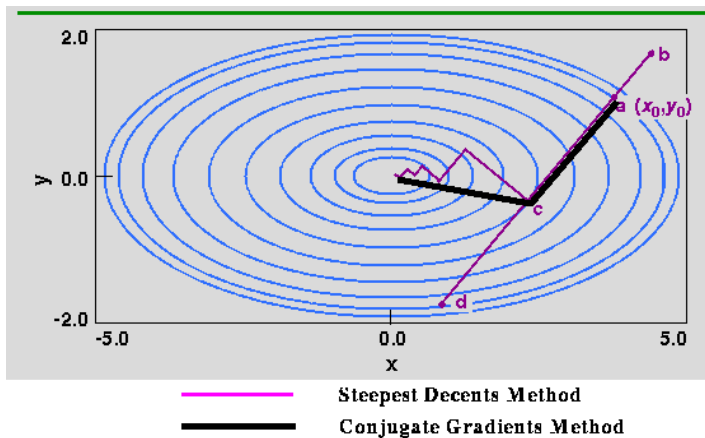
Convergence is guaranteed, but can be very slow if A is ill-conditioned.

Summary and extensions of gradient methods...

We have our functional $F(\underline{v}) := \frac{1}{2}(A\underline{v}, \underline{v}) - (\underline{b}, \underline{v})$ to minimize and use $\underline{x}^{(k+1)} = \underline{x}^{(k)} + \alpha_k \underline{p}^{(k)}$, where $\underline{p}^{(k)} = -\underline{\nabla} F(\underline{x}^{(k)}) = \underline{b} - A\underline{x}^{(k)}$. Possible alternatives are:

- to simplify the calculation of $\underline{p}^{(k)} = -\underline{\nabla} F(\underline{x}^{(k)})$, e.g. in the **stochastic gradient descent** method, used in machine learning: we save time per each iteration at the expenses of an increased number of iterations to reach a given accuracy;
- to find better descent directions $\underline{p}^{(k)}$, such that the convergence at a given tolerance requires less iterations, as in the **conjugate gradient** method

Steepest descents vs. Conjugate Gradient



Conjugate Gradient method

with $\underline{p}^{(0)} = -\underline{\nabla}F(\underline{x}^{(0)})$, at each iteration k take $\underline{p}^{(k)}$ in the plane $\text{span}\{\underline{r}^{(k)}, \underline{p}^{(k-1)}\}$, that is:

$$\underline{p}^{(k)} = \underline{r}^{(k)} - \beta_k \underline{p}^{(k-1)}$$

where β_k is chosen so that $\underline{p}^{(k)}$ is A -orthogonal to $\underline{p}^{(k-1)}$, i.e.
 $(\underline{p}^{(k)})^T A \underline{p}^{(k-1)} = 0$ (orthogonal in the scalar product associated with A).
It can be proven that

$$(\underline{p}^{(k)})^T A \underline{p}^{(j)} = 0, \quad j = 1, \dots, k-1.$$

This approach is faster than the steepest descent. Actually, the method converges in less than n iterations (n =dimension of the system), so it can be considered a direct method.

Matlab function: $\mathbf{x} = \text{pcg}(A, b, \dots)$

Pseudocode for Conjugate Gradient Method

Conjugate Gradient Method

Input: $A \in \mathbb{R}^{n \times n}$ SPD, $\underline{b} \in \mathbb{R}^n$, $\underline{x}^{(0)} \in \mathbb{R}^n$, $tol \in \mathbb{R}^+$, $maxiter \in \mathbb{N}$

$$\underline{r}^{(0)} = \underline{b} - A\underline{x}^{(0)}$$

$$\underline{p}^{(0)} = \underline{r}^{(0)}$$

for $k = 1, 2, \dots, maxiter$:

$$\underline{y} = A\underline{p}^{(k-1)}$$

$$\alpha_{k-1} = (\underline{p}^{(k-1)}, \underline{r}^{(k-1)}) / (\underline{y}, \underline{p}^{(k-1)})$$

$$\underline{x}^{(k)} = \underline{x}^{(k-1)} + \alpha_{k-1} \underline{p}^{(k-1)}$$

$$\underline{r}^{(k)} = \underline{b} - A\underline{x}^{(k)} = \underline{r}^{(k-1)} - \alpha_{k-1} \underline{y}$$

$$\beta_k = (\underline{y}, \underline{r}^{(k)}) / (\underline{y}, \underline{p}^{(k-1)})$$

$$\underline{p}^{(k)} = \underline{r}^{(k)} - \beta_k \underline{p}^{(k-1)}$$

If Stopping criteria are satisfied exit the loop

end

Output: $\underline{x}^{(k)}$

Conjugate Gradient Method: Cost and Convergence

Also for the Conjugate Gradient Method the dominant computational cost is given by the matrix-vector product with A . Hence the cost is roughly $2n^2$ FLOPs per iteration.

Theorem 1

The Conjugate Gradient method converges in at most n iterations for all initial guess $\underline{x}^{(0)}$. Moreover it holds:

$$\left\| x - x^{(k)} \right\|_A \leq 2 \left(\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^k \left\| x - x^{(0)} \right\|_A$$

where $\|v\|_A = \sqrt{v^T A v}$ is the A -norm.

Note that when A is SPD it holds

$$\kappa_2(A) = \|A\| \|A^{-1}\| = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$$

It follows that convergence is fast if $\lambda_{\max}(A) \approx \lambda_{\min}(A)$ (i.e. A is well-conditioned), or in other words if the eigenvalues are clustered. On the other hand, it might be slow if $\lambda_{\max}(A) \gg \lambda_{\min}(A)$ (i.e. A is ill-conditioned).

Preconditioners

To speedup the convergence, we can use a preconditioner. In this case, the original linear system $Ax = b$ with the equivalent one

$$P^{-1}Ax = P^{-1}b$$

where P is a nonsingular matrix called preconditioner.

A good preconditioner has two features:

- An iterative method applied to the new system should converge in less iterations than for the original system. This typically means that **the eigenvalues of $P^{-1}A$ should be clustered** (in the case of CG, this means $\lambda_{\max}(P^{-1}A) \approx \lambda_{\min}(P^{-1}A)$).
- At each iteration of an iterative method we need to compute a matrix-vector product with the system matrix. In the preconditioned case, this is done in two steps

$$v \longrightarrow Av \longrightarrow P^{-1}(Av)$$

Thus, **computing matrix-vector products with P^{-1} (or equivalently solving a linear system with P) should be fast.** Note that we never need to compute the matrix $P^{-1}A$ explicitly.

Preconditioners

- Hence a good preconditioner P should be as similar as possible to A , while being easy to invert:
- Let us consider two extreme cases: $P = I_n$ and $P = A$
 - If $P = A$, $P^{-1}A = I_n$ and any iterative method would converge in just 1 iteration (note that $\lambda_{\max}(P^{-1}A) = \lambda_{\min}(P^{-1}A) = 1$). On the other hand, applying P^{-1} is as difficult as solving the original system.
 - If $P = I_n$, then applying P^{-1} has no cost. On the other hand $P^{-1}A = A$, so there is no reduction in the number of iterations.
- A good preconditioner should find a balance between these two extremes.

Preconditioning for CG

- In general, the problem of finding a good preconditioner is very problem-specific.
- Some black-box preconditioners:
 - **Jacobi:** $P = \text{diag}(A)$.
 - **Symmetric Gauss-Seidel:** $P = L_* \text{diag}(A)^{-1} L_*^T$ where $L_* = \text{tril}(A)$.

Sparse Matrices

“A matrix is sparse if many of its coefficients are zero. The interest in sparsity arises because its exploitation can lead to enormous computational savings and because many large matrix problems that occur in practice are sparse.”

- Page 1, Direct Methods for Sparse Matrices, 2nd Edition, 2017.

The **sparsity** of an $n \times n$ matrix A is

$$\frac{\text{nnz}(A)}{n^2}$$

where $\text{nnz}(A) = \#$ of nonzero entries in A . A matrix is **sparse** if its sparsity is $\ll 1$. A matrix that is not sparse is **dense**.

Sparse matrices

- Sparse matrices are extremely common in engineering and computer science. Some examples:
 - Network theory (e.g. social networks).
 - Data analysis and machine learning.
 - Discretization of differential equations.
 - ...
- Sparse matrices represent problems with possibly a large number of variables, but where each variable “interacts” directly with few other variables (local interactions).

Formats of sparse matrices

- To save memory, it is convenient to store only the nonzero entries of a sparse matrix. There are several data structure that allow this.
- Compressed Sparse Column (CSC). The matrix A is specified by three arrays: `val`, `row_ind`, `col_ptr`, `nrows`.
 - `val` stores the nonzero entries of A , ordered from from top to bottom and left to right.
 - `row_ind` stores the row indices of the nonzero entries of A .
 - `col_ptr[i]` stores the index of the element in `val` that begins the first non-empty column starting from the i -th column.
 - `nrows` stores the number of rows.
 - `ncols` stores the number of cols.

This is the format used by Matlab.

- Example:

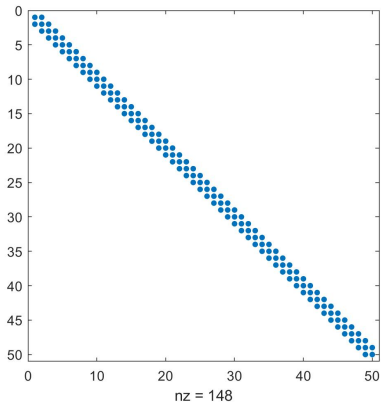
$$A = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 7 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 8 & 2 \end{bmatrix} \quad \begin{array}{lcl} \text{val} & = & [4 \ 7 \ 3 \ 8 \ 2] \\ \text{row_ind} & = & [1 \ 2 \ 3 \ 4 \ 4] \\ \text{col_ptr} & = & [1 \ 3 \ 3 \ 5] \\ \text{nrows} & = & 4, \quad \text{ncols} = 4 \end{array}$$

Sparsity and direct solvers

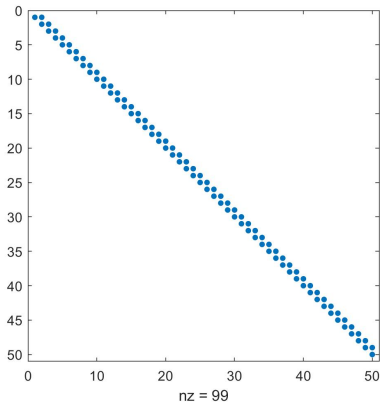
- We can take advantage of sparsity in Gaussian elimination, performing only the operations that are necessary (e.g. only the nonzero entries below a pivot are eliminated, and when summing two rows only the nonzero entries are summed). This allows to beat the $O(n^3)$ cost for dense matrices.
- This works particularly well in case of **banded matrices** (the nonzero are concentrated in a narrow “band” around the diagonal). In this case, a system can be solved in $O(n)$ operations.

Example of a sparse matrix

A from 1D Poisson problem on $[0, 1]$ (discretized with FEM/FD, $h = 0.02$):
sparsity pattern of A



sparsity pattern of U



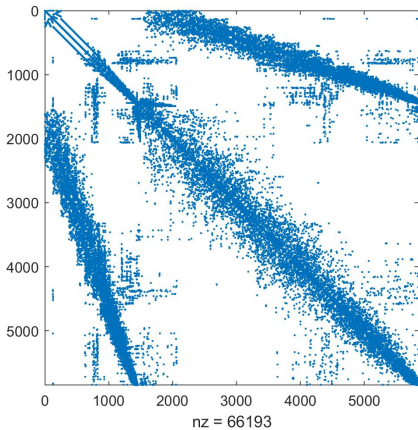
Fill-in

In general, the factor U (and L) can have much more nonzero entries than A . This phenomenon, known as **fill-in** significantly increases time and memory consumption, and represents the main drawback of direct solvers for sparse systems.

Example of fill-in

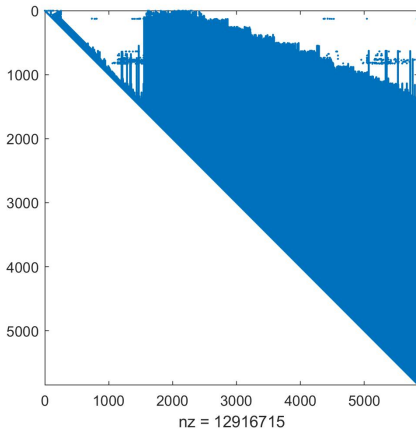
We consider a matrix A generated by a Finite Element Method for solving a 2D Poisson problem on the unit circle (meshsize $h = 0.05$).

sparsity pattern of A



$\text{memory}(A) \approx 1 \text{ MB}$

sparsity pattern of U



$\text{memory}(U) \approx 242 \text{ MB}$

Orderings

- The level of fill-in is often sensitive to the ordering of the variables
- Example:

$$A = \begin{bmatrix} x & x & x & x & x \\ x & x & & & \\ x & & x & & \\ x & & & x & \\ x & & & & x \end{bmatrix} \implies U = \begin{bmatrix} x & x & x & x & x \\ & x & x & x & x \\ & & x & x & x \\ & & & x & x \\ & & & & x \end{bmatrix}$$

A is sparse but U is completely dense.

- But if we re-order rows and columns from last to first:

$$A = \begin{bmatrix} x & & & & x \\ & x & & & x \\ & & x & & x \\ & & & x & x \\ x & x & x & x & x \end{bmatrix} \implies U = \begin{bmatrix} x & & & & x \\ & x & & & x \\ & & x & & x \\ & & & x & x \\ & & & & x \end{bmatrix}$$

In this case, there is no fill-in.

Sparse matrices and iterative solvers

- Recall that at each iteration of an iterative method we have to compute a matrix-vector product

$$v \longrightarrow Av$$

If A is sparse, only the nonzero entries of A are involved in the computation:

$$(Av)_i = \sum_{j=1}^n a_{ij}v_j = \sum_{j \text{ s.t. } a_{ij} \neq 0}^n a_{ij}v_j$$

The cost of a matrix-vector product is then $2\text{nnz}(A)$, versus $2n^2$ for dense matrices.

- Iterative solvers do not suffer from fill-in. In particular, the main memory consumption is just the storing of A . Hence iterative methods typically require much less memory than direct methods.

Summary on Linear Systems

Let us summarize the methods available for solving $A\underline{x} = \underline{b}$, with $A \in \mathbb{R}^{n \times n}$ non singular.

Direct Methods

Methods	Requirements on A	Cost
GEM / LU	¹ $\det(A_i) \neq 0, i = 1, \dots, n$	$\sim 2/3 n^3$ FLOPs
GEM / LU + Pivoting	none	$\sim 2/3 n^3$ FLOPs

¹ A_i is the matrix obtained considering only the first i rows and the first i columns of A . This condition is automatically satisfied if A is diagonally dominant or if A is SPD.

Iterative Methods ($\sim 2n^2$ FLOPs for each iteration)

Methods	Requirements on A	Sufficient conditions for convergence
Jacobi	$A_{ii} \neq 0, i = 1, \dots, n$	A diagonally dominant
Gauss-Seidel	$A_{ii} \neq 0, i = 1, \dots, n$	A diagonally dominant or A SPD
Steepest Descent Method	A SPD	always ensured
Conjugate Gradient Method	A SPD	always ensured in less than n iterations

Eigenvalues and eigenvectors

Let $A \in \mathbb{R}^{n \times n}$. If $0 \neq v \in \mathbb{C}^n$ and $\lambda \in \mathbb{C}$ satisfy

$$Av = \lambda v$$

then λ is called **eigenvalue**, and v is called **eigenvector**.

Given a matrix, we want to approximate its eigenvalues and eigenvectors.
Some applications:

- Structural engineering (natural frequency, heartquakes)
- Electromagnetics (resonance cavity)
- Google's Pagerank algorithm
- ...

The characteristic polynomial

The eigenvalues of a matrix are the roots of **the characteristic polynomial**

$$p(\lambda) := \det(\lambda I - A) = 0$$

However, computing the roots of a polynomial is a very ill-conditioned problem! We cannot use this approach to compute the eigenvalues.

Eigenvalues and eigenvectors

Algorithms that compute the eigenvalues/eigenvectors of a matrix are divided into two categories:

- ① Methods that compute all the eigenvalues/eigenvectors at once.
- ② Methods that compute only a few (possibly one) eigenvalues/eigenvectors.

The methods are also different whether the matrix is symmetric or not. In this lesson we will discuss methods of type 2.

Diagonalizable matrices

Definition

We say that a matrix $A \in \mathbb{C}^{n \times n}$ is diagonalizable if there exists a non singular matrix U and a diagonal matrix D such that $U^{-1}AU = D$.

The diagonal element of D are the eigenvalue of A and the column u_i of U is an eigenvector of A relative to the eigenvalue $D_{i,i}$.

Since a scalar multiple of an eigenvector is still an eigenvector, we can choose U such that $\|u_i\|_2 = 1$ for $i = 1, \dots, n$.

Finally, we observe that if A is diagonalizable, since U is non singular, then the vectors $\{u_1, \dots, u_n\}$ form a basis of $\mathbb{C}^n$.

From now on, we assume that the eigenvalues are numbered in decreasing order (in module), i.e.

$$|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$$

Eigenvalues/eigenvectors of a symmetric matrix

Theorem

All the eigenvalues of a real symmetric matrix are **real**. Moreover, there exists a basis of eigenvectors $u_1, \dots, u_n$, i.e.

$$Au_i = \lambda_i u_i$$

that are orthonormal, i.e.

$$(u_i, u_j) = \delta_{ij}$$

The power method

We want to approximate the eigenvalue of A that is largest in module.

v_0 = some vector with $\|v_0\| = 1$.

for $k = 1, 2, \dots$

$$w = Av_{k-1}$$

apply A

$$v_k = w / \|w\|$$

normalize

$$\mu_k = (v_k)^H A v_k$$

Reyleigh quotient

end

Theorem

Let $A \in \mathbb{C}^{n \times n}$ be a diagonalizable matrix. Assume $|\lambda_1| > |\lambda_2|$ and $v_0 = \sum_{i=1}^n \alpha_i u_i$, with $\alpha_1 \neq 0$. Then there exists $C > 0$, independent of k , such that

$$\|\tilde{v}_k - u_1\|_2 \leq C \left| \frac{\lambda_2}{\lambda_1} \right|^k,$$

$$\text{where } \tilde{v}_k = \frac{\|A^k v_0\|}{\alpha_1 \lambda_1^k} v_k.$$

Proof

We expand v_0 on the eigenvector basis $\{u_1, \dots, u_n\}$ chosen s.t. $\|u_i\| = 1$ for $i = 1, \dots, n$:

$$v_0 = \sum_{i=1}^n \alpha_i u_i, \quad \text{with } \alpha_1 \neq 0$$

It holds

$$A^k v_0 = \sum_{i=1}^n \alpha_i \lambda_i^k u_i \quad \text{and} \quad v_k = \frac{A^k v_0}{\|A^k v_0\|}$$

Hence, we can write

$$\tilde{v}_k = \frac{A^k v_0}{\alpha_1 \lambda_1^k} = u_1 + \sum_{i=2}^n \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k u_i$$

At this point, it holds

$$\|\tilde{v}_k - u_1\|_2 = \left\| \sum_{i=2}^n \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k u_i \right\|_2 \leq \sum_{i=2}^n \left\| \frac{\alpha_i}{\alpha_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k u_i \right\|_2 = \sum_{i=2}^n \left| \frac{\alpha_i}{\alpha_1} \right| \left| \frac{\lambda_i}{\lambda_1} \right|^k$$

So, we obtain

$$\|\tilde{v}_k - u_1\|_2 \leq \sum_{i=2}^n \left| \frac{\alpha_i}{\alpha_1} \right| \left| \frac{\lambda_i}{\lambda_1} \right|^k \leq (n-1) \cdot \max_{i=2,\dots,n} \left(\left| \frac{\alpha_i}{\alpha_1} \right| \right) \left| \frac{\lambda_2}{\lambda_1} \right|^k = C \left| \frac{\lambda_2}{\lambda_1} \right|^k,$$

where we have defined $C = (n-1) \cdot \max_{i=2,\dots,n} \left(\left| \frac{\alpha_i}{\alpha_1} \right| \right)$. Since C does not depend on k , this concludes the proof.

The previous theorem implies that the sequence $\{\tilde{v}_k\}$ converges to the eigenvector u_1 . Since $\tilde{v}_k$ is a scalar multiple of v_k , they have the same direction and this direction converges to the direction of u_1 . As a result, for k that goes to $+\infty$ the vector v_k tends to have the same direction of u_1 . Thus v_k tends to be an eigenvector relative to λ_1 .

Remark

if $|\lambda_2| \ll |\lambda_1|$ the convergence will be fast. On the other hand, if $\lambda_2 \approx \lambda_1$ the convergence will be slow.

We also have a convergence results for the approximation of the eigenvalue λ_1 .

Corollary

Let $A \in \mathbb{C}^{n \times n}$ be a diagonalizable matrix. Assume $|\lambda_1| > |\lambda_2|$ and $v_0 = \sum_{i=1}^n \alpha_i u_i$, with $\alpha_1 \neq 0$. Then it holds

$$|\mu_k - \lambda_1| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right), \quad \text{for } k \rightarrow +\infty.$$

For symmetric real matrices, we have a better convergence results:

Corollary

Let $A \in \mathbb{R}^{n \times n}$ be a symmetric matrix. Assume $|\lambda_1| > |\lambda_2|$ and $v_0 = \sum_{i=1}^n \alpha_i u_i$, with $\alpha_1 \neq 0$. Then it holds

$$|\mu_k - \lambda_1| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^{2k}\right), \quad \text{for } k \rightarrow +\infty.$$

Some observations

One of the hypothesis of the previous results is $\alpha_1 \neq 0$, where α_i are defined such that $v_0 = \sum_{i=1}^n \alpha_i u_i$. Clearly, $u_1, \dots, u_n$ are unknown and we cannot check if v_0 satisfies this hypothesis.

Practically this is not a real obstacle. Consider for simplicity the case of $A \in \mathbb{R}^{n \times n}$ symmetric. If we choose v_0 s.t $\alpha_1 = 0$, there are two possible cases:

- in *exact arithmetic*, we get $\lim_{k \rightarrow +\infty} \tilde{v}_k = u_2$ and $\lim_{k \rightarrow +\infty} \mu_k = \lambda_2$, as long as $|\lambda_2| > |\lambda_3|$ and $\alpha_2 \neq 0$.
- in *finite arithmetic*, during the iterations of the Power Method, round-off errors can cause the appearance of a non-zero component in the direction of u_1 in a certain v_k . If this happens, the method immediately starts to converge towards the dominant eigenvalue λ_1 and its corresponding eigenvector u_1 .

For more general $A \in \mathbb{C}^{n \times n}$ (possibly, real and non symmetric) the same happens but one has to use complex finite arithmetic and initialize v_0 as a vector with nonzero real and imaginary parts.

Stopping criterion

A simple stopping criterion for the power method is based on the residual:

$$\text{Stop when } \|Av_k - \mu_k v_k\| \leq \text{tol}$$

How can we compute other eigenvalues and eigenvectors?

Let $\mu \in \mathbb{C}$ a user-specified parameter that is not an eigenvalue of A , we want to approximate the closest eigenvalue of A to μ , i.e.

$$\lambda_J = \underset{i}{\operatorname{argmin}} |\mu - \lambda_i|$$

Inverse Power method

Input: $A \in \mathbb{C}^{n \times n}$, $v_0 \in \mathbb{C}^n$ with $\|v_0\| = 1$, $\text{MAXITER} \in \mathbb{N}$, $\text{tol} \in \mathbb{R}^+$.

for $k = 1, 2, \dots, \text{MAXITER}$

$$w = (A - \mu I)^{-1} v_{k-1} \quad (\text{equivalently, solve } (A - \mu I) w = v_{k-1})$$

$$v_k = w / \|w\|$$

$$\mu_k = (v_k)^H A v_k \quad (\text{Rayleigh quotient with } A)$$

Check the Stopping criterion

end

Output: μ_k and v_k .

Beware

Since μ is not an eigenvalue of A , the matrix $A - \mu I$ is non singular.

Since $Au_i = \lambda_i u_i$, then $(A - \mu I)u_i = (\lambda_i - \mu)u_i$, and then $\frac{1}{\lambda_i - \mu} u_i = (A - \mu I)^{-1} u_i$. Let λ_J be the eigenvalue of A closest to μ , the largest (in module) eigenvalue of $(A - \mu I)^{-1}$ is then $\frac{1}{\lambda_J - \mu}$, and the relative eigenvector is u_J . The inverse power method is just a power method applied to $(A - \mu I)^{-1}$, and the previous results apply: $\tilde{v}_k$ converges to u_J . Since the Rayleigh quotient μ_k is computed with A instead of $(A - \mu I)^{-1}$, it converges to λ_J .

Theorem

Assume $|\mu - \lambda_J| < |\mu - \lambda_i| \forall i = 1, \dots, n, i \neq J$ and $v_0 = \sum_{i=1}^n \alpha_i u_i$, with $\alpha_J \neq 0$. Then

$$\lim_{k \rightarrow +\infty} \mu_k = \lambda_J$$

and

$$\lim_{k \rightarrow +\infty} \|\tilde{v}_k - u_J\|_2 = 0, \quad \text{where } \tilde{v}_k = \frac{\|A^k v_0\|}{\alpha_1 \lambda_1^k} v_k.$$

Note that if $\mu = 0$, the method approximates the eigenvalue of A that is smallest in module.

Least squares method

Used to approximate solutions of overdetermined systems of equations, i.e., systems where the number of equations is bigger than the number of unknowns:

$$Ax = b, \quad A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, m > n$$

Standard approach in regression analysis, and is widely used for “data fitting”. The name “least squares” means that the solution minimises the sum of the squares of the errors made in every single equation.

In data fitting, the best fit in the least square sense minimises the sum of the squares of the residuals, each residual being the difference between the observed value and the value provided by the model used.

most popular example: linear regression

We have a set of data $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$, where x_i are the **independent variables**, (all distinct), and y_i are the **observations**; **m is big**.

We want to fit the data with a straight line $p(x) = a_1 + a_2x$; finding a straight line such that $p(x_i) = y_i$, $i = 1, \dots, m$ is impossible, unless all the y_i are aligned. Then we look for the straight line that minimises

$$S = \sum_{i=1}^m (y_i - p(x_i))^2.$$

Each quantity $y_i - p(x_i)$ is a residual, that is, the difference between the observed value y_i and the value $p(x_i)$ predicted by our model (a straight line in this case). By using the expression of $p(x)$ in S , we see that we have to minimise a function of two unknowns, a_1 , and a_2 :

$$F(a_1, a_2) = \sum_{i=1}^m (y_i - (a_1 + a_2x_i))^2.$$

$$F(a_1, a_2) = \sum_{i=1}^m (y_i - (a_1 + a_2 x_i))^2$$

F , as a function of a_1 and a_2 , is a second degree polynomial:

$$\begin{aligned} F(a_1, a_2) &= \sum_{i=1}^m (y_i^2 + a_1^2 + x_i^2 a_2^2 + 2x_i a_1 a_2 - 2y_i a_1 - 2y_i x_i a_2) \\ &= m a_1^2 + \left(\sum_{i=1}^m x_i^2 \right) a_2^2 + 2 \left(\sum_{i=1}^m x_i \right) a_1 a_2 - 2 \left(\sum_{i=1}^m y_i \right) a_1 - 2 \left(\sum_{i=1}^m y_i x_i \right) a_2 + \sum_{i=1}^m y_i^2. \end{aligned}$$

$$F(a_1, a_2) =$$

$$= \begin{bmatrix} a_1 & a_2 \end{bmatrix} \begin{bmatrix} m & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} -2 \sum_{i=1}^m y_i & -2 \sum_{i=1}^m y_i x_i \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} + \sum_{i=1}^m y_i^2$$

The Hessian of F is

$$HF(a_1, a_2) = 2 \begin{bmatrix} m & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \end{bmatrix}$$

Now, we show that $HF(a_1, a_2)$ is positive definite. Let us start by observing that:

$$\text{trace}(HF) = 2 \left(m + \sum_{i=1}^m (x_i)^2 \right) > 0$$

and

$$\det(HF) = 2 \left(m \sum_{i=1}^m (x_i)^2 - \left(\sum_{i=1}^m x_i \right)^2 \right)$$

We use the Cauchy-Schwarz inequality to obtain

$$\sum_{i=1}^m x_i \equiv \sum_{i=1}^m x_i \cdot 1 \leq \left(\sum_{i=1}^m (x_i)^2 \right)^{1/2} \left(\sum_{i=1}^m 1^2 \right)^{1/2},$$

$((\underline{x}, \underline{1}) \leq \|\underline{x}\| \|\underline{1}\|)$. In our case, since x_i are distinct, $(\underline{x}, \underline{1}) < \|\underline{x}\| \|\underline{1}\|$ and we square it to obtain:

$$\left(\sum_{i=1}^m x_i \right)^2 < m \left(\sum_{i=1}^m (x_i)^2 \right).$$

Thus also $\det(HF) > 0$. Recalling that:

$$\lambda_1(HF) + \lambda_2(HF) = \text{trace}(HF) > 0$$

$$\lambda_1(HF) \cdot \lambda_2(HF) = \det(HF) > 0,$$

it follows $\lambda_1(HF) > 0$ and $\lambda_2(HF) > 0$, which means that HF is positive definite. Hence F is strictly convex. As a consequence, F has a unique minimum point.

To find the minimum point, we look for stationary points, which are points (a_1, a_2) where $\nabla F(a_1, a_2) = 0$. It holds:

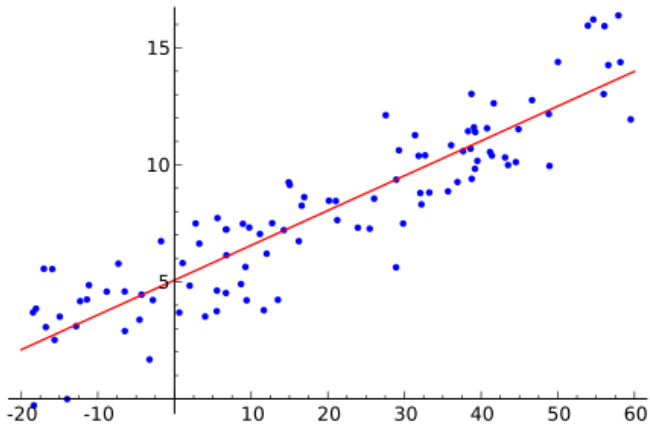
$$\nabla F(a_1, a_2) = 2 \begin{bmatrix} m & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} + \begin{bmatrix} -2 \sum_{i=1}^m y_i \\ -2 \sum_{i=1}^m y_i x_i \end{bmatrix}$$

By imposing $\nabla F = 0$ we obtain a system of two equations in the two unknowns a_1, a_2 :

$$(LS1) \quad \begin{bmatrix} m & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m y_i \\ \sum_{i=1}^m x_i y_i \end{bmatrix}$$

Let $\bar{a}_1, \bar{a}_2$ be the solution, then

$p(x) = \bar{a}_1 + \bar{a}_2 x$ is the linear regression line



Other models

Least square method is widely applied in many fields (economics, statistics, stock-market and the like) to predict the behaviour of a phenomenon for which the values (x_i, y_i) , for $i = 1, \dots, m$ are samples (or experimental data).

In different cases, our “guess” could be different from the linear case discussed so far. Actually different models are used, in different circumstances, in order to have a better fitting of the data.

For example, if the data show a quadratic distribution we might use a parabola $p(x) = a_1 + a_2x + a_3x^2$. In this case S would become

$$F(a_1, a_2, a_3) = \sum_{i=1}^m (y_i - (a_1 + a_2x_i + a_3x_i^2))^2.$$

Other models

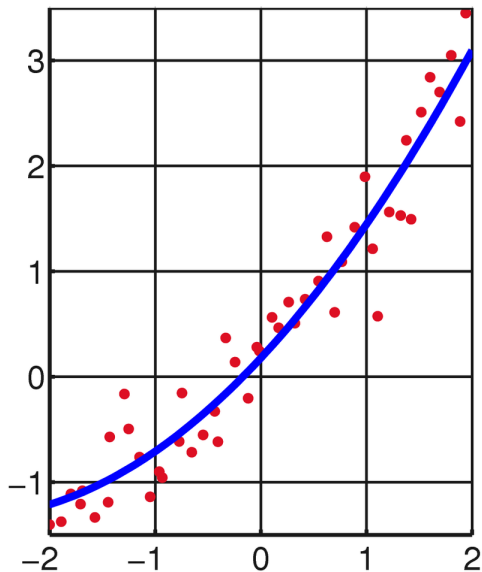
$$F(a_1, a_2, a_3) = \sum_{i=1}^m (y_i - (a_1 + a_2 x_i + a_3 x_i^2))^2.$$

Proceeding as before, computing ∇F and imposing $\nabla F = 0$ to find the point of minimum we obtain a 3×3 system in the 3 unknowns a_1, a_2, a_3 :

$$(LS2) \quad \begin{bmatrix} m & \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 & \sum_{i=1}^m (x_i)^3 \\ \sum_{i=1}^m (x_i)^2 & \sum_{i=1}^m (x_i)^3 & \sum_{i=1}^m (x_i)^4 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m y_i \\ \sum_{i=1}^m x_i y_i \\ \sum_{i=1}^m (x_i)^2 y_i \end{bmatrix}$$

The solution of the system is the parabola that gives the best fit of the given data.

Example of a least square parabola



More general models

More generally, least square method can be used to fit a set of data with a linear combination of functions (not necessarily monomials) chosen to best fit the distribution of a given cloud of data.

Let S_n (with $n \ll m$) be a finite dimensional space:

$S_n = \text{span}\{\varphi_1, \varphi_2, \dots, \varphi_n\}$. We look for a function $p(x) = \sum_{j=1}^n a_j \varphi_j(x)$

such that

$$F(a_1, a_2, \dots, a_n) := \sum_{i=1}^m (y_i - \sum_{j=1}^n a_j \varphi_j(x_i))^2 = \text{minimum}$$

More general models

As before, computing ∇F and imposing $\nabla F = 0$, the point of minimum will be the solution of the $n \times n$ linear system in the n unknowns

$a_1, a_2, \dots, a_n$:

$$\begin{bmatrix} \sum_{i=1}^m (\varphi_1(x_i))^2 & \sum_{i=1}^m \varphi_1(x_i) \varphi_2(x_i) & \cdots & \sum_{i=1}^m \varphi_1(x_i) \varphi_n(x_i) \\ & \sum_{i=1}^m (\varphi_2(x_i))^2 & \cdots & \sum_{i=1}^m \varphi_2(x_i) \varphi_n(x_i) \\ & & \ddots & \vdots \\ & & & \sum_{i=1}^m (\varphi_n(x_i))^2 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m y_i \varphi_1(x_i) \\ \sum_{i=1}^m y_i \varphi_2(x_i) \\ \vdots \\ \sum_{i=1}^m y_i \varphi_n(x_i) \end{bmatrix}$$

symm

A different approach

The least square systems ($LS1$), ($LS2$), and the general one here above can be obtained with a different procedure. Let us see how, in the simplest case of the linear regression line.

We are looking for a line of equation $p(x) = a_1 + a_2x$ such that $p(x_i) = y_i$, $i = 1, \dots, m$: we have m equations in 2 unknowns which, in matrix form, is the overdetermined system

$$\underbrace{\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_m \end{bmatrix}}_A \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}}_b \quad (1)$$

By left-multiplying the system by A^T we obtain $A^T(A\underline{a} - \underline{b}) = 0$:

$$\underbrace{\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_1 & x_2 & \cdots & x_m \end{bmatrix}}_{A^T} \underbrace{\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_m \end{bmatrix}}_A \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_1 & x_2 & \cdots & x_m \end{bmatrix}}_{A^T} \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}}_{\underline{b}}$$

$\Downarrow$

$$(LS1) \quad \underbrace{\begin{bmatrix} m & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \end{bmatrix}}_{A^T A} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \underbrace{\begin{bmatrix} \sum_{i=1}^m y_i \\ \sum_{i=1}^m x_i y_i \end{bmatrix}}_{A^T \underline{b}}$$

This is the same 2×2 system obtained with the least square approach.

The solution of (LS1) is called the least square solution of (1), and it exists provided that $A^T A$ is non-singular. This is true if the matrix A has full rank (rank 2 in this case). Indeed:

$A^T A$ is always symmetric and positive semidefinite, for every matrix $A \neq 0$:

$$(A^T A \underline{x}, \underline{x}) = \|A \underline{x}\|^2 \geq 0$$

$A^T A$ is positive definite if A has full rank (that is: if $A \underline{x} = \underline{0}$ implies that $\underline{x} = \underline{0}$).

In fact, if $A \underline{x} = \underline{0} \rightarrow \underline{x} = \underline{0}$ then

$$(A^T A \underline{x}, \underline{x}) = \|A \underline{x}\|^2 = 0 \iff \underline{x} = \underline{0}.$$

Note: if A has full rank, (LS1) has always a solution, even if system (1) has no solutions

Solution of nonlinear equations

Goal: find the roots (or zeroes) of a nonlinear function:

given $f : [a, b] \rightarrow \mathbb{R}$, find $\alpha \in \mathbb{R}$ such that $f(\alpha) = 0$.

Various applications, e.g. optimization: finding stationary points of a function leads to compute the roots of f' .

When f is linear (and its graphic is a straight line) the problem is very easy. But when the analytic expression of f is more complicated, even though we have an idea of the location of its roots (with the help of graphics), we are unable to compute them exactly. Even finding the roots of polynomials of higher degree is difficult.

All the methods available in the literature are iterative, that is, they construct a sequence of values converging to the root(s): starting from an initial guess $x^{(0)}$, we construct a sequence of values $x^{(k)}$ such that

$$\lim_{k \rightarrow \infty} x^{(k)} = \alpha.$$

Questions/comments regarding iterative methods:

- Does the sequence converge?
- Does convergence depend on the initial guess $x^{(0)}$? (In general, yes)
- How fast is the convergence? (or: what is the *order of convergence*?)
- When to stop the procedure? (how many iterations should we do? need for reliable stopping criteria)

Bisection method

Simplest and robust method, based on the intermediate value theorem:

Theorem

(Bolzano) Let $f : [a, b] \rightarrow \mathbb{R}$ be a continuous function that has opposite signs in $[a, b]$ (meaning, to be precise, that $f(a)f(b) < 0$). Then there exists $\alpha \in]a, b[$ such that $f(\alpha) = 0$.

Remark

Note that the root α does not need to be unique (take $f(x) = \cos(x)$ on $[0, 3\pi]$). Hence, under the hypotheses of Bolzano's theorem, we will look for *a root of the equation* essentially without choosing which one.

Bisection method

Idea: to construct a sequence by repeatedly bisecting the interval and selecting to proceed the sub-interval where the function has opposite signs. In the hypotheses of the intermediate value Theorem, we will

- divide the interval in two by computing the midpoint c of the interval:
$$c = (a + b)/2;$$
- compute the value $f(c)$;
- if $f(c) = 0$ we found the root (very unlikely!),
- if not, that is, if $f(c) \neq 0$, there are two possibilities:
 - ▶ $f(a)f(c) < 0$ (and then f has opposite signs on $[a, c]$),
 - ▶ or $f(b)f(c) < 0$ (and then f has opposite signs on $]c, b[$).

The method selects the subinterval where f has opposite signs as the new interval to be used in the next step. In this way **an interval that contains a zero of f is reduced in width by 50% at each step**. The process is continued until the interval is sufficiently small.

Pseudocode

Input: the function f , the endpoints a, b , the tolerance $\text{tol} \in \mathbb{R}^+$ and $\text{MAXITER} \in \mathbb{N}$

$a^{(1)} \leftarrow a$ and $b^{(1)} \leftarrow b$

for $k = 1, \dots, \text{MAXITER}$

$c^{(k)} = (a^{(k)} + b^{(k)})/2$

if $f(c^{(k)}) = 0$ or $(a^{(k)} + b^{(k)})/2 < \text{tol}$

Output: $c^{(k)}$

Stop the algorithm

end

if $\text{sign}(f(c^{(k)})) = \text{sign}(f(a^{(k)}))$

$a^{(k+1)} \leftarrow c^{(k)}$ and $b^{(k+1)} \leftarrow b^{(k)}$

else

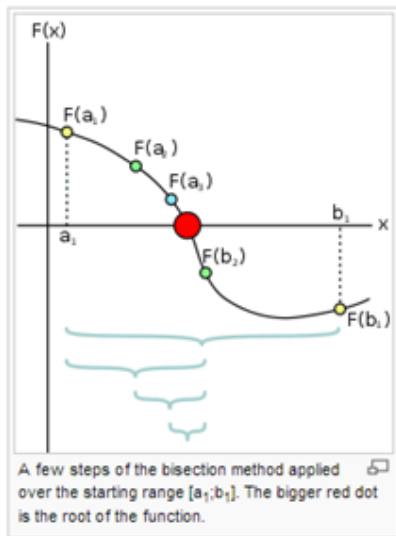
$a^{(k+1)} \leftarrow a^{(k)}$ and $b^{(k+1)} \leftarrow c^{(k)}$

end

end

Output: The method exceed the maximum number of iteration without reaching the desired tolerance.

Example



Analysis

In the hypotheses of Bolzano's theorem (f continuous with opposite signs at the endpoints of its interval of definition) the bisection method **converges always** to a root of f , but it is very slow: the absolute value of the error is halved at each step, that is, the method converges linearly.

If $c^{(1)}$ is the midpoint of $[a,b]$, and $c^{(n)}$ is the midpoint of the interval at the n -th step, the error is bounded by

$$|c^{(n)} - \alpha| \leq \frac{b-a}{2^n}$$

This relation can be used to determine in advance the number of iterations needed to converge to a root within a given tolerance:

$$\frac{b-a}{2^n} \leq TOL \implies n \geq \log_2(b-a) - \log_2 TOL$$

Example: $b-a=1$, $TOL=10^{-3}$ gives $n \geq 3 \log_2 10$, $TOL=10^{-4}$ gives $n \geq 4 \log_2 10$ and so on. Since $\log_2 10 \simeq 3.32$, to gain one order of accuracy we need a little more than 3 iterations.

Newton's method

For each iterate x_k , the function f is approximated by its tangent in x_k :

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

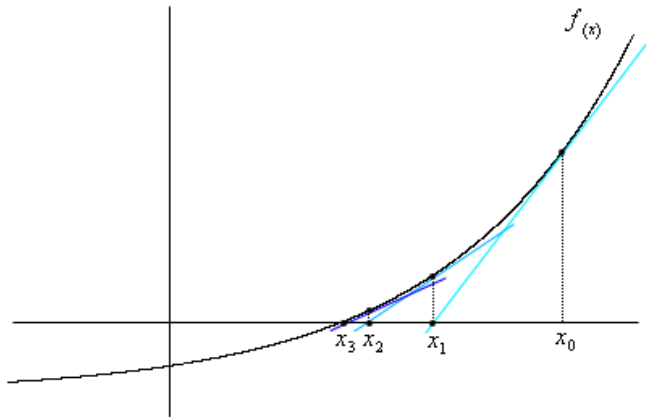
Then we impose that the right-hand side is 0 for $x = x_{k+1}$. Thus,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

More assumptions needed on f :

- f must be differentiable, and f' must not vanish.
- the initial guess x_0 must be chosen well, otherwise the method might fail
- suitable stopping criteria have to be introduced to decide when to stop the procedure (no intervals here.....).

Example



Newton's method: Convergence theorem

Theorem

Let $f \in C^2([a, b])$ such that:

- ① $f(a)f(b) < 0$ (*)
- ② $f'(x) \neq 0 \quad \forall x \in [a, b]$ (**)
- ③ $f''(x) \neq 0 \quad \forall x \in [a, b]$ (***)

Let the initial guess x_0 be a *Fourier point* (i.e., a point where f and f'' have the same sign). Then Newton sequence

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad k = 0, 1, 2, \dots \quad (1)$$

converges to the **unique** α such that $f(\alpha) = 0$. Moreover, the order of convergence is 2, that is:

$$\exists C > 0 : \quad |x_{k+1} - \alpha| \leq C|x_k - \alpha|^2. \quad (2)$$

Newton's method: Proof of the Theorem

Proof

Since f is continuous and has opposite signs at the endpoints then the equation $f(x) = 0$ has at least one solution, say α . Moreover condition (***) implies that α is unique (f is monotone).

To prove convergence, let us assume for instance that f is as follows: $f(a) < 0$, $f(b) > 0$, $f' > 0$, $f'' > 0$ (the other cases can be treated in a similar way), so that the initial guess x_0 is any point where $f(x_0) > 0$. We shall prove that Newton's sequence $\{x_n\}$ is a monotonic decreasing sequence bounded by below.

continue...

continuation of the proof

Let us use Taylor expansion centered in x_0 , with Lagrange remainder^a:

$$0 = f(\alpha) = f(x_0) + (\alpha - x_0)f'(x_0) + \underbrace{\frac{(\alpha - x_0)^2}{2}f''(z)}_{>0}$$

with z between α and x_0 . Thus it holds

$$f(x_0) + (\alpha - x_0)f'(x_0) < 0 \quad \text{i.e.} \quad \alpha < x_0 - \frac{f(x_0)}{f'(x_0)} = x_1$$

Hence, $\alpha < x_1 < x_0$, implying, in particular, that $f(x_1) > 0$ so that x_1 is itself a Fourier point. Then we restart with x_1 as initial point, and repeating the same argument as before we would get $\alpha < x_2 < x_1$, with $f(x_2) > 0$.

^a see

https://en.wikipedia.org/wiki/Taylor%27s_theorem#Explicit_formulas_for_the_remainder

continuation of the proof

Proceeding in this way we have

$$\alpha < x_k < x_{k-1} < \dots < x_0$$

for all positive integer k .

Hence, $\{x_n\}$ being a monotonic decreasing sequence bounded by below, it has a limit, that is,

$$\exists \eta \quad \text{such that} \quad \lim_{k \rightarrow \infty} x_k = \eta.$$

Taking the limit in (1) for $k \rightarrow \infty$ (and remembering that both f and f' are continuous, and f' is always $\neq 0$), we have

$$\lim_{k \rightarrow \infty} (x_{k+1}) = \lim_{k \rightarrow \infty} \left(x_k - \frac{f(x_k)}{f'(x_k)} \right) \implies \eta = \eta - \frac{f(\eta)}{f'(\eta)} \implies f(\eta) = 0$$

Then, η is a root of $f(x) = 0$, and since the root is unique, $\eta \equiv \alpha$.

continuation of the proof

It remains to prove (2). For this, use Taylor expansion centered in x_k , with Lagrange remainder

$$f(\alpha) = f(x_k) + (\alpha - x_k)f'(x_k) + \frac{(\alpha - x_k)^2}{2}f''(z), \quad z \text{ between } \alpha \text{ and } x_k.$$

Now: $f(\alpha) = 0$, $f'(x)$ is always $\neq 0$ so we can divide by $f'(x_k)$ and get

$$0 = \underbrace{\frac{f(x_k)}{f'(x_k)} - x_k}_{-x_{k+1}} + \alpha + \frac{(\alpha - x_k)^2}{2f'(x_k)}f''(z)$$

We found

$$0 = \underbrace{\frac{f(x_k)}{f'(x_k)} - x_k}_{-x_{k+1}} + \alpha + \frac{(\alpha - x_k)^2}{2f'(x_k)}f''(z)$$

end of the proof

that we re-write as

$$x_{k+1} - \alpha = \frac{(\alpha - x_k)^2}{2f'(x_k)} f''(z).$$

Thus,

$$|x_{k+1} - \alpha| = \frac{(\alpha - x_k)^2}{2} \frac{|f''(z)|}{|f'(x_k)|} \leq \frac{(\alpha - x_k)^2}{2} \frac{\max |f''(x)|}{\min |f'(x)|}$$

Therefore (2) holds with

$$C = \frac{\max |f''(x)|}{\min |f'(x)|}$$

which exists since both $|f'(x)|$ and $|f''(x)|$ are continuous on the closed interval, and $f'(x)$ is always different from zero.

Newton's method: Practical use of the theorem

The practical use of the above Convergence theorem is not easy.

- Often difficult, if not impossible, to check that all the assumptions are verified.

In practice, we interpret the Theorem as: *if x_0 is “close enough” to the (unknown) root, the method converges, and converges fast.*

- Suggestions: the graphics of the function (if available), and a few bisection steps help in locating the root with a rough approximation. Then choose x_0 in order to start Newton's method and obtain a much more accurate evaluation of the root.

If α is a multiple root ($f'(\alpha) = 0$) the method is in troubles.

Newton's method: Stopping criteria 1

Unlike with bisection method, here there are no intervals that become smaller and smaller, but just the sequence of iterates.

A reasonable criterion could be

- **test on the iterates**: stop at the first iteration n such that

$$|x_n - x_{n-1}| \leq Tol,$$

and take x_n as “root”.

This would work, unless the function is very **steep** in the vicinity of the root (that is, if $|f'(\alpha)| \gg 1$): the tangents being almost vertical, two iterates might be very close to each other but not close enough to the root to make $f(x_n)$ also small, and the risk is to stop when $f(x_n)$ is still big.

Newton's method: Stopping criteria 2

In this situation it would be better to use the

- **test on the residual**: stop at the first iteration n such that

$$|f(x_n)| \leq Tol,$$

and take x_n as “root”.

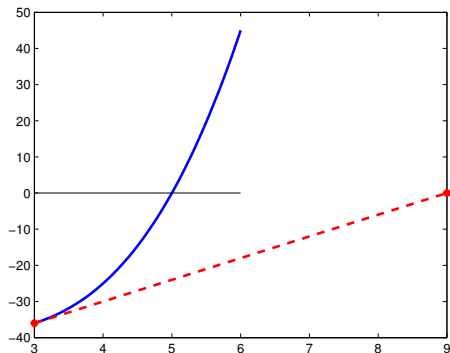
In contrast to the previous criterion, this one would fail if the function is very **flat** in the vicinity of the root (that is, if $|f'(\alpha)| \ll 1$). In this case $|f(x_n)|$ could be small, but x_n could still be far from the root.

What to do then??

Safer to use both criteria, and stop when both of them are verified.

Newton's method: Examples of choices of x_0

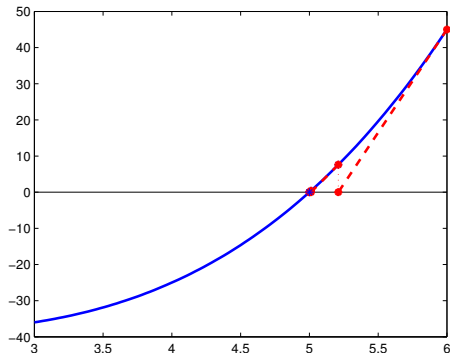
$$f(x) = x^3 - 5x^2 + 9x - 45 \quad \text{in } [3, 6] \quad \alpha = 5$$



Bad x_0 : $x_0 = 3 \Rightarrow x_1 = 9$ outside $[3, 6]$

Newton's method: Examples of choices of x_0

$$f(x) = x^3 - 5x^2 + 9x - 45 \quad \text{in } [3, 6] \quad \alpha = 5$$



Good x_0 : 3 iterations with $Tol = 1.e - 3$

Newton's method: Solution of nonlinear systems

We have to solve a system of N nonlinear equations:

$$\begin{cases} f_1(x_1, x_2, \dots, x_N) = 0 \\ f_2(x_1, x_2, \dots, x_N) = 0 \\ \vdots \\ f_N(x_1, x_2, \dots, x_N) = 0 \end{cases}$$

or, in compact form,

$$\underline{F}(\underline{x}) = \underline{0},$$

having set

$$\underline{x} = (x_1, x_2, \dots, x_N), \quad \underline{F} = (f_1, f_2, \dots, f_N)$$

Newton method

We mimic what done for a single equation $f(x) = 0$: starting from an initial guess x_0 we constructed a sequence by linearizing f at each point and replacing it by its tangent, i.e., its Taylor polynomial of degree 1.

For systems we do the same:

starting from a point $\underline{x}^{(0)} = (x_1^{(0)}, x_2^{(0)}, \dots, x_N^{(0)})$ we construct a sequence $\{\underline{x}^{(k)}\}$ by

- linearising $\underline{F}$ at each point through its Taylor expansion of degree 1:

$$\underline{F}(\underline{x}) \simeq \underline{F}(\underline{x}^{(k)}) + J_F(\underline{x}^{(k)})(\underline{x} - \underline{x}^{(k)})$$

- and then defining $\underline{x}^{(k+1)}$ as the solution of

$$\underline{F}(\underline{x}^{(k)}) + J_F(\underline{x}^{(k)})(\underline{x}^{(k+1)} - \underline{x}^{(k)}) = \underline{0}.$$

$J_F(\underline{x}^{(k)})$ is the **jacobian matrix** of $\underline{F}$ evaluated at the point $\underline{x}^{(k)}$:

$$J_F(\underline{x}) = \begin{bmatrix} \frac{\partial f_1(\underline{x})}{\partial x_1} & \frac{\partial f_1(\underline{x})}{\partial x_2} & \dots & \frac{\partial f_1(\underline{x})}{\partial x_N} \\ \frac{\partial f_2(\underline{x})}{\partial x_1} & \frac{\partial f_2(\underline{x})}{\partial x_2} & \dots & \frac{\partial f_2(\underline{x})}{\partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_N(\underline{x})}{\partial x_1} & \frac{\partial f_N(\underline{x})}{\partial x_2} & \dots & \frac{\partial f_N(\underline{x})}{\partial x_N} \end{bmatrix},$$

System $\underline{F}(\underline{x}^{(k)}) + J_F(\underline{x}^{(k)})(\underline{x}^{(k+1)} - \underline{x}^{(k)}) = \underline{0}$ can obviously be written as: $\underline{x}^{k+1} = \underline{x}^{(k)} - (J_F(\underline{x}^{(k)}))^{-1}\underline{F}(\underline{x}^{(k)})$.

In the actual computation of $\underline{x}^{k+1}$ we **do not** compute the inverse matrix $(J_F(\underline{x}^{(k)}))^{-1}$, but we solve the system

$$J_F(\underline{x}^{(k)})\underline{x}^{k+1} = J_F(\underline{x}^{(k)})\underline{x}^{(k)} - \underline{F}(\underline{x}^{(k)}).$$

Newton's method: Algorithm

Given $\underline{x}^{(0)} \in \mathbb{R}^N$, for $k = 0, 1, \dots$

solve $J_F(\underline{x}^{(k)})\underline{x}^{(k+1)} = J_F(\underline{x}^{(k)})\underline{x}^{(k)} - \underline{F}(\underline{x}^{(k)})$ by the following steps

- solve $J_F(\underline{x}^{(k)})\underline{\delta}^{(k)} = -\underline{F}(\underline{x}^{(k)})$
- set $\underline{x}^{(k+1)} = \underline{x}^{(k)} + \underline{\delta}^{(k)}$

At each iteration k we have to solve a linear system with matrix $J_F(\underline{x}^{(k)})$ (that is the most expensive part of the algorithm).

Note that by introducing the unknown $\underline{\delta}^{(k)}$ we pay an extra sum ($\underline{x}^{(k+1)} = \underline{x}^{(k)} + \underline{\delta}^{(k)}$) but we save the (much more expensive) matrix-vector multiplication $J_F(\underline{x}^{(k)})\underline{x}^{(k)}$.

Newton's method: Stopping criteria

They are the same two criteria that we saw for scalar equations:

- **test on the iterates**: stop at iteration k such that

$$\|\underline{x}^{(k)} - \underline{x}^{(k-1)}\| \leq Tol$$

for some vector norm, and take $\underline{x}^{(k)}$ as “root”.

- **test on the residual**: stop at iteration k such that

$$\|F(\underline{x}^{(k)})\| \leq Tol,$$

and take $\underline{x}^{(k)}$ as “root”.

Here too, it would be wise in practice to use **both** criteria, and stop when both of them are satisfied.

Approximation of functions by polynomials

Why?

Sometimes the analytic expression of a function is complicated, and it might be difficult to integrate it or to differentiate it.

Sometimes the analytic expression is not available, the function is known only at some points at we need to reconstruct the underlying function.

A way to overcome problems of this kind is to approximate functions with polynomials (easy to deal with).

Lagrange interpolation

Theorem 1

Let $g : [a, b] \rightarrow \mathbb{R}$ be a smooth enough function. Given $k + 1$ distinct points $x_1, x_2, \dots, x_{k+1}$ in $[c, d]$, there exists a unique polynomial $\Pi_k(x)$ of degree $\leq k$ such that

$$\Pi_k(x_i) = g(x_i) \quad i = 1, 2, \dots, k + 1. \quad (*)$$

Π_k is called “Lagrange interpolant of g with respect to the points $x_1, x_2, \dots, x_{k+1}$ ”.

The points $x_1, x_2, \dots, x_{k+1}$ are called “interpolation nodes”.

Proof

For proving the existence, we shall write explicitly the expression of Π_k . Let $L_i(x)$, $i = 1, 2, \dots, k+1$ be $k+1$ polynomials, of degree exactly k , defined by:

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{k+1} \frac{(x - x_j)}{(x_i - x_j)} \quad i = 1, 2, \dots, k+1$$

For example, for $k = 3$ and nodes x_1, x_2, x_3, x_4 :

$$\begin{aligned} L_1(x) &= \frac{(x - x_2)(x - x_3)(x - x_4)}{(x_1 - x_2)(x_1 - x_3)(x_1 - x_4)}, & L_2(x) &= \frac{(x - x_1)(x - x_3)(x - x_4)}{(x_2 - x_1)(x_2 - x_3)(x_2 - x_4)} \\ L_3(x) &= \frac{(x - x_1)(x - x_2)(x - x_4)}{(x_3 - x_1)(x_3 - x_2)(x_3 - x_4)}, & L_4(x) &= \frac{(x - x_1)(x - x_2)(x - x_3)}{(x_4 - x_1)(x_4 - x_2)(x_4 - x_3)} \end{aligned}$$

These are **characteristic Lagrange polynomials**, that fulfil

$$L_i \in \mathbb{P}_k : \quad L_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{for } i = j \\ 0 & \text{for } i \neq j \end{cases} \quad i = 1, 2, \dots, k+1$$

continue...

...continuation of the proof.

Claim:

$$\Pi_k(x) := \sum_{i=1}^{k+1} g(x_i) L_i(x)$$

Indeed, it is easy to check that Π_k verifies (*), and this proves **existence**. For proving the uniqueness, we observe that the problem of finding the polynomial

$$\Pi_k(x) := \alpha_1 + \alpha_2 x \dots + \alpha_{k+1} x^k$$

that interpolate g at the $k+1$ distinct nodes $x_1, x_2, \dots, x_{k+1}$ is a linear problem, that takes the form:

$$\begin{bmatrix} 1 & x_1 & \dots & x_1^k \\ 1 & x_2 & \dots & x_2^k \\ 1 & \vdots & \dots & \vdots \\ 1 & x_{k+1} & \dots & x_{k+1}^k \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_{k+1} \end{bmatrix} = \begin{bmatrix} g(x_1) \\ g(x_2) \\ \vdots \\ g(x_{k+1}) \end{bmatrix}$$

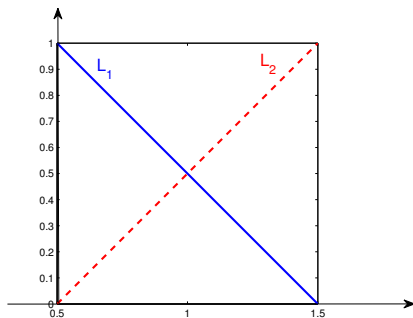
The linear system has dimension $(k+1) \times (k+1)$. Since the problem has always a solution (existence is proved in the previous slide) then the solution is unique . □

Example of characteristic polynomials

Ex. 1: 2 points $x_1 \neq x_2$ ($k + 1 = 2$)

$$L_1(x) = \frac{(x - x_2)}{(x_1 - x_2)}, \quad L_2(x) = \frac{(x - x_1)}{(x_2 - x_1)}$$

degree $k = 1$.



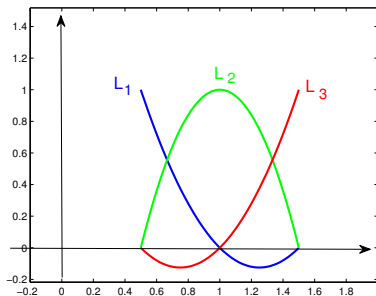
Example of characteristic polynomials

Ex. 2: 3 points $x_1 \neq x_2 \neq x_3$ ($k+1=3$)

$$L_1(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)}, \quad L_2(x) = \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)}$$

$$L_3(x) = \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)}$$

degree $k = 2$.



Cases of interest

Case 1: g approximated by a constant (a polynomial of degree 0)

1 point: $x_1 = \frac{a+b}{2}$ (midpoint of the interval)

$$g(x) \simeq \Pi_0(x) = g(x_1)$$

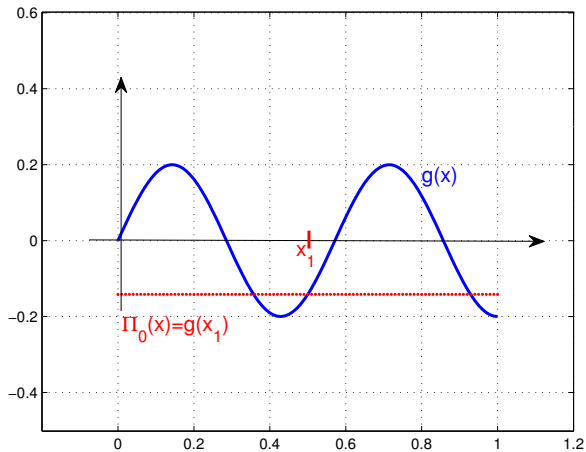
Case 2: g approximated by a polynomial of degree 1 (straight line)

2 point: $x_1 = a$, $x_2 = b$

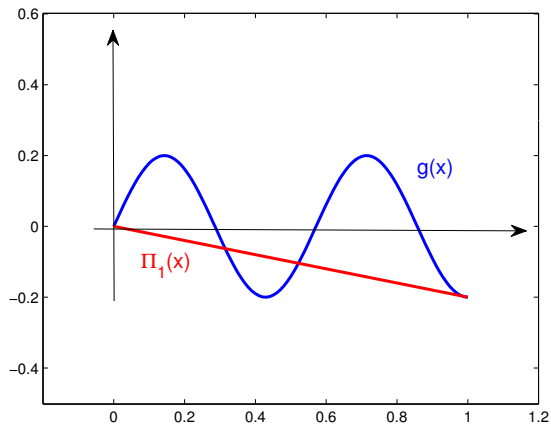
$$\begin{aligned} g(x) &\simeq \Pi_1(x) = g(x_1)L_1(x) + g(x_2)L_2(x) \\ &= g(x_1)\frac{(x-x_2)}{(x_1-x_2)} + g(x_2)\frac{(x-x_1)}{(x_2-x_1)} \end{aligned}$$

Remark: in both cases, if we change the nodes we obtain a different interpolant

Example 1



Example 2



General interpolation error

In both cases, the approximation induces an error that we want to estimate. How big is it? Is the approximation satisfactory? The following Theorem gives a precise expression of the error.

Theorem 2

Let $g \in C^{k+1}([a, b])$, and let $x_1, x_2, \dots, x_{k+1}$ be $k + 1$ distinct points in $[a, b]$. For any generic point $x \in [a, b]$ there exists a $\xi \in [a, b]$ (depending on x) such that the interpolation error is given by

$$e_k(x) := g(x) - \Pi_k(x) = \frac{g^{(k+1)}(\xi)}{(k+1)!} \prod_{j=1}^{k+1} (x - x_j) \quad (1)$$

Proof.

Let x be fixed and $t \in [a, b]$. Define the function $\omega_{k+1}(t) = \prod_{j=1}^{k+1} (t - x_j)$ let us introduce

$$G(t) := e_k(t) - \omega_{k+1}(t)e_k(x)/\omega_{k+1}(x).$$

Since $g \in C^{k+1}([a, b])$ and ω_{k+1} is a polynomial, $G \in C^{k+1}([a, b])$ and vanishes at the $k + 2$ distinct points $x_1, x_2, \dots, x_{k+1}$ and x . Indeed:

$$G(x_i) = e_k(x_i) - \omega_{k+1}(x_i)e_k(x)/\omega_{k+1}(x) = 0 \quad i = 1, 2, \dots, k + 1$$

$$G(x) = e_k(x) - \omega_{k+1}(x)e_k(x)/\omega_{k+1}(x) = 0$$

Thanks to the meanvalue theorem^a, G' will have $k + 1$ distinct zeros, and going recursively, $G^{(j)}$ will have $k + 2 - j$ distinct zeros. Hence, $G^{(k+1)}$ will have one zero, say ξ . Since $\omega_{k+1}^{(k+1)}(t) = (k + 1)!$ and $e_k^{(k+1)}(t) = g^{(k+1)}(t)$ we obtain the result

$$G^{(k+1)}(\xi) = g^{(k+1)}(\xi) - (k + 1)!e_k(x)/\omega_{k+1}(x) = 0$$

and the proof is concluded. □

^ahttps://en.wikipedia.org/wiki/Mean_value_theorem

From (1) we can deduce for instance the following bound:

$$\max_{x \in [a,b]} |e_k(x)| \leq \frac{(b-a)^{k+1}}{(k+1)!} \max_{x \in [a,b]} |g^{(k+1)}(x)| \quad (*)$$

If we know the position of the nodes we can have sharper estimates for the term $\prod_{j=1}^{k+1} (x - x_j)$ that appears in (1). For instance, for the [Case 1](#) we would obtain

[Case 1 \(constant approximation\)](#): Relation (1) in this case gives

$$e_0(x) = g'(\xi)(x - x_1) \quad x_1 = \frac{a+b}{2}$$

No matter where x is situated within $[a, b]$, its distance from the midpoint will be smaller than or equal to the length of half the interval (that is, $(b-a)/2$). Hence

$$\implies \max_{x \in [a,b]} |e_0(x)| \leq \frac{b-a}{2} \max_{x \in [a,b]} |g'(x)| \quad (2)$$

(($b-a$)/2 instead of $b-a$)

Case 2 (linear approximation): For $x_1 = a$ and $x_2 = b$ we can observe that the function $x \rightarrow |(x - a)(x - b)|$, in the interval $[a, b]$ has its maximum for $x = (a + b)/2$ and such a maximum is $((b - a)/2)^2$. Hence

$$\max_{x \in [a, b]} |e_1(x)| \leq \frac{(b - a)^2}{2! \cdot 4} \max_{x \in [a, b]} |g''(x)| \quad (3)$$

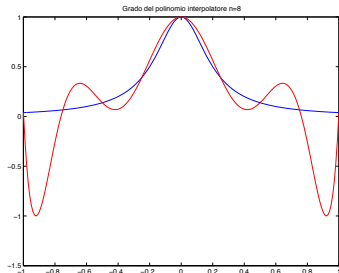
(and we have $(b - a)^2/8$ instead of $(b - a)^2/2$).

Runge function

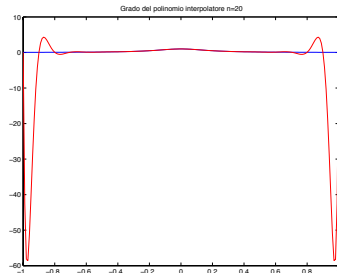
Apparently, increasing the degree k of the Lagrange polynomial should improve the error, which should become smaller and smaller. This is not always the case if the nodes are equally spaced. The classical example is given by the so-called Runge's function:

$$g(x) = \frac{1}{1+x^2}$$

This is what happens



degree 8



degree 20

A simple remedy: Composite Lagrange interpolation

Use of Lagrange interpolation (piecewise) to have a good approximation of a function.

Given $f : [a, b] \rightarrow \mathbb{R}$ (smooth enough), subdivide $[a, b]$ in N subintervals, for simplicity of notation all equal. We have then a uniform subdivision of $[a, b]$ into intervals of length $h = (b - a)/N$. In each subinterval we approximate f with a Lagrange interpolant polynomial.

To fix ideas, let us consider again Cases 1 and 2.

Case 1: $N \rightarrow h = (b - a)/N$, $x_1 = a, x_2 = x_1 + h, \dots, x_{N+1} = b$

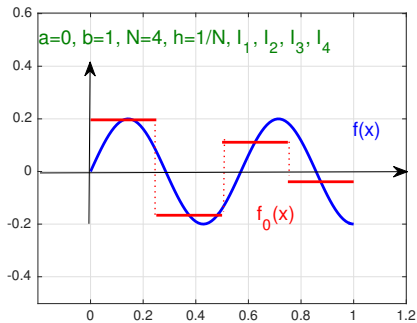
$I_1 = [x_1, x_2], \dots, I_j = [x_j, x_{j+1}], \dots, I_N = [x_N, x_{N+1}]$

x_j^M = midpoint of the interval I_j : $x_j^M = (x_j + x_{j+1})/2$

$f(x)|_{[a,b]} \simeq f_0(x)$ piecewise constant function given by

$$f_0(x)|_{I_j} = f(x_j^M) \quad j = 1, 2, \dots, N$$

Piecewise constant interpolation



Let us study the error $E_0(x) := f(x) - f_0(x)$ $x \in [a, b]$

The maximum error (in absolute value) will be achieved on one subinterval, say $\bar{I}_j$. Then,

$$\max_{x \in [a, b]} |E_0(x)| = \max_{x \in [a, b]} |f(x) - f_0(x)| = \max_{x \in \bar{I}_k} |f(x) - f_0(x_k^M)|$$

Using the mean value theorem¹, here exists c between x and x_k^M such that:

$$\frac{f(x) - f_0(x_k^M)}{x - x_k^M} = f'(c)$$

Then $f(x) - f_0(x_k^M) = (x - x_k^M)f'(c)$ and²

$$\max_{x \in [a, b]} |E_0(x)| = \max_{x \in \bar{I}_k} |f(x) - f_0(x_k^M)| \leq \frac{h}{2} \max_{c \in \bar{I}_k} |f'(c)| \leq \frac{h}{2} \max_{c \in [a, b]} |f'(c)|$$

If f' exists and is bounded, the interpolation error goes to zero as $C h$

¹https://en.wikipedia.org/wiki/Mean_value_theorem

²Or, use (2), with $[a, b] = [x_i, x_{i+1}] \implies b - a = h$ to get the blue estimate

Case 2: $N \rightarrow h = (b - a)/N$, $x_1 = a, x_2 = x_1 + h, \dots, x_{N+1} = b$

$I_1 = [x_1, x_2], \dots, I_j = [x_j, x_{j+1}], \dots, I_N = [x_N, x_{N+1}]$

$f(x)|_{[a,b]} \simeq f_1(x)$ piecewise linear function which, on each interval I_j , is the Lagrange interpolant of degree ≤ 1 with respect to the endpoints of I_j , i.e.,

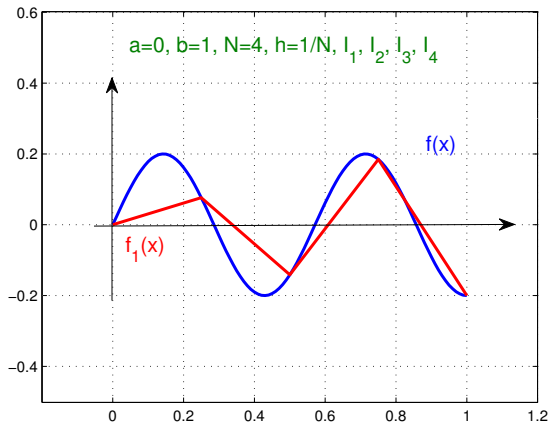
$$f_1(x)|_{I_j} = f(x_j) \frac{(x - x_{j+1})}{(x_j - x_{j+1})} + f(x_{j+1}) \frac{(x - x_j)}{(x_{j+1} - x_j)} \quad j = 1, 2, \dots, N$$

Using the bound (3) for the error $E_1(x) = f(x) - f_1(x)$ and proceeding as before we then obtain

$$\max_{[a,b]} |E_1(x)| \leq \frac{h^2}{8} \max_{[a,b]} |f''(x)| = C h^2 \quad C = \frac{\max_{[a,b]} |f''(x)|}{8}$$

Hence: if f'' exists and is bounded, the interpolation error goes to zero quadratically with h (if you halve h the error is divided by four), and we can choose h as small as we want!

Piecewise linear interpolation



Numerical integration

(a.k.a quadrature formulas or quadrature rules)

Quadrature rules are used to approximate integrals of functions that we are not able to compute exactly.

Given $g : [a, b] \rightarrow \mathbb{R}$, the most common quadrature rules look like

$$\int_a^b g(x) dx \simeq \sum_{i=1}^{k+1} \omega_i g(x_i)$$

where: $x_1, x_2, \dots, x_{k+1}$ are the quadrature “points” or “nodes” of the rule and $\omega_1, \omega_2, \dots, \omega_{k+1}$ are called quadrature “weights”

Definition: The order of precision of a quadrature rule is the maximum degree of the polynomials which are integrated exactly by the rule.

Among the numerous quadrature rules, we shall see the so-called interpolatory rules.

Interpolatory quadrature rules

The function g is approximated by its Lagrange interpolant $\Pi_k(x)$ (of degree $\leq k$) with respect to the given nodes $x_1, x_2, \dots, x_{k+1}$, and then the integral of the polynomial is computed exactly.

Then

$$\begin{aligned}\int_a^b g(x) dx &\simeq \int_a^b \Pi_k(x) dx = \int_a^b \sum_{i=1}^{k+1} g(x_i) L_i(x) dx \\ &= \sum_{i=1}^{k+1} g(x_i) \underbrace{\int_a^b L_i(x) dx}_{\omega_i}\end{aligned}$$

The order of precision of an interpolator formula will be **at least** k (if $g \in \mathbb{P}_k$, it is integrated exactly)

Error analysis for interpolatory quadrature rules

Bounds for the quadrature error are derived by the bounds for the interpolation error (see the slides on Lagrange interpolation):

$$\max_{x \in [a,b]} |g(x) - \Pi_k(x)| \leq \frac{(b-a)^{k+1}}{(k+1)!} \max_{x \in [a,b]} |g^{(k+1)}(x)| \quad (*)$$

Thus,

$$\begin{aligned} \left| \int_a^b g(x) dx - \int_a^b \Pi_k(x) dx \right| &= \left| \int_a^b (g(x) - \Pi_k(x)) dx \right| \leq \int_a^b |g(x) - \Pi_k(x)| dx \\ &\leq \int_a^b \max_{[a,b]} |g(x) - \Pi_k(x)| dx = (b-a) \max_{[a,b]} |g(x) - \Pi_k(x)| \\ &\leq \frac{(b-a)^{k+2}}{(k+1)!} \max_{x \in [a,b]} |g^{(k+1)}(x)| \end{aligned} \quad (1)$$

(Observe that if $g \in \mathbb{P}_k$, then $g^{(k+1)}(x) \equiv 0$. Hence, the quadrature error is $= 0$)

This estimate is obtained using the generic bound for the interpolation error. Sharper estimates can be obtained by analysing each rule directly, as we shall see.

Use of quadrature rules

It should be clear by now that if we want a good approximation of an integral we have to use properly the rules in order to make the error as smaller as we want. Exactly like we did for Lagrange interpolation, we will construct piecewise integration rules, also called **composite integration rules**.

Given $f : [a, b] \rightarrow \mathbb{R}$ (smooth enough), subdivide $[a, b]$ in N subintervals, for simplicity of notation all equal. We have then a uniform subdivision of $[a, b]$ into intervals of length $h = (b - a)/N$:

$I_1 = [x_1, x_2], \dots, I_j = [x_j, x_{j+1}], \dots, I_N = [x_N, x_{N+1}]$. In each subinterval we approximate f with a Lagrange interpolant polynomial of degree k .

Thus,

$$\int_a^b f(x) dx = \sum_{j=1}^N \int_{x_j}^{x_{j+1}} f(x) dx \simeq \sum_{j=1}^N \int_{x_j}^{x_{j+1}} \Pi_k(x) dx$$

Let us see some examples.

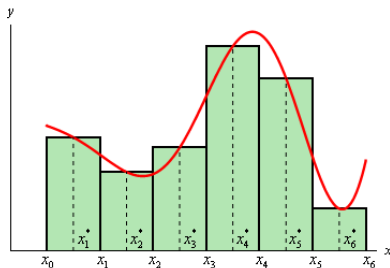
Composite midpoint rule

x_j^M = midpoint of the interval I_j : $x_j^M = (x_j + x_{j+1})/2$

$f(x)|_{[a,b]} \simeq f_0(x)$ piecewise constant function given by

$$f_0(x)|_{I_j} = f(x_j^M) \quad j = 1, 2, \dots, N$$

$$\int_a^b f(x) dx \simeq \int_a^b f_0(x) dx = \sum_{j=1}^N \int_{x_j}^{x_{j+1}} f(x_j^M) dx = h \sum_{j=1}^N f(x_j^M)$$



Composite midpoint rule: pseudocode

Composite midpoint rule

Input: f , a , b , N .

$h = (b - a)/N$

for $i = 1, 2, \dots, N + 1$

$x_i = a + (i - 1)h$

end

for $i = 1, 2, \dots, N$

$x_i^M = (x_i + x_{i+1})/2$

end

$S = 0$

for $i = 1, 2, \dots, N$

$S = S + f(x_i^M)h$

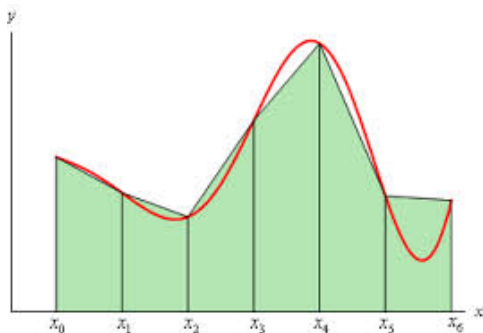
end

Output: S .

Composite trapezoidal rule

$f(x)|_{[a,b]} \simeq f_1(x)$ piecewise linear function which, on each interval I_j , is the Lagrange interpolant of degree ≤ 1 with respect to the endpoints of I_j

$$\int_a^b f(x) dx \simeq \int_a^b f_1(x) dx = \sum_{j=1}^N \int_{x_j}^{x_{j+1}} \Pi_1(x) dx = \frac{h}{2} \sum_{j=1}^N [f(x_j) + f(x_{j+1})]$$

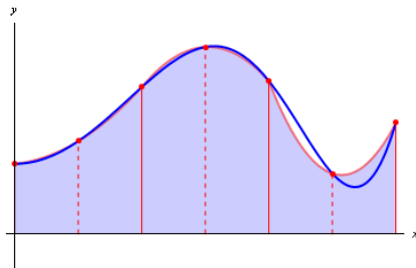


Composite Simpson rule

$f(x)|_{[a,b]} \simeq f_2(x)$ piecewise quadratic function which, on each interval I_j , is the Lagrange interpolant of degree ≤ 2 with respect to the endpoints and the midpoint of I_j

$$\int_a^b f(x) dx \simeq \int_a^b f_2(x) dx = \frac{h}{6} \sum_{j=1}^N [f(x_j) + 4f(x_j^M) + f(x_{j+1})]$$

Exercise: prove the formula using slide 2/15 in each interval...



Composite Midpoint rule: error bound

$$ERR = \left| \sum_{j=1}^N \underbrace{\int_{x_j}^{x_{j+1}} (f(x) - f(x_j^M)) dx}_{E_j} \right|$$

The first possibility is to use (1), which gives in our case:

$$E_j = \left| \int_{x_j}^{x_{j+1}} f(x) - \Pi_0(x) dx \right| \leq h^2 \max_{x \in [x_j, x_{j+1}]} |f'(x)| = O(h^2)$$

Then the global error ERR is the sum of the error in each subinterval:

$$ERR \leq \sum_{j=1}^N |E_j| \leq \max_{[a,b]} |f'(x)| h^2 N = \max_{[a,b]} |f'(x)| (b-a)h = O(h)$$

(we used $Nh = b - a$ and $\max_{[l_j]} |f'(x)| \leq \max_{[a,b]} |f'(x)|$).

Composite Midpoint rule: error bound (sharp)

$$ERR = \underbrace{\left| \sum_{j=1}^N \int_{x_j}^{x_{j+1}} (f(x) - f(x_j^M)) dx \right|}_{E_j}$$

Second possibility: use in each I_j the Taylor expansion centered in x_j^M

$$f(x) = f(x_j^M) + (x - x_j^M)f'(x_j^M) + \frac{(x - x_j^M)^2}{2!}f''(z) \quad (\text{for } z \text{ between } x \text{ and } x_j^M)$$

$$\begin{aligned} E_j &= \underbrace{\int_{x_j}^{x_{j+1}} (x - x_j^M)f'(x_j^M) dx}_{=0} + \int_{x_j}^{x_{j+1}} \frac{(x - x_j^M)^2}{2!}f''(z) dx \\ &\leq \frac{1}{2} \max_{[I_j]} |f''(x)| \int_{x_j}^{x_{j+1}} (x - x_j^M)^2 dx = \frac{\max_{[I_j]} |f''(x)|}{24} h^3 \end{aligned}$$

Composite Midpoint rule: error bound (sharp)

Again, the global error is the sum of the error in each subinterval.

$$ERR \leq \sum_{j=1}^N |E_j| \leq \frac{\max_{[a,b]} |f''(x)|}{24} h^3 N = \frac{\max_{[a,b]} |f''(x)|}{24} (b-a) h^2$$

(we used $Nh = b - a$ and $\max_{[I_j]} |f''(x)| \leq \max_{[a,b]} |f''(x)|$).

We see that the error is zero if $f'' \equiv 0$ in each I_j , i.e., if f is a piecewise polynomial of degree 1.

Hence, the order of precision of the midpoint rule is actually 1 (even though we are projecting onto piecewise constants!)

To summarize:

$$ERR \leq C h^2 \quad \text{with } C = \frac{\max_{[a,b]} |f''(x)|}{24} (b-a)$$

We see that $ERR \rightarrow 0$ for $h \rightarrow 0$ quadratically with h (halving h reduces the error by 4).

Hints on Gaussian rules * NOT FOR THE EXAM *

Conclusions The order of precision of an interpolatory quadrature rule using n nodes is at least $n - 1$ (that is, the rule integrates exactly polynomials of degree up to $n - 1$).

If the n nodes are **Gauss points** (special points that are defined as roots of Legendre polynomials) the order of precision is higher: precisely, $2n - 1$.

Operatively: Gauss points are computed in the open interval $] - 1, 1[$:

$$n = 1 \qquad \int_{-1}^1 g(x) dx \approx 2g(0)$$

$$n = 2 \qquad \int_{-1}^1 g(x) dx \approx g\left(-\frac{1}{\sqrt{3}}\right) + g\left(+\frac{1}{\sqrt{3}}\right)$$

$$n = 3 \qquad \int_{-1}^1 g(x) dx \approx \frac{5}{9}g\left(-\frac{\sqrt{3}}{\sqrt{5}}\right) + \frac{8}{9}g(0) + \frac{5}{9}g\left(\frac{\sqrt{3}}{\sqrt{5}}\right)$$

$$n = 4 \qquad \dots \text{ see the reference books}$$

Hints on Gaussian rules * NOT FOR THE EXAM *

To compute Gauss points on a generic interval $[a, b]$ and use them for evaluating $\int_a^b g(x) dx$ is simple. Consider the map

$$F : [-1, 1] \rightarrow [a, b], \text{ that is, } \hat{x} \in [-1, 1] \rightarrow x = F(\hat{x}) \in [a, b]$$

It is easy to check that the map is linear, given by $x = \frac{b-a}{2}\hat{x} + \frac{a+b}{2}$. Hence, if $\hat{x}_1$ is a Gauss point in $] -1, 1[$, the point

$$x_1 = \frac{b-a}{2}\hat{x}_1 + \frac{a+b}{2} \text{ is a Gauss point in } [a, b]$$

Instead, the quadrature weights get scaled by a factor $F' = \frac{b-a}{2}$ (use change of variable to prove it).

Ordinary Differential Equations (Ode's)

We shall consider some numerical schemes to solve initial-value problems (Cauchy's problems) written as: Find $y = y(t)$ solution of

$$\begin{cases} y'(t) = f(t, y(t)) & t \in [t_0, T] \\ y(t_0) = y_0. \end{cases} \quad (1)$$

We assume $y : [t_0, T] \rightarrow \mathbb{R}$ but can be generalized to $y : [t_0, T] \rightarrow \mathbb{R}^d$

In general $f(t, y(t))$ is a non-linear function describing the evolution in time of $y(t)$. The true solution $y(t)$ of (1) evolves continuously in time, and we want to follow it by a discrete approximation.

Both exact and discrete solution of (1) start from the same initial value y_0 at t_0 . The discrete one takes finite steps Δt , and after n steps it reaches a value y_n . We hope and expect that y_n is close to the exact value $y(t_0 + n\Delta t)$. We shall see that this may or may not happen.

Numerical methods for Ode's

Let us see some schemes to solve numerically (1). They are numerous, and a first distinction is among **1-step methods** and **multi-step methods**. Let us see 1-step methods. They can all be derived in the following way.

Let $t_0, t_1, \dots, t_N = T$ be a set of points in $[t_0, T]$; as usual, for simplify notation we take them equally spaced: (N given, we define $\Delta t = \frac{T - t_0}{N}$ and we set $t_0, t_1 = t_0 + \Delta t, t_2 = t_1 + \Delta t, \dots, t_N = T$). At each step, (on each subinterval $[t_n, t_{n+1}]$) we *integrate the differential equations...*

$$y'(t) = f(t, y(t))$$

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} y'(t) dt = \int_{t_n}^{t_{n+1}} f(t, y(t)) dt. \quad (*)$$

Then, as $y(t)$ in the interval $[t_n, t_{n+1}]$ is unknown to us (and moreover, in general, we are unable to compute the integral exactly), we use some quadrature formula.

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} y'(t) dt = \int_{t_n}^{t_{n+1}} f(t, y(t)) dt. \quad (*)$$

$$\int_{t_n}^{t_{n+1}} f(t, y(t)) dt \sim \text{quadrature formula.}$$

Different choices of quadrature formulas give rise to different schemes.

Examples of numerical schemes

Example 1 We consider first the quadrature formula

$$\int_c^d g(s)ds \simeq (d - c) g(c) \quad (2)$$

that, indeed, is very poor (and is exact only for $g = \text{constant}$). Then

$$\int_{t_n}^{t_{n+1}} f(t, y(t))dt \simeq (t_{n+1} - t_n) f(t_n, y(t_n)) \quad n = 0, 1, 2, \dots \quad (3)$$

Using (3) into $y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} y'(t)dt = \int_{t_n}^{t_{n+1}} f(t, y(t))dt$ we get:

$$y(t_1) \simeq y(t_0) + \Delta t f(t_0, y(t_0)) = y_0 + \Delta t f(t_0, y_0) =: y_1$$

$$y(t_2) \simeq y(t_1) + \Delta t f(t_1, y(t_1)) \simeq y_1 + \Delta t f(t_1, y_1) =: y_2$$

$\vdots$

$$y(t_N) \simeq y(t_{N-1}) + \Delta t f(t_{N-1}, y(t_{N-1})) \simeq y_{N-1} + \Delta t f(t_{N-1}, y_{N-1}) =: y_N$$

Examples of numerical schemes

It is clear from this that errors accumulate at each step and might produce unexpected results. We will analyse the scheme later on. Let us write it in a compact form:

$$\begin{cases} y_0 \text{ given} \\ y_{n+1} = y_n + \Delta t f(t_n, y_n) \quad n = 0, 1, \dots, N-1 \end{cases} \quad (EE)$$

This is called **EXPLICIT EULER** method or **FORWARD EULER** method: at each step, the value y_n can be explicitly computed using values at the previous steps. It is very simple and inexpensive but, as we shall see, there is a “but” ...

Examples of numerical schemes

Example 2 This time we consider the quadrature formula

$$\int_c^d g(s) ds \simeq (d - c) g(d) \quad (4)$$

that is also very poor and is exact only if $g = \text{constant}$, like the previous one. However the resulting scheme will be very different. In fact, applying it to our case we get

$$\int_{t_n}^{t_{n+1}} f(t, y(t)) dt \simeq (t_{n+1} - t_n) f(t_{n+1}, y(t_{n+1})) \quad n = 0, 1, 2, \dots$$

that used into $y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} y'(t) dt = \int_{t_n}^{t_{n+1}} f(t, y(t)) dt$ gives

$$y(t_1) \simeq y(t_0) + \Delta t f(t_1, y(t_1)) \simeq y_0 + \Delta t f(t_1, y_1) =: y_1$$

$$y(t_2) \simeq y(t_1) + \Delta t f(t_2, y(t_2)) \simeq y_1 + \Delta t f(t_2, y_2) =: y_2$$

$\vdots$

$$y(t_N) \simeq y(t_{N-1}) + \Delta t f(t_N, y(t_N)) \simeq y_{N-1} + \Delta t f(t_N, y_N) =: y_N$$

Examples of numerical schemes

The scheme becomes

$$\begin{cases} y_0 \text{ given} \\ y_{n+1} = y_n + \Delta t f(t_{n+1}, y_{n+1}) \quad n = 0, 1, \dots, N-1 \end{cases} \quad (IE)$$

This is called **IMPLICIT EULER** method or **BACKWARD EULER** method. Note that, at every time step, the unknown y_{n+1} in (IE) appears both on the left-hand side *and in the right-hand side*, and in order to perform the step we must *solve an equation* in the unknown y_{n+1} . Since f is in general non-linear, at each step, to find y_n we need to solve a non-linear equation (for example, with Newton method). The method is obviously more expensive than Explicit Euler.

Examples of numerical schemes

Example 3 As a third example we consider the quadrature formula

$$\int_c^d g(s)ds \simeq (d - c) \left(\frac{g(c) + g(d)}{2} \right) \quad (5)$$

(trapezoidal rule) that is better than the previous ones since it is exact whenever g is a polynomial of degree ≤ 1 . Applying it to our case we get

$$\int_{t_n}^{t_{n+1}} f(t, y(t))dt \simeq \frac{(t_{n+1} - t_n)}{2} \left(f(t_n, y(t_n)) + f(t_{n+1}, y(t_{n+1})) \right) \quad \forall n$$

The corresponding scheme becomes

$$\begin{cases} y_0 \text{ given} \\ y_{n+1} = y_n + \frac{\Delta t}{2} \left(f(t_n, y_n) + f(t_{n+1}, y_{n+1}) \right) \quad n = 0, 1, \dots, N-1 \end{cases}$$

Examples of numerical schemes

$$\begin{cases} y_0 \text{ given} \\ y_{n+1} = y_n + \frac{\Delta t}{2} \left(f(t_n, y_n) + f(t_{n+1}, y_{n+1}) \right) \quad n = 0, 1, \dots, N-1 \end{cases}$$

This is called **CRANK-NICOLSON** method. It is an implicit method (and hence, as the previous Implicit Euler, expensive) but it has a good accuracy, as we shall see.

Examples of numerical schemes

Example 4 If we replace, in the Crank-Nicolson scheme, y_{n+1} with $y_{n+1}^* = y_n + \Delta t f(t_n, y_n)$, that is, with the value predicted by Explicit Euler, we get rid of the implicit part and obtain a new explicit method, called **HEUN** method, which reads

$$\begin{cases} y_0 \text{ given} \\ y_{n+1}^* = y_n + \Delta t f(t_n, y_n) & (HEUN) \\ y_{n+1} = y_n + \frac{\Delta t}{2} \left(f(t_n, y_n) + f(t_{n+1}, y_{n+1}^*) \right) & n = 0, 1, \dots, N-1 \end{cases}$$

So we have two classes: **explicit methods**, and **implicit methods**.

In all cases we want the sequence $\{y_0, y_1, \dots, y_N\}$ to converge to the sequence $\{y_0, y(t_1), \dots, y(T)\}$.

If, given a method, we can prove that

$$\exists C > 0 \text{ such that } \max_n |y_n - y(t_n)| \leq C \Delta t^p$$

with C independent of Δt and $p > 0$, then we say that *the method is convergent*, and *the order of convergence is p* (the bigger p , the faster the convergence).

So we have two classes: **explicit methods**, and **implicit methods**.

In all cases we want the sequence $\{y_0, y_1, \dots, y_N\}$ to converge to the sequence $\{y_0, y(t_1), \dots, y(T)\}$.

If, given a method, we can prove that

$$\exists C > 0 \text{ such that } \max_n |y_n - y(t_n)| \leq C \Delta t^p$$

with C independent of Δt and $p > 0$, then we say that *the method is convergent*, and *the order of convergence is p* (the bigger p , the faster the convergence).

Theorem 1 (Lax)

*If a scheme is **consistent** and **stable**, then it is convergent, and the order of convergence is the order of consistency.*

We have however yet to define what *consistency* and *stability* are....

Intuitive explanation of *consistency* and *stability*

- the *consistency error* at a given time step measures the error which is created at that step; it is defined as *the error made when the scheme is applied to the exact solution of the problem*. It can be seen that the consistency error measures of how much the discrete scheme resembles the differential problem.
- *stability* measures how the error, created and accumulated during the previous steps, goes to the next step (is it amplified? does it decay?...)

Consistency of a scheme

Consistency is a measure of how much the discrete scheme resembles the differential problem: the *consistency error* is *the error made when the scheme is applied to the exact solution of the problem*.

The *consistency error* of a given scheme is defined as

$$\tau = \max_{n=1,\dots,N} |\tau_n|$$

where τ_n is the *local truncation error* at the step n , that is defined as:

$$\tau_n = \frac{y(t_{n+1}) - \tilde{y}_{n+1}}{\Delta t}$$

where $\tilde{y}_{n+1}$ is obtained applying one step of the method to the exact solution at the previous time instant $y(t_n)$. We say that the scheme is consistent of order p if:

$$\exists C > 0, \text{ independent of } \Delta t, \text{ s. t. } \tau \leq C \Delta t^p$$

We observe that if the numerical scheme is consistent, then $\tau \rightarrow 0$ for $\Delta t \rightarrow 0$.

Consistency of Explicit Euler

The exact solution fulfils: $y'(t_n) - f(t_n, y(t_n)) = 0$, $n = 0, 1, 2, \dots$

If we apply the Explicit Euler scheme to the exact solution we will have:

$$\tilde{y}_{n+1} = y(t_n) + \Delta t f(t_n, y(t_n))$$

Since $y'(t_n) = f(t_n, y(t_n))$, by Taylor expansion centered in t_n , there exists $z \in [t_n, t_{n+1}]$ such that

$$y(t_{n+1}) = y(t_n) + \Delta t y'(t_n) + \frac{\Delta t^2}{2} y''(z) = y(t_n) + \Delta t f(t_n, y(t_n)) + \frac{\Delta t^2}{2} y''(z)$$

Thus, the local truncation error becomes

$$\tau_n = \frac{y(t_{n+1}) - \tilde{y}_{n+1}}{\Delta t} = \frac{y(t_{n+1}) - y(t_n)}{\Delta t} - f(t_n, y(t_n)) = \frac{\Delta t}{2} y''(z),$$

and the consistency error is then

$$\tau = \max |\tau_n| \leq \frac{\Delta t}{2} \max_{t \in [t_0, T]} |y''(t)| = C \Delta t.$$

Thus, Explicit Euler scheme is consistent with **order of consistency 1**.

Consistency of Implicit Euler method

If we apply the Implicit Euler scheme to the exact solution we will have:

$$y_{n+1} = y(t_n) + \Delta t f(t_{n+1}, y_{n+1})$$

Since $f(t_{n+1}, y(t_{n+1})) = y'(t_{n+1})$, by Taylor expansion centered in t_{n+1} , there exists $z \in [t_n, t_{n+1}]$ such that

$$\begin{aligned} y(t_n) &= y(t_{n+1}) + (t_n - t_{n+1}) y'(t_{n+1}) + \frac{\Delta t^2}{2} y''(z) \\ &= y(t_{n+1}) - \Delta t f(t_{n+1}, y(t_{n+1})) + \frac{\Delta t^2}{2} y''(z) \end{aligned}$$

Hence we obtain

$$\begin{aligned} |\tau_n| &= \left| \frac{y(t_{n+1}) - y_{n+1}}{\Delta t} \right| = \frac{1}{\Delta t} |y(t_{n+1}) - y(t_n) - \Delta t f(t_{n+1}, y_{n+1})| \\ &= \frac{1}{\Delta t} \left| \Delta t f(t_{n+1}, y(t_{n+1})) - \frac{\Delta t^2}{2} y''(z) - \Delta t f(t_{n+1}, y_{n+1}) \right| \\ &\leq \left| \frac{\Delta t}{2} y''(z) \right| + |f(t_{n+1}, y(t_{n+1})) - f(t_{n+1}, y_{n+1})| \end{aligned}$$

Assuming f Lipschitz with respect to the second variable, with Lipschitz constant L , we have:

$$|\tau_n| \leq \frac{\Delta t}{2} |y''(z)| + L |y(t_{n+1}) - y_{n+1}| = \left| \frac{\Delta t}{2} y''(z) \right| + L \Delta t |\tau_n|$$

The it holds

$$(1 - L\Delta t) |\tau_n| \leq \frac{\Delta t}{2} |y''(z)|$$

For $\Delta t \leq 1/(2L)$, it holds

$$|\tau_n| \leq \Delta t |y''(z)|.$$

and

$$\tau = \max |\tau_n| \leq \Delta t \max_{t \in [t_0, T]} |y''(t)| = C \Delta t.$$

Thus, also Implicit Euler scheme is consistent with **order of consistency 1**.

A closer look at consistency error

We found that:

- for the explicit Euler methods, the consistency error is zero whenever $y'' \equiv 0$, that is, whenever the solution of the Cauchy Problem is a polynomial of degree up to 1.
- the consistency error for Heun method is zero whenever $y''' \equiv 0$, that is, whenever the solution of the Cauchy Problem is a polynomial of degree up to 2.

This suggests that *to have order of consistency p* means that *the scheme computes exactly the solution of the Cauchy Problem whenever this solution is a polynomial of degree up to p .*

This is true in general (no proof provided in this course) and is indeed another easier way of checking consistency of a scheme.

Consistency of implicit methods: review of explicit Euler

$$\frac{y_{n+1} - y_n}{\Delta t} - f(t_n, y_n) = 0 \quad \forall n$$

Applying this scheme to the exact solution of the Cauchy Problem means considering

$$\frac{y(t_{n+1}) - y(t_n)}{\Delta t} - y'(t_n)$$

which is in general $\neq 0$. Is the order of consistency 1? For this we should have the above $= 0$ when the solution of the Cauchy Problem is 1, t .

$$\text{When } y(t) = 1, y' = 0 = f, \implies \frac{1 - 1}{\Delta t} - 0 = 0;$$

$$\text{When } y(t) = t, y' = 1 = f, \implies \frac{t_{n+1} - t_n}{\Delta t} - 1 = 1 - 1 = 0;$$

Hence, the order of consistency of EE method is 1. Note that

$$\text{When } y(t) = t^2, y' = 2t = f, \implies \frac{t_{n+1}^2 - t_n^2}{\Delta t} - 2t_n = t_{n+1} - t_n \neq 0$$

Consistency of implicit methods: implicit Euler

$$\frac{y_{n+1} - y_n}{\Delta t} - f(t_{n+1}, y_{n+1}) = 0 \quad \forall n$$

Applying this scheme to the exact solution of the Cauchy Problem means considering

$$\frac{y(t_{n+1}) - y(t_n)}{\Delta t} - y'(t_{n+1})$$

which is in general $\neq 0$. Is the order of consistency 1? For this we should have the above $= 0$ when the solution of the Cauchy Problem is 1, t .

$$\text{When } y(t) = 1, y' = 0 = f, \implies \frac{1 - 1}{\Delta t} - 0 = 0;$$

$$\text{When } y(t) = t, y' = 1 = f, \implies \frac{t_{n+1} - t_n}{\Delta t} - 1 = 1 - 1 = 0;$$

Hence, the order of consistency of IE method is 1. Note that

$$\text{When } y(t) = t^2, y' = 2t = f, \implies \frac{t_{n+1}^2 - t_n^2}{\Delta t} - 2t_{n+1} = t_n - t_{n-1} \neq 0$$

Consistency of implicit methods: Crank-Nicolson

$$\frac{y_{n+1} - y_n}{\Delta t} - \frac{1}{2} \left(f(t_n, y_n) + f(t_{n+1}, y_{n+1}) \right) = 0 \quad \forall n$$

Applying this scheme to the exact solution means considering

$$\frac{y(t_{n+1}) - y(t_n)}{\Delta t} - \frac{1}{2} \left(y'(t_n) + y'(t_{n+1}) \right).$$

Let's check the above expression when the solution of the Cauchy Problem is $1, t, t^2$:

$$\text{When } y(t) = 1, y' = 0 = f, \implies \frac{1 - 1}{\Delta t} - \frac{1}{2}(0 + 0) = 0;$$

$$\text{When } y(t) = t, y' = 1 = f, \implies \frac{t_{n+1} - t_n}{\Delta t} - \frac{1}{2}(1 + 1) = 1 - 1 = 0;$$

$$\text{When } y(t) = t^2, y' = 2t = f, \implies \frac{t_{n+1}^2 - t_n^2}{\Delta t} - \frac{1}{2}(2t_n + 2t_{n+1}) = 0$$

Hence, the order of consistency of Crank-Nicolson is 2.

Absolute Stability

The concept of *stability* is another very important and useful concept whose precise definition has to be made precise at various occurrences. Roughly speaking, stability is what guarantees that the errors generated during a numerical procedure do not grow too much.

With Ode stability is a delicate issue, especially when the phenomenon under study has to be followed for a long time. To better see what happens, let us consider a simple model problem, for which we know the exact solution:

$$\begin{cases} y'(t) = ay(t) & t > 0 \\ y(0) = y_0 \end{cases} \quad a \in \mathbb{C}$$

which exact solution is

$$y(t) = y_0 e^{(\operatorname{Re} a)t} (\cos((\operatorname{Im} a)t) + i \sin((\operatorname{Im} a)t))$$

If $\operatorname{Re}(a) > 0$ the exact solution *grows* exponentially. We cannot expect (and we do not want!) that the discrete scheme remains bounded, and it is not even the case to discuss “stability”.

Instead, if $\operatorname{Re}(a) < 0$ the exact solution not only is bounded, but *decays* exponentially:

$$a < 0 \longrightarrow |y(t)| \leq |y_0| \quad \text{and} \quad \lim_{t \rightarrow \infty} |y(t)| = 0.$$

In this case we need to analyse the discrete schemes, and see whether the discrete solution decays too, and behaves like the exact solution. Hence, let $\operatorname{Re}(a) < 0$, and let $\{y_n\}$ be the sequence generated by a numerical scheme. Does $\{y_n\}$ satisfy the following relation?

$$a \in \mathbb{C} \text{ with } \operatorname{Re} a < 0 \longrightarrow |y_n| \leq |y_0| \quad \text{and} \quad \lim_{n \rightarrow \infty} |y_n| = 0?$$

If this happens, the scheme is called *Absolutely stable*, or *A-stable*.

Checking A-stability for Explicit Euler

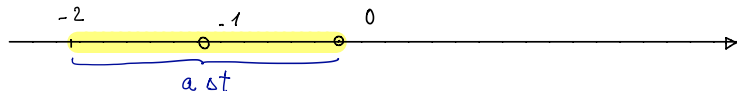
By applying **Explicit Euler** method to the model problem, we get
($y_{n+1} = y_n + \Delta t f(t_n, y_n)$ with $f(t_n, y_n) = ay_n$)

$$y_{n+1} = (1 + a\Delta t)y_n \quad n = 0, 1, \dots \implies y_n = y_0(1 + a\Delta t)^{n+1}.$$

The exact solution decays exponentially from the initial value y_0 , while the growth-decay factor for the discrete scheme is $G = 1 + a\Delta t$.

For having $\lim_{n \rightarrow \infty} |y_n| = 0$ we need $|G| < 1$. We observe that $|1 + a\Delta t|$ is the distance between -1 and $a\Delta t$, so $|G| < 1$ if and only if $a\Delta t$ is contained in the unit circle centered in -1 .

Stability for Explicit Euler: the real setting

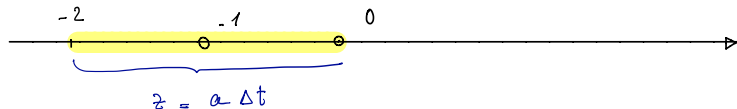


$$EE \text{ is stable} \iff |1 + a \Delta t| < 1$$

$$\iff -2 < a \Delta t < 0$$

$$\iff -\frac{2}{a} > \Delta t > 0 \quad (\text{since } a < 0)$$

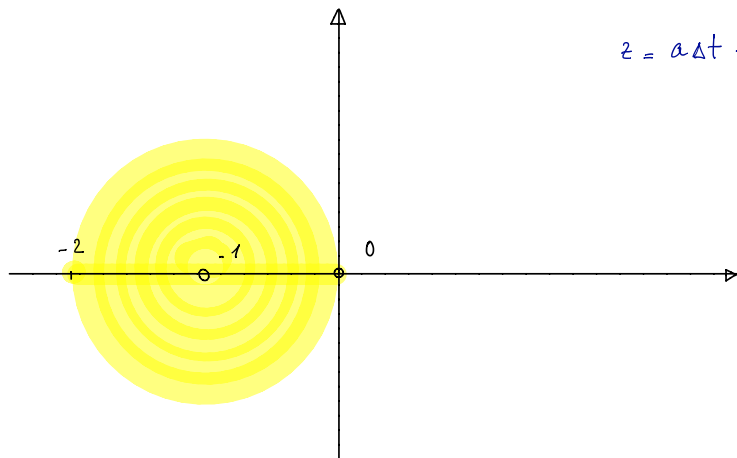
Stability for Explicit Euler: the real setting



$$\begin{aligned} \text{EE is stable} &\iff |1 + a \Delta t| < 1 \\ &\iff -2 < \underbrace{a \Delta t}_z < 0 \end{aligned}$$

Stability for Explicit Euler: the complex setting

$$z = a\Delta t \in \mathbb{C}$$



$$\begin{aligned} \text{EE is stable} &\iff |1 + \overbrace{a\Delta t}^z| < 1 \\ &\iff z \in \text{circle with radius 1 and center -1} \end{aligned}$$

It holds:

$$|1 + a\Delta t| < 1 \iff |1 + a\Delta t|^2 < 1$$

but we have

$$|1 + a\Delta t|^2 = (1 + a\Delta t)(1 + \bar{a}\Delta t) = 1 + 2\operatorname{Re}(a)\Delta t + |a|^2\Delta t^2$$

Thus

$$|1 + a\Delta t| < 1 \iff 0 < \Delta t < -\frac{2\operatorname{Re}(a)}{|a|^2} =: \text{A-Stability condition for EE}$$

This is the drawback of Explicit Euler scheme, and of all the explicit schemes: for small enough time steps the stability condition is satisfied, but when $\operatorname{Re}(a)$ is strongly negative (exactly the case of rapid decay in the true solution) we are compelled to keep Δt small.

Checking stability for Implicit Euler

Applying **Implicit Euler** method to the model problem gives
($y_{n+1} = y_n + \Delta t f(t_{n+1}, y_{n+1})$ with $f(t_{n+1}, y_{n+1}) = ay_{n+1}$)

$$y_{n+1} = y_n + a\Delta t y_{n+1} \quad n = 0, 1, \dots \implies y_{n+1} = \frac{1}{1 - a\Delta t} y_n = G y_n.$$

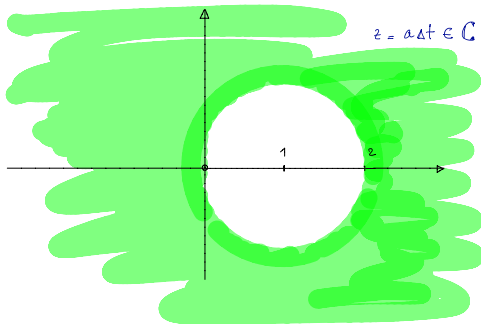
We have $|G| < 1 \iff |1 - a\Delta t| > 1$ and this happens as soon as $a\Delta t$ is outside the unit circle centered in 1.

If $\operatorname{Re}(a) < 0$, then $|G| < 1$ and we say that Implicit Euler is A-stable: the A-stability condition can be written as

$\text{If } \operatorname{Re} a < 0 \text{ then } |G| < 1 \quad =: \text{A-stability}$

However, for Implicit Euler method, the A-stability region is too large: if $a\Delta t$ is outside the unit circle centered in 1, y_n decays even if the exact solution do not.

Stability for Implicit Euler: the complex setting



The scheme is A-stable:

$$|G| < 1 \Leftrightarrow |1 - a\Delta t| > 1 \Leftrightarrow a\Delta t \text{ is outside the circle above} \Leftrightarrow \operatorname{Re}(a) < 0$$

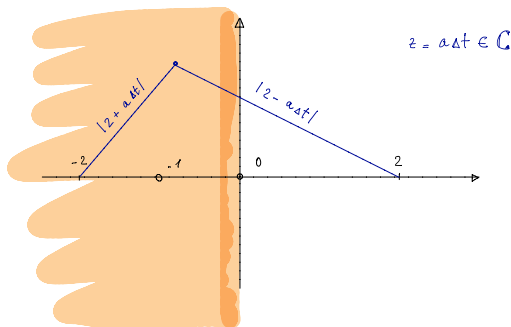
Checking stability for Crank-Nicolson

Crank-Nicolson gives

$$y_{n+1} = y_n + \frac{a\Delta t}{2}(y_n + y_{n+1}) \quad n = 0, 1, \dots \implies y_{n+1} = \frac{1 + \frac{a\Delta t}{2}}{1 - \frac{a\Delta t}{2}} y_n = G y_n.$$

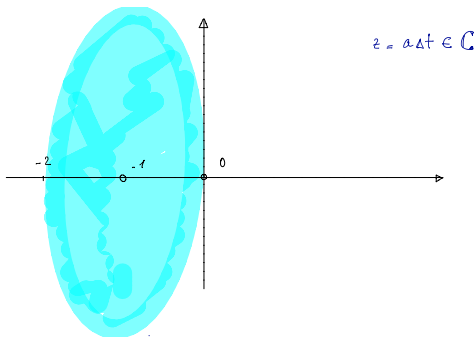
The scheme is A-stable:

$$|G| < 1 \Leftrightarrow |2 + a\Delta t| < |2 - a\Delta t| \Leftrightarrow \operatorname{Re}(a)\Delta t < 0 \Leftrightarrow \operatorname{Re}(a) < 0$$



Checking stability for Heun

Heun method is not A-stable, only **conditionally A-stable** like Explicit Euler. In fact, $G = 1 + a\Delta t + \frac{(a\Delta t)^2}{2}$, and the condition $|G| < 1$ is satisfied if $\Delta t < (2/|a|)$.



Stability regions: comparison

For each of these methods we have defined the stability region in the complex plane as

$$A := \{a\Delta t \in \mathbb{C} : \lim_{n \rightarrow \infty} |y_n| = 0\} \equiv \{a\Delta t \in \mathbb{C} : |G| < 1\}$$

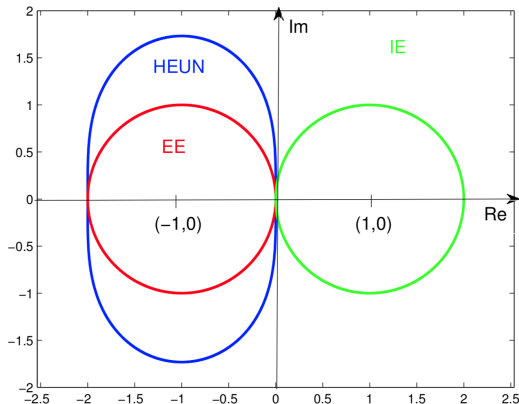
and compared it with the stability region of the continuous problem: the half plane $\operatorname{Re} a < 0$.

For Explicit Euler $A = \{a\Delta t \in \mathbb{C} : |1 + a\Delta t| < 1\}$ is a circle with center $(-1, 0)$ and radius 1 (too small!).

For Implicit Euler $A = \{a\Delta t \in \mathbb{C} : |1 - a\Delta t| > 1\}$ is the whole plane minus a circle with center $(1, 0)$ and radius 1 (too big!)

For Crank-Nicolson the region is the left-half plane, exactly as for the true solution (the best you can have).

Stability regions



A-stability regions for EE (the region inside the red circle), Heun (the region inside the blue ellipse), IE (green, the region outside the circle with center $(1, 0)$ and radius 1

Conclusions (for the four basic methods)

In general, explicit schemes are **never A-stable**, only conditionally A-stable, meaning that to satisfy the A-stability property they need to proceed by small time steps. Some implicit schemes are A-stable.

For the method we have considered:

Method	Consistency	Stability
Explicit Euler	yes, order 1	conditionally A-stable
Implicit Euler	yes, order 1	A-stable
Crank-Nicolson	yes, order 2	A-stable
Heun	yes, order 2	conditionally A-stable

Generalizing Heun idea: Runge-Kutta methods

The celebrated Runge-Kutta methods are compound 1-step methods.

The basic idea is very simple: choose a high precision quadrature formula for $\int f$ on each interval $[t_n, t_{n+1}]$. Then, since the values at the quadrature nodes are not known, we predict them somehow (and this is where the detailed description could become quite complicated).

The simplest explicit RK is Heun: the starting point is the trapezoidal rule for $\int_{t_n}^{t_{n+1}} f$, and since we want to go explicit, instead of the value y_{n+1} (that would be needed in the trapezoidal rule) we use $y_n + \Delta t f(t_n, y_n)$, that is, the value predicted by Explicit Euler.

Runge-Kutta methods

Heun scheme:

$$\begin{cases} y_0 \text{ given} \\ y_{n+1}^* = y_n + \Delta t f(t_n, y_n) \\ y_{n+1} = y_n + \frac{\Delta t}{2} \left(f(t_n, y_n) + f(t_{n+1}, y_{n+1}^*) \right) \end{cases} \quad n = 0, 1, \dots, N-1$$

Denoting by K_1 and K_2 the values of f at the two nodes t_n and $t_{n+1} = t_n + \Delta t$ we can rewrite Heun method as:

$$\begin{aligned} K_1 &= f(t_n, y_n), \quad K_2 = f(t_n + \Delta t, y_n + \Delta t K_1) \\ y_{n+1} &= y_n + \frac{\Delta t}{2} (K_1 + K_2) \quad n = 0, 1, \dots \end{aligned}$$

Runge-Kutta methods

The more famous version of Runge-Kutta, **RK4**, is compounded *four times*, it is based on *Simpson rule*, and achieves order $p = 4$. Let us see how it is obtained.

Simpson rule uses on the interval $[t_n, t_{n+1}]$ three nodes t_n , $t_{n+1/2} = t_n + \Delta t/2$, and $t_{n+1} = t_n + \Delta t$, and would give

$$\int_{t_n}^{t_{n+1}} f(t, y(t)) dt \simeq \frac{\Delta t}{6} (f_n + 4f_{n+1/2} + f_{n+1}).$$

$$(f_n = f(t_n, y_n), f_{n+1/2} = f(t_{n+1/2}, y_{n+1/2}), f_{n+1} = f(t_{n+1}, y_{n+1}))$$

The value f_n is known, so we set $K_1 = f(t_n, y_n)$; then write $4f_{n+1/2}$ as $2f_{n+1/2}^{(1)} + 2f_{n+1/2}^{(2)}$, and we choose two different “predictions” for $y_{n+1/2}$:

$$f_{n+1/2}^{(1)} = f(t_{n+1/2}, y_n + \frac{\Delta t}{2} K_1) =: K_2$$

$$f_{n+1/2}^{(2)} = f(t_{n+1/2}, y_n + \frac{\Delta t}{2} K_2) =: K_3$$

RK4

We had

$$K_1 = f(t_n, y_n), \quad K_2 = f_{n+1/2}^{(1)} = f(t_{n+1/2}, y_n + \frac{\Delta t}{2} K_1),$$

$$K_3 = f_{n+1/2}^{(2)} = f(t_{n+1/2}, y_n + \frac{\Delta t}{2} K_2)$$

For f_{n+1} we take here $f_{n+1} = f(t_{n+1}, y_n + \Delta t K_3)$ (although other choices could have been possible), so that the scheme is:

$$K_1 = f(t_n, y_n), \quad K_2 = f(t_{n+1/2}, y_n + \frac{\Delta t}{2} K_1)$$

$$K_3 = f(t_{n+1/2}, y_n + \frac{\Delta t}{2} K_2), \quad K_4 = f(t_{n+1}, y_n + \Delta t K_3)$$

$$y_{n+1} = y_n + \frac{\Delta t}{6} (K_1 + 2K_2 + 2K_3 + K_4) \quad n = 0, 1, \dots$$

Explicit Runge-Kutta

The family of explicit Runge-Kutta methods is a generalisation of the RK4 scheme above:

$$y_0 \text{ given} \quad y_{n+1} = y_n + \Delta t \sum_{i=1}^s b_i K_i, \quad n = 0, 1, \dots \quad (1)$$

where

$$K_i = f(t_n + c_i \Delta t, y_n + \Delta t \sum_{j=1}^{i-1} a_{ij} K_j)$$

To specify a particular method one needs to provide the integer s (the number of stages), and the coefficients a_{ij} , b_i , c_i . The matrix $[a_{ij}]$ is called *Runge-Kutta matrix*, while the b_i and c_i are called *weights* and *nodes*, respectively. These coefficients are usually arranged in the *Butcher tableau*

Butcher tableau

0					
c_2	$a_{2,1}$				
c_3	$a_{3,1}$	$a_{3,2}$			
$\vdots$		$\vdots$	$\ddots$		
c_s	$a_{s,1}$	$a_{s,2}$	$\cdots$	$a_{s,s-1}$	
	b_1	b_2	$\cdots$	b_{s-1}	b_s

Consistency conditions

For consistency the coefficients must verify algebraic conditions;

a RK scheme is consistent **iff** $\sum_{i=1}^s b_i = 1$.

(This condition is always verified since the b_i are the weights of the quadrature formula used, which has to be exact at least on constants).

For higher order of consistency, other relations must be satisfied. For instance, for a 2 stage explicit RK to have order 2 we need, together with $b_1 + b_2 = 1$, also $b_2 c_2 = 1/2$ (check!)

Accuracy and stages of RK ** NOT FOR THE EXAM **

Theorem 1

An explicit s -stages Runge-Kutta method cannot have order of accuracy p greater than s . Moreover, there are no known explicit s -stages RK methods with order $p = s$ for $s \geq 5$.

order p	1	2	3	4	5	6	7	8
s_{min}	1	2	3	4	6	7	9	11

RK methods are very successful and widely used in the codes for their ductility: the time step can easily be modified from one interval to another if needed, the initial value y_0 is all what is needed to start the method, and they have high accuracy.

A first multistep method

Let us start with an example. Let $t_0, t_1, \dots, t_N = T$ be a set of *equally spaced* points in $[t_0, T]$, and let $\Delta t = \frac{T - t_0}{N}$ be the time step (this time the points **must** be equally spaced).

We want to construct an explicit scheme of order 2 going back two steps:

$$y_{n+1} = y_n + \Delta t \left(\alpha f(t_n, y_n) + \beta f(t_{n-1}, y_{n-1}) \right), n = 1, 2, \dots$$

This requires values at time $t_{n-1} = t_0 + (n-1)\Delta t$ as well as at time t_n . Therefore the initial value y_0 is not enough to start the procedure and we need to compute y_1 with a 1-step method. Then we have to find α and β such that the scheme has order 2.

Choosing the parameters in a multistep scheme....

The starting point is the same as for 1-step methods:

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} y'(t) dt = \int_{t_n}^{t_{n+1}} f(t, y(t)) dt. \quad (*)$$

The function f is then approximated by its Lagrange interpolant polynomial of degree ≤ 1 with respect to the nodes t_{n-1} and t_n :

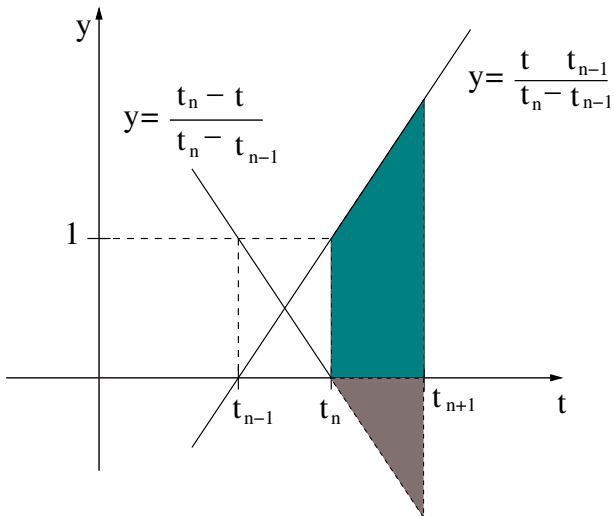
$$f(t, y(t)) \simeq \Pi_1(t) := \frac{t - t_{n-1}}{t_n - t_{n-1}} f(t_n, y_n) + \frac{t_n - t}{t_n - t_{n-1}} f(t_{n-1}, y_{n-1})$$

Consequently,

$$\int_{t_n}^{t_{n+1}} f(t, y(t)) dt \sim \int_{t_n}^{t_{n+1}} \Pi_1(t) dt = \frac{3}{2} \Delta t f(t_n, y_n) - \frac{1}{2} \Delta t f(t_{n-1}, y_{n-1})$$

The order accuracy is 2: if $f \in \mathbb{P}_1$, then $f \equiv \Pi_1$ and $\int f$ is computed exactly. On the other hand, $f \in \mathbb{P}_1$ implies $y \in \mathbb{P}_2$.

Integral of Π_1



Generalization

With the same approach we can design schemes that use values computed at p previous steps and are p -accurate:

on each interval $[t_n, t_{n+1}]$ the function f is replaced by its Lagrange interpolant polynomial (of degree $\leq p-1$) with respect to the p points $t_n, t_{n-1}, \dots, t_{n+1-p}$:

$$f(t, y(t)) \simeq \Pi_{p-1}(t) \quad \text{with } \Pi_{p-1} \in \mathbb{P}_{p-1} \text{ verifying}$$

$$\Pi_{p-1}(t_n) = f(t_n, y_n),$$

$$\Pi_{p-1}(t_{n-1}) = f(t_{n-1}, y_{n-1}),$$

$\dots$

$$\Pi_{p-1}(t_{n+1-p}) = f(t_{n+1-p}, y_{n+1-p}).$$

The $\int_{t_n}^{t_{n+1}} \Pi_{p-1}(t) dt$ is then computed exactly; to complete the p -step scheme we will need to compute $p-1$ “initial values” $y_1, y_2, \dots, y_{p-1}$ in addition to y_0 (for instance with a 1-step method).

Adams-Bashforth schemes

The resulting scheme will be:

$$\begin{cases} y_0 \text{ given, } y_1, y_2, \dots, y_{p-1} \text{ to be computed} \\ y_{n+1} = y_n + \Delta t \left(c_1 f_n + c_2 f_{n-1} + \dots + c_p f_{n+1-p} \right), \\ n = p-1, p, p+1, \dots \end{cases} \quad (2)$$

where $\Delta t c_1, \Delta t c_2, \dots$ are the integrals of the characteristic Lagrange polynomials, and $f_n = f(t_n, y_n)$, $f_{n-1} = f(t_{n-1}, y_{n-1})$ and so on.

The multistep methods obtained in this way are “**Adams-Bashforth**” methods: they are **explicit**, p -accurate. In Table 1 below the coefficients of the first four schemes.

Adams-Bashforth schemes ** NOT FOR THE EXAM **

	c_1	c_2	c_3	c_4
$p = 1$	1			
$p = 2$	$3/2$	$-1/2$		
$p = 3$	$23/12$	$-16/12$	$5/12$	
$p = 4$	$55/24$	$-59/24$	$37/24$	$-9/24$

Table: First Adams-Bashforth schemes of order p

Adams-Moulton

Note: A similar construction gives **implicit methods**, called “**Adams-Moulton**”. Compared with (2) they have an extra term $c_0 f_{n+1}$ at the new time level. Properly chosen, that adds one extra order of accuracy (as it did for the Crank-Nicolson scheme).

	c_1	c_2	c_3	c_4	A-stability	order
$p = 0$	1				yes	Δt
$p = 1$	1/2	1/2			yes	Δt^2
$p = 2$	5/12	8/12	-1/12		no	Δt^3
$p = 3$	9/24	19/24	-5/24	1/24	no	Δt^4

Table: First four Adams-Moulton schemes: order $p + 1$ ** NOT FOR THE EXAM **

Consistency of multistep meth.s ** NOT FOR EXAM **

As we have already seen when checking consistency of Heun method, this amounts to impose that the local truncation error is zero when the exact solution of the Cauchy Problem is a polynomial of degree up to 2: hence we must require that $\tau_n \equiv 0$ when the solution of the Cauchy Problem is $1, t, t^2$.

$$\tau_n = \frac{y(t_{n+1}) - y(t_n)}{\Delta t} - \left(\alpha f(t_n, y(t_n)) + \beta f(t_{n-1}, y(t_{n-1})) \right) \quad n = 1, 2, \dots$$

$$\text{When } y(t) = 1, y' = 0 = f, \implies \tau_n = \frac{1 - 1}{\Delta t} = 0;$$

$$\begin{aligned} \text{When } y(t) = t, y' = 1 = f, \implies \tau_n &= \frac{t_{n+1} - t_n}{\Delta t} - (\alpha + \beta) = 0; \\ \implies \alpha + \beta &= 1 \implies \beta = 1 - \alpha; \end{aligned}$$

$$\begin{aligned} \text{When } y(t) = t^2, y' = 2t = f, \implies \tau_n &= \frac{t_{n+1}^2 - t_n^2}{\Delta t} - (2\alpha t_n + 2\beta t_{n-1}) \\ &= t_{n+1} + t_n - 2(\alpha t_n + \beta t_{n-1}) = 0 \\ \implies \alpha t_n + \beta t_{n-1} &= \frac{t_{n+1} + t_n}{2}. \end{aligned}$$

Consistency of multistep meth.s ** NOT FOR EXAM **

$$\beta = 1 - \alpha, \quad \alpha t_n + \beta t_{n-1} = \frac{t_{n+1} + t_n}{2}$$

Inserting $\beta = 1 - \alpha$ in the second equation we have

$$\begin{aligned}\alpha(t_n - t_{n-1}) &= \frac{t_n + t_{n+1} - 2t_{n-1}}{2} \\ &= \frac{n\Delta t + (n+1)\Delta t - 2(n-1)\Delta t}{2} = \frac{3\Delta t}{2}.\end{aligned}$$

Therefore we obtain $\alpha = \frac{3}{2}$ and $\beta = -\frac{1}{2}$. The 2-step scheme is then

$$\begin{cases} y_0 \text{ given, } y_1 \text{ to be computed} \\ y_{n+1} = y_n + \Delta t \left(\frac{3}{2}f(t_n, y_n) - \frac{1}{2}f(t_{n-1}, y_{n-1}) \right), n = 1, 2, \dots \end{cases}$$

By construction the scheme is consistent of order 2. Being explicit, it will not be A-stable, only conditionally A-stable.

More schemes ** NOT FOR THE EXAM **

Another way of constructing explicit methods with a good accuracy is to choose an implicit scheme, and make it explicit with a very simple and successful trick:

- P: use the explicit formula to *predict* a new y_{n+1}^*
- E: use y_{n+1}^* to *evaluate* $f_{n+1}^* = f(t_{n+1}, y_{n+1}^*)$
- C: use f_{n+1}^* in the implicit formula to *correct* the new y_{n+1}

This is the **predictor-corrector** method (see Heun method). The stability is much improved if there is another E step to evaluate f_{n+1} with the corrected y_{n+1} . So PECE is the basic sequence continue the correction repeating the CE steps until y_{n+1} no longer changes: it has reached its final value for the implicit formula. Often two or three corrections are enough, and this is faster than using Newton's method in solving a single step of the implicit method.

Systems of Ode's

It is much more common to have systems of differential equations than a single equation. The unknown is now a vector $\underline{Y}(t)$, and so is the right-hand side $\underline{F}(t, \underline{Y}(t))$. The problem is: find $\underline{Y}(t)$ solution of

$$\begin{cases} \underline{Y}'(t) = \underline{F}(t, \underline{Y}(t)) & t \in [t_0, T] \\ \underline{Y}(t_0) = \underline{Y}^{(0)}. \end{cases} \quad (3)$$

with

$$\underline{Y}(t) = \begin{bmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_N(t) \end{bmatrix}, \quad \underline{F}(t, \underline{Y}(t)) = \begin{bmatrix} f_1(t, \underline{Y}(t)) \\ f_2(t, \underline{Y}(t)) \\ \vdots \\ f_N(t, \underline{Y}(t)) \end{bmatrix}, \quad \underline{Y}^{(0)} = \begin{bmatrix} y_1^{(0)} \\ y_2^{(0)} \\ \vdots \\ y_N^{(0)} \end{bmatrix}$$

Systems of Ode's

The numerical schemes used for a single equation apply directly to systems of Ode's.

For instance, the two Euler methods become:

$$(EE) \quad \begin{cases} \underline{Y}^{(0)} \text{ given} \\ \underline{Y}^{(n+1)} = \underline{Y}^{(n)} + \Delta t \underline{F}(t_n, \underline{Y}^{(n)}) \end{cases} \quad n = 0, 1, \dots$$

Ex: $N = 2$ equations, and 2 unknowns y_1, y_2 :

$$\begin{cases} y_1^{(0)}, y_2^{(0)} \text{ given} \\ y_1^{(n+1)} = y_1^{(n)} + \Delta t f_1(t_n, y_1^{(n)}, y_2^{(n)}) \\ y_2^{(n+1)} = y_2^{(n)} + \Delta t f_2(t_n, y_1^{(n)}, y_2^{(n)}) \end{cases} \quad n = 0, 1, \dots$$

Systems of Ode's

$$(IE) \quad \begin{cases} \underline{Y}^{(0)} \text{ given} \\ \underline{Y}^{(n+1)} = \underline{Y}^{(n)} + \Delta t \underline{F}(t_{n+1}, \underline{Y}^{(n+1)}) \quad n = 0, 1, \dots \end{cases}$$

Ex: $N = 2$ equations, and 2 unknowns y_1, y_2 :

$$\begin{cases} y_1^{(0)}, y_2^{(0)} \text{ given} \\ y_1^{(n+1)} = y_1^{(n)} + \Delta t f_1(t_{n+1}, y_1^{(n+1)}, y_2^{(n+1)}) \quad n = 0, 1, \dots \\ y_2^{(n+1)} = y_2^{(n)} + \Delta t f_2(t_{n+1}, y_1^{(n+1)}, y_2^{(n+1)}) \quad n = 0, 1, \dots \end{cases}$$

much more expensive now: at each step, to go from $\underline{Y}^{(n)}$ to $\underline{Y}^{(n+1)}$ requires the solution of a **non-linear system**!

** NOT FOR THE EXAM **

$$\underline{Y}^{(n+1)} = \underline{Y}^{(n)} + \Delta t \underline{F}(t_{n+1}, \underline{Y}^{(n+1)})$$

find $\underline{X}$ such that

$$\underline{G}(\underline{X}) := \underline{X} - \Delta t \underline{F}(t_{n+1}, \underline{X}) - \underline{Y}^{(n)} = 0,$$

and set $\underline{Y}^{(n+1)} := \underline{X}$

Newton would give:

$$\begin{cases} \underline{X}^{(0)} \text{ given} \\ J_{\underline{G}[\underline{X}^{(0)}]} \underline{\delta X} = -\underline{G}(\underline{X}^{(0)}) \\ \underline{X}^{(1)} = \underline{X}^{(0)} + \underline{\delta X} \end{cases}$$

$$\underline{X}^{(0)} = ?? \quad \text{for example: } \underline{X}^{(0)} = \underline{Y}^{(n)} + \Delta t \underline{F}(t_n, \underline{Y}^{(n)}) \quad (EE)$$

** NOT FOR THE EXAM **

Or you could use *PECE* with a few cycles of *CE*:

$$P : \underline{X}^{(0)} = \underline{Y}^{(n)} + \Delta t \underline{F}(t_n, \underline{Y}^{(n)}) \quad (EE)$$

$$E : \underline{F}^{(0)}(t_{n+1}, \underline{X}^{(0)})$$

$$C : \underline{X}^{(1)} = \underline{Y}^{(n)} + \Delta t \underline{F}^{(0)}(t_{n+1}, \underline{X}^{(0)})$$

$$E : \underline{F}^{(1)}(t_{n+1}, \underline{X}^{(1)})$$

$$C : \underline{X}^{(2)} = \underline{Y}^{(n)} + \Delta t \underline{F}^{(1)}(t_{n+1}, \underline{X}^{(1)})$$

$$E : \underline{F}^{(2)}(t_{n+1}, \underline{X}^{(2)})$$

Then set:

$$\underline{Y}^{(n+1)} = \underline{Y}^{(n)} + \Delta t \underline{F}^{(2)}(t_{n+1}, \underline{X}^{(2)})$$

Lack of A-stability ** NOT FOR THE EXAM **

For a single equation the lack of A-stability is not a major drawback: to have a good accuracy small Δt have to be used. Instead for systems it could be more severe when the problem has different time scales.

$$\underline{Y}'(t) = A\underline{Y}(t)$$

the eigenvalues λ_j of the square matrix A take the place of the single number a .

Explicit Euler would give

$$\underline{Y}^{(n+1)} = (I + \Delta t A)\underline{Y}^{(n)} \quad \forall n \implies \underline{Y}^{(n+1)} = (I + \Delta t A)^{n+1}\underline{Y}^{(0)}$$

The growth factor is now a matrix

$$G = (I + \Delta t A) \quad \text{with eigenvalues } g_j = 1 + \Delta t \lambda_j.$$

Lack of A-stability ** NOT FOR THE EXAM **

Suppose that both A and G are diagonalised. Then,

- each component of the discrete solution grows like g_j^n ,
- each component of the continuous solution grows like $e^{\lambda_j t}$.

The continuous solution is stable if all the λ_j are negative (or $\text{Re } \lambda_j < 0$): hence $e^{\lambda_j t} \rightarrow 0$ for $t \rightarrow \infty$.

The discrete solution is stable if all the $|g_j| < 1$, so that $g_j^n \rightarrow 0$ for $n \rightarrow \infty$.

If the problem has different time scales we are in trouble...

Since Δt is the same for all the components, its size is controlled by the most negative eigenvalue, which corresponds to the fastest decay and dies out first in the true solution.

Stiff systems ** NOT FOR THE EXAM **

When the matrix A has eigenvalues with very different magnitude, the system is called **stiff**.

Let us see a simple example.

$$\underline{Y}'(t) = \begin{bmatrix} -2 & 1 \\ 0 & -100 \end{bmatrix} \underline{Y}(t) \longrightarrow \begin{aligned} y_1'(t) &= -2y_1(t) + y_2(t) \\ y_2'(t) &= -100y_2(t) \end{aligned}$$

solution: $y_1(t) \simeq e^{-2t}$, $y_2(t) \simeq e^{-100t}$.

If we use Explicit Euler method we need

$$\Delta t < \frac{2}{|\lambda_1|} = 1 \quad \text{and} \quad \Delta t < \frac{2}{|\lambda_2|} = \frac{1}{50}.$$

Stability requires then $\Delta t < \frac{1}{50}$ even though it is e^{-2t} that controls the true solution: in fact, y_2 decays like e^{-100t} and dies out very fast, but its presence forces us to proceed by small time steps even when it has virtually disappeared and we are interested in following the e^{-2t} component.

Matlab functions

Most commonly used Matlab functions:

Non stiff problems:

ode23 (low order RK), ode45 (medium order RK), ode113 (variable order)

Stiff: ode15s (low to medium order), ode23s (low order RK)

Exercises NMES

Exercise 1

Apply the Gaussian elimination method, without pivoting, to solve the linear system $Ax = b$, where

$$\begin{matrix} (r_1) \\ (r_2) \\ (r_3) \end{matrix} \begin{bmatrix} 2 & 4 & 10 \\ 2 & 6 & 20 \\ 1 & 4 & 18 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -6 \\ 12 \\ 18 \end{bmatrix}$$

showing the intermediate computations.

Solution: First we eliminate the first column of A under the diagonal term $a_{1,1}$. Compute $l_{2,1} = \frac{a_{2,1}}{a_{1,1}} = 1$ and $l_{3,1} = \frac{a_{3,1}}{a_{1,1}} = \frac{1}{2}$. Then perform

$$\begin{matrix} (r_1) \\ (r_2) = (r_2) - l_{2,1} \cdot (r_1) \\ (r_3) = (r_3) - l_{3,1} \cdot (r_1) \end{matrix} \begin{bmatrix} 2 & 4 & 10 \\ 0 & 2 & 10 \\ 0 & 2 & 13 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -6 \\ 18 \\ 21 \end{bmatrix}$$

In the second step of the Gaussian elimination we eliminate the second column of A under the diagonal term $a_{2,2}$. Compute $l_{3,2} = \frac{a_{3,2}}{a_{2,2}} = 1$. Then perform

$$\begin{matrix} (r_1) \\ (r_2) \\ (r_3) = (r_3) - l_{3,2} \cdot (r_2) \end{matrix} \begin{bmatrix} 2 & 4 & 10 \\ 0 & 2 & 10 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -6 \\ 18 \\ 3 \end{bmatrix}$$

Finally compute the solution by *back-substitution* method:

$$3x_3 = 3; \rightarrow x_3 = 1;$$

$$2x_2 + 10x_3 = 18; \rightarrow 2x_2 = 18 - 10; \rightarrow x_2 = 4;$$

$$2x_1 + 4x_2 + 10x_3 = -6; \rightarrow 2x_1 = -6 - 10 - 16; \rightarrow x_1 = -16;$$

Finally the solution of the linear system is $x = \begin{bmatrix} -16 \\ 4 \\ 1 \end{bmatrix}$.

Exercise 2

Write the LU factorization, without pivoting, of:

$$\begin{matrix} (r_1) \\ (r_2) \\ (r_3) \end{matrix} \begin{bmatrix} 2 & 4 & 4 \\ 1 & 5 & 7 \\ 3 & 12 & 18 \end{bmatrix}$$

showing the intermediate computations.

Solution: The steps are the same as for the Gaussian elimination method. Let $L = I_n$ and compute the entries in the first column of L while eliminating the elements in the first column of A below the diagonal: $l_{2,1} = \frac{a_{2,1}}{a_{1,1}} = \frac{1}{2}$ and $l_{3,1} = \frac{a_{3,1}}{a_{1,1}} = \frac{3}{2}$. Now replace the row (r_2) and (r_3) with

$$(r_2) - l_{2,1} \cdot (r_1) = (1, 5, 7) - \frac{1}{2} \cdot (2, 4, 4) = (1, 5, 7) - (1, 2, 2) = (0, 3, 5)$$

$$(r_3) - l_{3,1} \cdot (r_1) = (3, 12, 18) - \frac{3}{2} \cdot (2, 4, 4) = (3, 12, 18) - (3, 6, 6) = (0, 6, 12)$$

respectively. Now the matrix A becomes

$$\begin{aligned} (r_1) &:= (r_1) \\ (r_2) &:= (r_2) - l_{2,1} \cdot (r_1) \\ (r_3) &:= (r_3) - l_{3,1} \cdot (r_1) \end{aligned} \begin{bmatrix} 2 & 4 & 4 \\ 0 & 3 & 5 \\ 0 & 6 & 12 \end{bmatrix}$$

and L is

$$\begin{bmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ 3/2 & 0 & 1 \end{bmatrix}.$$

Now repeat the computations above to eliminate the portion of the second row of A below the diagonal. So $l_{3,2} = \frac{a_{3,2}}{a_{2,2}} = 2$; replace now the third row (r_3) with $(r_3) - l_{3,2}(r_2)$.

$$(r_3) - l_{3,2} \cdot (r_2) = (0, 6, 12) - 2 \cdot (0, 3, 5) = (0, 6, 12) - (0, 6, 10) = (0, 0, 2).$$

So the matrix A has now become

$$\begin{aligned} (r_1) &:= (r_1) \\ (r_2) &:= (r_2) \\ (r_3) &:= (r_3) - l_{3,2} \cdot (r_2) \end{aligned} \begin{bmatrix} 2 & 4 & 4 \\ 0 & 3 & 5 \\ 0 & 0 & 2 \end{bmatrix}$$

and L is

$$\begin{bmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ 3/2 & 2 & 1 \end{bmatrix}.$$

Exercise 3

Compute the linear regression $r(x) = c_0 + c_1x$ for the set of points

$$(-3, 0), (-2, 0), (-1, 0), (1, 1), (2, 2), (3, 4).$$

Solution: To compute the solution recall the least square linear problem

$$\begin{bmatrix} m & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m y_i \\ \sum_{i=1}^m y_i x_i \end{bmatrix}.$$

which in this case is

$$\begin{bmatrix} 6 & 0 \\ 0 & 28 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 7 \\ 17 \end{bmatrix},$$

gives the solution which is

$$c_0 = \frac{7}{6}, \quad c_1 = \frac{17}{28}.$$

Exercise 4

Compute the quadratic least-square approximation $r(x) = c_0 + c_1x + c_2x^2$ for the set of points

$$(-2, \frac{5}{2}), (-1, 0), (0, -1), (1, 0), (2, \frac{5}{2}).$$

Solution: Here we want to minimize the quadratic function

$$F(c_0, c_1, c_2) = \sum_{i=1}^m (F(x_i) - c_0 - c_1 \cdot x_i - c_2 \cdot x_i^2)^2.$$

The problem is equivalent to solve

$$(LS2) \begin{bmatrix} m & \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 & \sum_{i=1}^m (x_i)^3 \\ \sum_{i=1}^m (x_i)^2 & \sum_{i=1}^m (x_i)^3 & \sum_{i=1}^m (x_i)^4 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m y_i \\ \sum_{i=1}^m x_i y_i \\ \sum_{i=1}^m (x_i)^2 y_i \end{bmatrix}.$$

For the given set of points, it holds

$$(LS2) \begin{bmatrix} 5 & 0 & 10 \\ 0 & 10 & 0 \\ 10 & 0 & 34 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 20 \end{bmatrix}.$$

We get immediately $c_1 = 0$. And then

$$\begin{cases} 5c_0 + 10c_2 = 4 \\ 10c_0 + 34c_2 = 20 \end{cases} \rightarrow \begin{cases} 5c_0 + 10c_2 = 4 \\ 14c_2 = 12 \end{cases} \rightarrow \begin{cases} c_0 + = \frac{4}{5} - \frac{60}{35} = -\frac{32}{35} \\ c_2 = \frac{6}{7} \end{cases}$$

Therefore the solution is $r(x) = -\frac{32}{35} + \frac{6}{7}x^2$.

Exercise 5

Starting from $x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$, compute 2 iterations of the Jacobi method applied to the system $Ax = b$, where

$$A = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 2 \end{bmatrix} \quad b = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

Solution: Recall that for iterative methods, we can split $A = M - N$, so that the general step of an iterative method is

$$x^{(k+1)} = x^{(k)} + M^{-1}r^{(k)}$$

In the case of Jacobi method, $M = \text{diag}(A)$

$$M = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

Before starting, compute the residual $r^{(0)}$

$$r^{(0)} = b - Ax^{(0)} = b = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

First iteration

Firstly, we have to compute $M^{-1}r^{(0)}$, which amounts to solving the linear system

$$Mu = r^{(0)}.$$

Since M is diagonal, it is easy to solve the linear system, obtaining the solution u

$$u = \begin{bmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \end{bmatrix},$$

so $x^{(1)} = x^{(0)} + u = (1/2, 0, 1/2)^T$. Then compute the residual $r^{(1)} = b - Ax^{(1)}$

$$r^{(1)} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 2 \end{bmatrix} \begin{bmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \begin{bmatrix} \frac{3}{2} \\ 0 \\ \frac{3}{2} \end{bmatrix} = - \begin{bmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \end{bmatrix}$$

Second iteration

As before, compute the solution of the linear system $Mu = r^{(1)}$, that yields

$$x^{(2)} = x^{(1)} + u = \begin{bmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \end{bmatrix} - \begin{bmatrix} \frac{1}{4} \\ 0 \\ \frac{1}{4} \end{bmatrix} = \begin{bmatrix} \frac{1}{4} \\ 0 \\ \frac{1}{4} \end{bmatrix}.$$

Exercise 6

Starting from $x^{(0)} = (0, 0, 0)^T$, compute 2 iterations of the Gauss-Seidel method applied to the system $Ax = b$, where

$$A = \begin{bmatrix} 1 & 4 & 1 \\ 2 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Solution: Recall that for iterative methods, we can split $A = M - N$, so that the general step of an iterative method is

$$x^{(k+1)} = x^{(k)} + M^{-1}r^{(k)}$$

In the case of Gauss-Seidel method, $M = \text{tril}(A)$

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix}.$$

Before starting, compute the residual $r^{(0)}$

$$r^{(0)} = b - Ax^{(0)} = b = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

First iteration

Firstly, we have to compute $M^{-1}r^{(0)}$, which amounts to solving the linear system

$$Mu = r^{(0)}.$$

Since M is lower triangular, it is easier to solve the linear system by forward-substitution, obtaining the solution u

$$u = \begin{bmatrix} 1 \\ -1 \\ 3 \end{bmatrix},$$

so $x^{(1)} = x^{(0)} + u = (1, -1, 3)^T$. Then compute the residual $r^{(1)} = b - Ax^{(1)}$

$$r^{(1)} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 & 4 & 1 \\ 2 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Second iteration As before, compute the solution of the linear system

$$Mu = r^{(1)}$$

using forward substitution. This yields

$$x^{(2)} = x^{(1)} + u = \begin{bmatrix} 1 \\ -1 \\ 3 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 \\ -3 \\ 7 \end{bmatrix}.$$

Exercise 7

Starting from $v^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, compute 2 iterations of the power method on the matrix

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

returning the eigenvector and eigenvalue approximation and showing the intermediate steps.

Solution: In the first iteration of the power method we compute:

$$\begin{aligned} w_1 &= Av_0 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ \|w_1\| &= \sqrt{2^2 + 1^2} = \sqrt{5}; \\ v_1 &= \frac{w_1}{\|w_1\|} = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix}; \\ \lambda_1 &= v_1^T Av_1 = \frac{1}{5} \begin{bmatrix} 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} 5 \\ 4 \end{bmatrix} = \frac{14}{5}. \end{aligned}$$

In the second iteration of the power method we compute:

$$\begin{aligned} w_2 &= Av_1 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \cdot \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{5}} \begin{bmatrix} 5 \\ 4 \end{bmatrix}; \\ \|w_2\| &= \frac{1}{\sqrt{5}} \sqrt{5^2 + 4^2} = \sqrt{\frac{41}{5}}; \\ v_2 &= \frac{w_2}{\|w_2\|} = \frac{1}{\sqrt{41}} \begin{bmatrix} 5 \\ 4 \end{bmatrix}; \\ \lambda_2 &= v_2^T Av_2 = \frac{1}{41} \begin{bmatrix} 5 & 4 \end{bmatrix} \cdot \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 5 \\ 4 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 5 & 4 \end{bmatrix} \cdot \begin{bmatrix} 14 \\ 13 \end{bmatrix} = \frac{122}{41}. \end{aligned}$$

Exercise 8

Apply two bisection iterations to solve the equation

$$x^3 + 3x - 2 = 0 \quad \text{in } [0, 1].$$

Solution: Let $F(x) = x^3 + 3x - 2$. We are looking the solution in the interval $[0, 1]$, so first check that $F(0)$ and $F(1)$ have different sign. Indeed

$$F(0) = -2, \quad F(1) = 2.$$

First iteration. We start with $a = 0$ and $b = 1$. Compute the midpoint $m = \frac{a+b}{2}$, and evaluate $F(m)$.

$$m = \frac{1}{2}, \quad F(m) = \frac{1}{8} + \frac{3}{2} - 2 = -\frac{3}{8}.$$

Therefore, since $F(m) < 0$, replace update a , $a := m$.

Second iteration. Compute the midpoint $m = \frac{a+b}{2}$ and evaluate $F(m)$.

$$m = \frac{3}{4}, \quad F(m) = \frac{27}{64} + \frac{9}{4} - 2 = \frac{43}{64}.$$

Since, $F(m) > 0$, then for the next iteration we update b , $b := m$.

Exercise 9

With initial guess $x^{(0)} = 1$ apply one Newton iteration to find an approximate solution of the equation

$$(2x - 1)(3x^2 - 2x + 1) = 0$$

Solution: We have to solve the equation $F(x) = 0$ where

$$F(x) := (2x - 1)(3x^2 - 2x + 1)$$

Recall that the generic Newton iteration is given as follows

$$x^{(k+1)} = x^{(k)} - \frac{F(x^{(k)})}{F'(x^{(k)})},$$

so firstly compute F'

$$F'(x) = 2(3x^2 - 2x + 1) + (2x - 1)(6x - 2).$$

So, compute the first iteration of the Newton method

$$x^{(1)} = x^{(0)} - \frac{F(x^{(0)})}{F'(x^{(0)})} = 1 - \frac{2}{8} = 1 - \frac{1}{4} = \frac{3}{4}.$$

Exercise 10

With initial guess $\underline{x}^{(0)} = [1, 1]^T$ apply one Newton iteration to find an approximate solution of the system

$$\underline{F}(\underline{x}) = \begin{bmatrix} x_1^2 - 2x_1 + x_2 + 7 \\ 2x_1 - x_2 + 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Solution: Recall that the Newton's iteration for multivariate functions is given by

$$\underline{x}^{(k+1)} = \underline{x}^{(k)} - \left(J_F(\underline{x}^{(k)}) \right)^{-1} \cdot \underline{F}(\underline{x}^{(k)}),$$

where J_F is the Jacobian matrix of $\underline{F}$. We have

$$J_F(\underline{x}) = \begin{bmatrix} \frac{\partial F_1}{\partial x_1} & \frac{\partial F_1}{\partial x_2} \\ \frac{\partial F_2}{\partial x_1} & \frac{\partial F_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 - 2 & 1 \\ 2 & -1 \end{bmatrix}.$$

For $k = 0$, that is the first iteration, we have

$$\underline{F}(\underline{x}^{(0)}) = \begin{bmatrix} 1^2 - 2 + 1 + 7 \\ 2 - 1 + 2 \end{bmatrix} = \begin{bmatrix} 7 \\ 3 \end{bmatrix}, \quad \text{and} \quad J_F(\underline{x}^{(0)}) = \begin{bmatrix} 0 & 1 \\ 2 & -1 \end{bmatrix},$$

and we have to compute $\underline{u} = [u_1, u_2]^T$ solution of $J_F(\underline{x}^{(0)}) \cdot \underline{u} = \underline{F}(\underline{x}^{(0)})$. The solution of this linear system is

$$\begin{cases} u_2 = 7 \\ 2u_1 - u_2 = 3 \end{cases} \rightarrow \begin{cases} u_2 = 7 \\ u_1 = \frac{3+7}{2} = 5 \end{cases}$$

and the first newton iteration step gives

$$\underline{x}^{(1)} = \underline{x}^{(0)} - \underline{u} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 5 \\ 7 \end{bmatrix} = - \begin{bmatrix} 4 \\ 6 \end{bmatrix}.$$

Exercise 11

Given the function $f(x) = \cos(2\pi x)$ compute its Lagrange interpolant of degree 2 through the points $x_1 = 0$, $x_2 = 1/2$, $x_3 = 1$.

Solution 1: Recall that the Lagrange interpolant of f of degree k , over the points $x_1, x_2, \dots, x_{k+1}$, is

$$\Pi_k(f) := \sum_{i=1}^{k+1} f(x_i) L_i(x), \quad \text{where} \quad L_i(x) := \prod_{\substack{j=1 \\ j \neq i}}^{k+1} \frac{(x - x_j)}{(x_i - x_j)}.$$

In order to compute $\Pi_2(f)$ on the points x_1, x_2, x_3 , we have

$$\begin{aligned} L_1(x) &= \frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)} = \frac{(x-\frac{1}{2})(x-1)}{(-\frac{1}{2})(-1)} = (2x-1)(x-1), \\ L_2(x) &= \frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)} = \frac{(x-0)(x-1)}{(\frac{1}{2})(-\frac{1}{2})} = -4x(x-1), \\ L_3(x) &= \frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)} = \frac{(x-0)(x-\frac{1}{2})}{(1)(\frac{1}{2})} = x(2x-1), \end{aligned}$$

and also

$$\begin{aligned} f(x_1) &= \cos(2\pi x_1) = \cos(0) = 1, \\ f(x_2) &= \cos(2\pi x_2) = \cos(\pi) = -1, \\ f(x_3) &= \cos(2\pi x_3) = \cos(2\pi) = 1. \end{aligned}$$

Finally

$$\begin{aligned} \Pi_2(f) &= f(x_1)L_1(x) + f(x_2)L_2(x) + f(x_3)L_3(x) \\ &= 1 \cdot (2x-1)(x-1) + (-1) \cdot (-4x)(x-1) + 1 \cdot x(2x-1) \\ &= 2x^2 - 2x - x + 1 + 4x^2 - 4x + 2x^2 - x \\ &= 8x^2 - 8x + 1. \end{aligned}$$

Solution 2: An alternative solution consists in the following observation. $\Pi_2(f)$ is the degree two polynomial $c_1 + c_2x + c_3x^2$, where the coefficients c_1, c_2, c_3 are solution of the following linear system

$$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \end{bmatrix}$$

For this exercise we have

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & \frac{1}{2} & \frac{1}{4} \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$$

In particular $c_1 = 1$ and it remains to solve

$$\begin{cases} 2c_2 + c_3 = -8 \\ c_2 + c_3 = 0 \end{cases} \rightarrow \begin{cases} 2c_2 + c_3 = -8 \\ c_3 = -c_2 \end{cases} \rightarrow \begin{cases} c_2 = -8 \\ c_3 = 8 \end{cases}$$

Again we found that $\Pi_2(f) = 1 - 8x + 8x^2$.

Exercise 12

Given the Cauchy problem:

$$\begin{cases} y'(t) = -2ty(t) + 2t^3 \text{ for } t > 0 \\ y(0) = -1; \end{cases}$$

compute two steps by the implicit Euler method, with $\Delta t = 1$, in order to approximate $y(2)$. Report the intermediate computations.

Solution: We denote by $y_n := y(t_n)$, where $t_n = t_0 + n\Delta t$, and here we have $t_0 = 0$, $t_1 = 1$ and $t_2 = 2$. The Implicit Euler (IE) method is given by

$$\frac{y_{n+1} - y_n}{\Delta t} = f(t_{n+1}, y_{n+1}),$$

and for this exercise, we have $f(t, y(t)) = -2ty(t) + 2t^3$. Thus, the IE method applied to our Cauchy problem leads to solving, for each step, the following equation:

$$\frac{y_{n+1} - y_n}{\Delta t} = -2t_{n+1}y_{n+1} + 2t_{n+1}^3, \quad (\text{Implicit in the unknown } y_{n+1}).$$

Therefore we manipulate it as

$$\begin{aligned} y_{n+1} + 2\Delta t \cdot t_{n+1} \cdot y_{n+1} &= y_n + 2\Delta t \cdot t_{n+1}^3; \\ y_{n+1} &= \frac{y_n + 2\Delta t \cdot t_{n+1}^3}{1 + 2\Delta t \cdot t_{n+1}}, \quad (\text{Explicitated in } y_{n+1}). \end{aligned}$$

Finally, the computation of the first step is

$$y_1 = \frac{y_0 + 2\Delta t \cdot t_1^3}{1 + 2\Delta t \cdot t_1} = \frac{-1 + 2}{1 + 2} = \frac{1}{3},$$

and the computation of the second step is

$$y_2 = \frac{y_1 + 2\Delta t \cdot t_2^3}{1 + 2\Delta t \cdot t_2} = \frac{1/3 + 16}{1 + 4} = \frac{1 + 48}{15} = \frac{49}{15},$$

Exercise 13

Write the pseudo-code of the composite trapezoidal quadrature rule, then use the composite trapezoidal quadrature rule to compute an approximation of

$$\int_0^{2\pi} \sin^2(t) dt$$

by splitting the integration interval $[0, 2\pi]$ into four uniform subintervals. Report the intermediate computations.

Solution:

The composite trapezoidal quadrature rule amounts to approximating the function to be integrated with the Lagrange piecewise-linear approximation and integrating it. The extrema of the four subintervals are $t_1 = 0$, $t_2 = \frac{\pi}{2}$, $t_3 = \pi$, $t_4 = \frac{3\pi}{2}$, $t_5 = 2\pi$. The width of such subintervals is $\frac{\pi}{2}$. So

$$\int_0^{2\pi} \sin^2(t) dt \simeq \frac{\pi}{4} \sum_{i=1}^4 \int_{t_i}^{t_{i+1}} \Pi_1(\sin^2)(t) dt = \frac{\pi}{4} \sum_{i=1}^4 [\sin^2(t_i) + \sin^2(t_{i+1})].$$

Compute each term of the sum individually:

- $\sin^2(0) + \sin^2(\frac{\pi}{2}) = 1$,
- $\sin^2(\frac{\pi}{2}) + \sin^2(\pi) = 1$,
- $\sin^2(\pi) + \sin^2(\frac{3\pi}{2}) = 1$,
- $\sin^2(\frac{3\pi}{2}) + \sin^2(2\pi) = 1$,

combining all the terms yields

$$\int_0^{2\pi} \sin^2(t) dt \simeq \frac{\pi}{4} (1 + 1 + 1 + 1) = \pi.$$

Exercise 14

Describe the Crank-Nicolson scheme for the solution of an ODE and explain its relation with the trapezoidal quadrature rule. Then, compute one step of the Crank-Nicolson scheme for the problem

$$\begin{cases} y'(t) = 2t(1 - y(t)) \\ y(0) = 3 \end{cases}$$

selecting $\Delta t = 1$.

Solution: Let $t_n = t_0 + n\Delta t$ and $y_n = y(t_n)$. We are given $t_0 = 0$ and $\Delta t = 1$, and have to compute one step of Crank-Nicolson, i.e. compute y_1 , for $t_1 = 1$.

For the Crank-Nicolson scheme, we approximate the derivative using a finite difference and the ODE field with the average of the fields at two consecutive timesteps

$$\frac{y_n - y_{n-1}}{\Delta t} = \frac{1}{2} (f(t_n, y_n) + f(t_{n-1}, y_{n-1})).$$

In our case, $f(t, y) = 2t(1 - y)$ and $\Delta = 1$, therefore

$$y_n - y_{n-1} = \frac{1}{2} (2t_n(1 - y_n) + 2t_{n-1}(1 - y_{n-1})).$$

In order to compute y_1 , substitute t_0, t_1, y_0 in the equation above and solve for y_1 .

$$\begin{aligned}y_1 - y_0 &= \frac{1}{2}(2t_1(1 - y_1) + 2t_0(1 - y_0)) \\y_1 - 3 &= \frac{1}{2}(2 \cdot 1 \cdot (1 - y_1) + 2 \cdot 0 \cdot (1 - 3)),\end{aligned}$$

to obtain $y_1 = 2$.

Exercise 1

Compute the first two iterations of the Jacobi method for solving the linear system $Ax = b$, with:

$$A = \begin{bmatrix} 4 & 2 & 0 \\ 0 & 2 & 1 \\ 1 & -1 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}, \quad x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Solution

For computing $x^{(1)}$ and $x^{(2)}$, we can rely on the formula for the Jacobi method:

$$x_i^{(k)} = \frac{1}{a_{i,i}} \left(b_i - \sum_{j=1, j \neq i}^n a_{i,j} x_j^{(k-1)} \right)$$

Let us start from $x^{(1)} = [x_1^{(1)}, x_2^{(1)}, x_3^{(1)}]$:

$$x_1^{(1)} = \frac{1}{a_{1,1}} \left(b_1 - \sum_{j=2}^3 a_{1,j} x_j^{(0)} \right) = \frac{1}{4} (0 - 2 \cdot 0 - 0 \cdot 0) = 0$$

$$x_2^{(1)} = \frac{1}{a_{2,2}} \left(b_2 - \sum_{j=1, j \neq 2}^3 a_{2,j} x_j^{(0)} \right) = \frac{1}{2} (2 - 0 \cdot 0 - 1 \cdot 0) = 1$$

$$x_3^{(1)} = \frac{1}{a_{3,3}} \left(b_3 - \sum_{j=1}^2 a_{3,j} x_j^{(0)} \right) = \frac{1}{4} (0 - 1 \cdot 0 - (-1) \cdot 0) = 0$$

So $x^{(1)} = [0, 1, 0]$. Now we can compute $x^{(2)} = [x_1^{(2)}, x_2^{(2)}, x_3^{(2)}]$:

$$x_1^{(2)} = \frac{1}{a_{1,1}} \left(b_1 - \sum_{j=2}^3 a_{1,j} x_j^{(1)} \right) = \frac{1}{4} (0 - 2 \cdot 1 - 0 \cdot 0) = -\frac{1}{2}$$

$$x_2^{(2)} = \frac{1}{a_{2,2}} \left(b_2 - \sum_{j=1, j \neq 2}^3 a_{2,j} x_j^{(1)} \right) = \frac{1}{2} (2 - 0 \cdot 0 - 1 \cdot 0) = 1$$

$$x_3^{(2)} = \frac{1}{a_{3,3}} \left(b_3 - \sum_{j=1}^2 a_{3,j} x_j^{(1)} \right) = \frac{1}{4} (0 - 1 \cdot 0 - (-1) \cdot 1) = \frac{1}{4}$$

So $x^{(2)} = [-\frac{1}{2}, 1, \frac{1}{4}]$.

Exercise 2

Compute the first two iterations of the Gauss-Seidel method for solving the linear system $Ax = b$, with:

$$A = \begin{bmatrix} 4 & 2 & 0 \\ 0 & 2 & 1 \\ 1 & -1 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}, \quad x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Solution

For computing $x^{(1)}$ and $x^{(2)}$, we can rely on the formula for the Gauss-Seidel method:

$$x_i^{(k)} = \frac{1}{a_{i,i}} \left(b_i - \sum_{j=1}^{i-1} a_{i,j} x_j^{(k)} - \sum_{j=i+1}^n a_{i,j} x_j^{(k-1)} \right)$$

Let us start from $x^{(1)} = [x_1^{(1)}, x_2^{(1)}, x_3^{(1)}]$:

$$x_1^{(1)} = \frac{1}{a_{1,1}} \left(b_1 - \sum_{j=2}^3 a_{1,j} x_j^{(0)} \right) = \frac{1}{4} (0 - 2 \cdot 0 - 0 \cdot 0) = 0$$

$$x_2^{(1)} = \frac{1}{a_{2,2}} \left(b_2 - a_{2,1} x_1^{(1)} - a_{2,3} x_3^{(0)} \right) = \frac{1}{2} (2 - 0 \cdot 0 - 1 \cdot 0) = 1$$

$$x_3^{(1)} = \frac{1}{a_{3,3}} \left(b_3 - \sum_{j=1}^2 a_{3,j} x_j^{(1)} \right) = \frac{1}{4} (0 - 1 \cdot 0 - (-1) \cdot 1) = \frac{1}{4}$$

So $x^{(1)} = [0, 1, \frac{1}{4}]$. Now we can compute $x^{(2)} = [x_1^{(2)}, x_2^{(2)}, x_3^{(2)}]$:

$$x_1^{(2)} = \frac{1}{a_{1,1}} \left(b_1 - \sum_{j=2}^3 a_{1,j} x_j^{(1)} \right) = \frac{1}{4} \left(0 - 2 \cdot 1 - 0 \cdot \frac{1}{4} \right) = -\frac{1}{2}$$

$$x_2^{(2)} = \frac{1}{a_{2,2}} \left(b_2 - a_{2,1} x_1^{(2)} - a_{2,3} x_3^{(1)} \right) = \frac{1}{2} \left(2 - 0 \cdot \left(-\frac{1}{2} \right) - 1 \cdot \frac{1}{4} \right) = \frac{7}{8}$$

$$x_3^{(2)} = \frac{1}{a_{3,3}} \left(b_3 - \sum_{j=1}^2 a_{3,j} x_j^{(2)} \right) = \frac{1}{4} \left(0 - 1 \cdot \left(-\frac{1}{2} \right) - (-1) \cdot \frac{7}{8} \right) = \frac{11}{32}$$

So $x^{(2)} = [-\frac{1}{2}, \frac{7}{8}, \frac{11}{32}]$.

Exercise 3

Solve the linear system $Ax = b$ using the Conjugate Gradient method, with:

$$A = \begin{bmatrix} 5 & 4 & 1 \\ 4 & 6 & 4 \\ 1 & 4 & 5 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}, \quad x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Solution

Since $A \in \mathbb{R}^{3 \times 3}$, we observe that, in exact arithmetic, Conjugate Gradient Method will converge in at most 3 iteration. Let us recall the pseudocode for the Conjugate Gradient Method:

$$r^{(0)} = b - Ax^{(0)}$$

$$p^{(0)} = r^{(0)}$$

for $k = 1, 2, \dots$, maxiter:

$$y = Ap^{(k-1)}$$

$$\alpha_{k-1} = (p^{(k-1)}, r^{(k-1)}) / (y, p^{(k-1)})$$

$$x^{(k)} = x^{(k-1)} + \alpha_{k-1}p^{(k-1)}$$

$$r^{(k)} = b - Ax^{(k)} = r^{(k-1)} - \alpha_{k-1}y$$

$$\beta_{k-1} = (y, r^{(k)}) / (y, p^{(k-1)})$$

$$p^{(k)} = r^{(k)} - \beta_{k-1}p^{(k-1)}$$

If Stopping criteria are satisfied exit the loop

end

Let us start by computing the initial residual $r^{(0)}$, that is also the initial descent direction $p^{(0)}$:

$$r^{(0)} = b - Ax^{(0)} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} - \begin{bmatrix} 5 & 4 & 1 \\ 4 & 6 & 4 \\ 1 & 4 & 5 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}, \quad p^{(0)} = r^{(0)} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

Now, we need to compute the initial step length α_0 , to do that we firstly compute the quantity $Ap^{(0)}$:

$$Ap^{(0)} = \begin{bmatrix} 5 & 4 & 1 \\ 4 & 6 & 4 \\ 1 & 4 & 5 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ -4 \\ -2 \end{bmatrix}$$

At this point, we can compute α_0 :

$$\alpha_0 = \frac{(p^{(0)}, r^{(0)})}{(Ap^{(0)}, p^{(0)})} = \frac{\begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}}{\begin{bmatrix} -2 & -4 & -2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}} = \frac{6}{4} = \frac{3}{2}$$

We are now able to compute the first approximate solution $x^{(1)}$ as

$$x^{(1)} = x^{(0)} + \alpha_0 \cdot p^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \frac{3}{2} \cdot \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3/2 \\ -3 \\ 3/2 \end{bmatrix}$$

and the next residual $r^{(1)}$ is

$$r^{(1)} = b - Ax^{(1)} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} - \begin{bmatrix} 5 & 4 & 1 \\ 4 & 6 & 4 \\ 1 & 4 & 5 \end{bmatrix} \cdot \begin{bmatrix} 3/2 \\ -3 \\ 3/2 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} - \begin{bmatrix} -3 \\ -6 \\ -3 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix}$$

Now, to perform the next iteration, we need to determine the next descent direction $p^{(1)} = r^{(1)} - \beta_0 \cdot p^{(0)}$. Hence, we need to compute β_0 :

$$\beta_0 = \frac{(Ap^{(0)}, r^{(1)})}{(Ap^{(0)}, p^{(0)})} = \frac{\begin{bmatrix} -2 & -4 & -2 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix}}{\begin{bmatrix} -2 & -4 & -2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}} = -\frac{32}{4} = -8$$

At this point, we obtain

$$p^{(1)} = r^{(1)} - \beta_0 \cdot p^{(0)} = \begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix} - (-8) \cdot \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 12 \\ -12 \\ 12 \end{bmatrix}$$

Now, we need to compute the next step length α_1 , to do that we firstly compute the quantity $Ap^{(1)}$:

$$Ap^{(1)} = \begin{bmatrix} 5 & 4 & 1 \\ 4 & 6 & 4 \\ 1 & 4 & 5 \end{bmatrix} \cdot \begin{bmatrix} 12 \\ -12 \\ 12 \end{bmatrix} = \begin{bmatrix} 24 \\ 24 \\ 24 \end{bmatrix}$$

and thus α_1 is

$$\alpha_1 = \frac{(p^{(1)}, r^{(1)})}{(Ap^{(1)}, p^{(1)})} = \frac{\begin{bmatrix} 12 & -12 & 12 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix}}{\begin{bmatrix} 24 & 24 & 24 \end{bmatrix} \cdot \begin{bmatrix} 12 \\ 12 \\ 12 \end{bmatrix}} = \frac{48}{24 \cdot 12 \cdot 3} = \frac{1}{6}$$

We are now able to compute the second approximate solution $x^{(2)}$ as

$$x^{(2)} = x^{(1)} + \alpha_1 \cdot p^{(1)} = \begin{bmatrix} 3/2 \\ -3 \\ 3/2 \end{bmatrix} + \frac{1}{6} \cdot \begin{bmatrix} 12 \\ -12 \\ 12 \end{bmatrix} = \begin{bmatrix} 7/2 \\ -5 \\ 7/2 \end{bmatrix}$$

and the next residual is

$$r^{(2)} = b - Ax^{(2)} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} - \begin{bmatrix} 5 & 4 & 1 \\ 4 & 6 & 4 \\ 1 & 4 & 5 \end{bmatrix} \cdot \begin{bmatrix} 7/2 \\ -5 \\ 7/2 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Since we have obtained the null vector as the residual, we have that $x^{(2)}$ is the exact solution.

EXERCISE 1: Matlab implementation

- Write a function that, given as input an upper triangular matrix U and a vector b , solves the system $Ux = b$ using the backsubstitution method and test it on the system

$$\begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & -5 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ -4 \\ 2 \end{bmatrix}$$

whose exact solution is $x_1 = x_2 = x_3 = 1$

- Write a function that, given as input a matrix A and a vector b , solves the system $Ax = b$ using Gaussian elimination

EXERCISE 2: GEM by hands and on Matlab

- Using the Gauss elimination method, solve, by pencil and paper, the linear system below; then check by Matlab the solution (with your own GEM function and with the Matlab command: $A \backslash b$).

$$\begin{bmatrix} 4 & 0 & 12 \\ -2 & 6 & -3 \\ 1 & 2 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- Same as before, with

$$\begin{bmatrix} -5 & 3 & 4 \\ 10 & -8 & -9 \\ 15 & 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \\ 1 \end{bmatrix}$$

- Same as before, with

$$\begin{bmatrix} 1 & -3 & 4 \\ -1 & 5 & -3 \\ 4 & -8 & 23 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 12 \\ -12 \\ 58 \end{bmatrix}$$

EXERCISE 3: LU factorization

- Write a Matlab function that, given as input a matrix A , returns the L and U factors of the LU factorization without pivoting and test it on the following matrices:

$$\begin{bmatrix} 4 & 0 & 12 \\ -2 & 6 & -3 \\ 1 & 2 & 5 \end{bmatrix}$$

$$\begin{bmatrix} -5 & 3 & 4 \\ 10 & -8 & -9 \\ 15 & 1 & 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -3 & 4 \\ -1 & 5 & -3 \\ 4 & -8 & 23 \end{bmatrix}$$

EXERCISE 4: GEM with pivoting

- Write a Matlab function that, given as input a matrix A and a vector b , solves the system $Ax = b$ using Gaussian elimination with pivoting and test it on the two systems:

$$\begin{bmatrix} 4 & 0 & 12 \\ -2 & 6 & -3 \\ 1 & 2 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}; \quad \begin{bmatrix} 2 & 2 & 0 \\ 1 & 1 & -1 \\ 3 & -2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ 5 \end{bmatrix}$$

whose exact solutions are respectively $x = [-3, -1/2, 1]^T$ and $x = [1, 1, 1]^T$.

EXERCISE 5: solving linear systems and more...

- Using the LU factorization function above, write a Matlab function that returns the inverse of an input matrix.
- Modify the above function to compute the determinant of an input matrix A , and test it on the matrices in the previous slide. Finally, check the results using Matlab command **det**
- Solve a system $Ax = f$ with user-made functions above, where f arbitrary chosen and

$$A = \begin{bmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \dots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & & -1 & 2 & -1 \\ 0 & 0 & \dots & -1 & 2 \end{bmatrix}$$

In particular, compute x with either the GEM/LU methods implemented above and by explicitly computing $x = A^{-1}f$ (i.e., computing A^{-1} with the function above). Using the Matlab commands **tic** and **toc**, measure the time required by the two approaches to compute the solution. The matrix size should be chosen large enough so that the time difference is relevant.

EXERCISE 6: LU with pivoting (optional)

- Write a Matlab function that, given as input a matrix A , returns the P , L and U factors of the LU factorization with pivoting and test it on the following matrix:

$$\begin{bmatrix} 2 & 2 & 0 \\ 1 & 1 & -1 \\ 3 & -2 & 4 \end{bmatrix}$$

EXERCISE 7: sparse matrices (optional)

When the vast majority of a matrix A entries are zero, it is convenient to store only the nonzero values (and their position) in the memory. The Matlab function **sparse** can convert a non-sparse (dense) matrix into a sparse one.

- Let $Au = f$ be the matrix in the previous slide. Using the Matlab command **whos**, compare the memory usage when A is stored as dense and as sparse, for a large enough matrix size. Compare also the time spent to solve the system (use Matlab LU factorisation, and Matlab solver to invert the trinagular systems).
- Consider a similar system $Bu = f$, where

$$B = \begin{bmatrix} 2 & -1 & -1 & \dots & -1 \\ -1 & 2 & 0 & \dots & 0 \\ -1 & 0 & \ddots & & \vdots \\ \vdots & & & 2 & 0 \\ -1 & 0 & \dots & 0 & 2 \end{bmatrix}$$

Note that A and B have the same sparisity, i.e. the same number of nonzero entries. Compare again the memory and solution time required when B is stored as sparse or as dense. Do you observe any difference with the previous case? If yes, why? You might want to compare the sparsity pattern (Matlab command **spy**) of the L U factors.

EXERCISE 11

- Write a Matlab function `c = linear_regression(x,y)` that computes the coefficients of the line that fits the input points (x_i, y_i) , $i = 1, \dots$, in the least squares sense. Compute the coefficient vector c by solving the system of normal equations.
- Test the above function on the data in the file `least_squares_data.mat`. Make a graph that representing the data points and the line.
- Write another function `c = quadratic_regression(x,y)`, similar to the previous one but where a polynomial of degree 2 is used for the fitting. Plot again the results for the data points above.

EXERCISE 12

- Write two Matlab functions:

- `[x,its] = bisection(f,a,b,tol,maxits)`, that implements the bisection method.
- `[x,its] = newton(f,derf,x0,tol,maxits)`, that implements the Newton method.

Implement suitable stopping criteria for the two methods. Use both functions to solve the equation $e^x + \cos(x) - 2 = 0$, whose exact solution is $x = 0$ (choose appropriately the initial guess $x_0 \neq 0$ or the initial interval). Plot the error $|x_k - 0|$ vs k . Repeat the same for solving $e^x - \cos(x) - 2 = 0$ whose solution is not known analytically and can be approximated by the previous methods with small tolerance or by `fsolve(@(x) exp(x)-cos(x)-2,0)` which calls the MATLAB solver.

- Use both methods to solve the equation $\sqrt[3]{x - \pi} = 0$ (use the function `nthroot` to compute cubic roots). Plot the error $|x_k - \pi|$ vs k . Discuss the results.

Exercise: composite quad. rules and order of convergence

Consider the three composite quadrature rules:

- composite midpoint rule
- composite trapezoidal rule
- composite Simpson rule

and write a MATLAB function that uses them (on a uniform subdivision in elements of mesh-size h) to approximate

$$\int_0^1 \sin(\pi x) dx = \frac{2}{\pi}$$

Then, plot the error versus h , and identify on a log-log plot the order of convergence of the three rules (include in the same figure the reference plots: h , h^2 , h^3 , ...)

Exercise: ODE solvers

Consider the following Cauchy problem

$$\begin{cases} y'(t) = -3y(t) + 2t - 2 & t \in [0, 1] \\ y(0) = -1. \end{cases} \quad (1)$$

- compute the approximation of $y(1)$ by applying two steps of IE
- implement in MATLAB the methods: EE, IE, CN (Crank-Nicolson) and Heun, for solving (1); after checking the answer of the previous question, use the four methods to compute the solution on $[0, 1]$; plot the results for $\Delta t = .5$, $\Delta t = .2$, $\Delta t = .1$ together with the exact solution

$$y(t) = (2t)/3 - 1/9 e^{-3t} - 8/9.$$

EXERCISE 8

- Write Matlab functions of the form $[x, res, its] = \text{Jacobi}(A, b, tol, maxit, x_0)$ that implement the Jacobi method. Do the same for the Gauss-Seidel method. Here res is the vector of residual norms for each iteration. Take the following algorithm as reference (as stopping criteria, use $\|r_k\| / \|r_0\| < tol$ and $\|x^{(k)} - x^{(k-1)}\| / \|x^{(k-1)}\| < tol$. Stop in any case if $its > maxits$).

Chosen M and N (as in $A = M - N$) and x_0 , compute r_0 .

For $k = 1, 2, \dots$, until convergence:

$$x_k = x_{k-1} + M^{-1}r_{k-1} \quad (\text{do not compute the matrix } M^{-1}, \text{ just solve the system})$$

$$r_k = b - Ax_k$$

- Test the two methods on the problem $Ax = b$, where

$$A = \begin{bmatrix} 7 & 6 & 3 \\ 2 & 5 & -4 \\ -4 & -3 & 8 \end{bmatrix}, \quad b = \begin{bmatrix} 16 \\ 3 \\ 1 \end{bmatrix},$$

using $tol = 10^{-6}$, $maxits = 1000$, $x_0 = (0, \dots, 0)^T$. Compare the history of convergence of the two methods, i.e. plot the iteration number vs. the norm of the residual. What is the value of $\max_i |\lambda_i(B)|$, where B is the iteration matrix?

Useful Matlab commands: $M = \text{diag}(\text{diag}(A))$, $M = \text{tril}(A)$, $\text{plot}()$, $\text{eig}()$

EXERCISE 9

Consider the system

$$Ax = b$$

with $A = \text{gallery}(\text{'poisson'}, n)$ and $b = (1, \dots, 1)^T$.

- For $n = 40$ solve the system using the Jacobi, Gauss-Seidel and CG (as implemented by the `pcg` function) methods. Compare the history of convergence of the three methods.
- For $n = 10, 20, \dots, 100$ solve the system using CG. Check how the number of iterations varies, by plotting the number of iterations vs. n . Check also how the condition number $\lambda_{\max}(A)/\lambda_{\min}(A)$ varies (use the function `eigs`), and try to relate it with the number of iterations.

EXERCISE 10

- Write a Matlab function `[x,mu]=power(A,maxit,tol)` that implements the power method to approximate the dominant eigenvalue of a matrix (use a random vector as initial guess). Test it on the matrix

$$A = Q \cdot \text{diag}(1:10) \cdot \text{inv}(Q), \text{ with } Q = \text{orth}(\text{randn}(10,10))$$

Note that the spectrum of A is $\{1, \dots, 10\}$. Modify the function to be able to plot the relative error $|\lambda_1 - \mu_k| / |\lambda_1|$ and also $(|\lambda_2 / \lambda_1|)^k$, $k = 1, 2, \dots$. What do you observe?

- Same as above, but test on the nonsymmetric matrix obtained using $Q = \text{randn}(10,10)$. Compare the results with the previous case.
- Write another function, `inverse_power`, that implements the inverse power method (use “backslash” to solve the linear system), and test it on the matrix used in the previous point to approximate $\lambda_9 = 2$ using

$$\mu = 1.55, 1.65, 1.75, 1.85, 1.95.$$

Plot the number of iterations required to converge vs. the value $|\lambda_9 - \mu| / |\lambda_{10} - \mu|$ and comment.

EXERCISE 10

- Write a Matlab function `[x,mu]=power(A,maxit,tol)` that implements the power method to approximate the dominant eigenvalue of a matrix (use a random vector as initial guess). Test it on the matrix

$$A = Q \cdot \text{diag}(1:10) \cdot \text{inv}(Q), \text{ with } Q = \text{orth}(\text{randn}(10,10))$$

Note that the spectrum of A is $\{1, \dots, 10\}$. Modify the function to be able to plot the relative error $|\lambda_1 - \mu_k| / |\lambda_1|$ and also $(|\lambda_2 / \lambda_1|)^k$, $k = 1, 2, \dots$. What do you observe?

- Same as above, but test on the nonsymmetric matrix obtained using $Q = \text{randn}(10,10)$. Compare the results with the previous case.
- Write another function, `inverse_power`, that implements the inverse power method (use “backslash” to solve the linear system), and test it on the matrix used in the previous point to approximate $\lambda_9 = 2$ using

$$\mu = 1.55, 1.65, 1.75, 1.85, 1.95.$$

Plot the number of iterations required to converge vs. the value $|\lambda_9 - \mu| / |\lambda_{10} - \mu|$ and comment.

EXERCISE 10

- Write a Matlab function `[x,mu]=power(A,maxit,tol)` that implements the power method to approximate the dominant eigenvalue of a matrix (use a random vector as initial guess). Test it on the matrix

$$A = Q \cdot \text{diag}(1:10) \cdot \text{inv}(Q), \text{ with } Q = \text{orth}(\text{randn}(10,10))$$

Note that the spectrum of A is $\{1, \dots, 10\}$. Modify the function to be able to plot the relative error $|\lambda_1 - \mu_k| / |\lambda_1|$ and also $(|\lambda_2 / \lambda_1|)^k$, $k = 1, 2, \dots$. What do you observe?

- Same as above, but test on the nonsymmetric matrix obtained using $Q = \text{randn}(10,10)$. Compare the results with the previous case.
- Write another function, `inverse_power`, that implements the inverse power method (use “backslash” to solve the linear system), and test it on the matrix used in the previous point to approximate $\lambda_9 = 2$ using

$$\mu = 1.55, 1.65, 1.75, 1.85, 1.95.$$

Plot the number of iterations required to converge vs. the value $|\lambda_9 - \mu| / |\lambda_{10} - \mu|$ and comment.

- ☐ I want to take the BASIC EXAM (maximum grade is 24/30)
☐ I want to take the ADVANCED EXAM (maximum grade is 30/30 cum laude)

Total time is 1 hour.

BASIC EXAM

1. Starting from $x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$, compute two iterations of the Jacobi method in order to compute an approximate solution of the system $Ax = b$, where

$$A = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & -3 \\ 0 & -3 & 6 \end{bmatrix}, \quad b = \begin{bmatrix} -2 \\ 6 \\ -12 \end{bmatrix}$$

Report intermediate computations.

2. Write the pseudo-code of the Bisection Method. Apply two bisection iterations to find an approximation in $[0, 2]$ of the solution of the equation

$$2x^3 + 4x^2 - x - 6 = 0.$$

ADVANCED EXAM

3. Explain what is the LU factorization of a matrix, without pivoting, and give its pseudo-code. Why pivoting is necessary in the general case? Give an example to support your answer. Then, compute the LU factorization, without pivoting, of

$$A = \begin{bmatrix} -4 & -4 & -1 \\ -20 & -15 & 0 \\ 0 & 15 & 19 \end{bmatrix}$$

Report the intermediate computations.

4. State and prove the theorem on the existence and uniqueness of the Lagrange interpolation polynomial of a given function.

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Write the pseudocode of the backward substitution method used to solve linear systems where the matrix is upper triangular. Show how it works when solving the system $Ax = b$, where

$$A = \begin{bmatrix} 2 & 2 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 1 \\ 4 \end{bmatrix}.$$

2. Describe the implicit Euler method. Then, given the Cauchy problem:

$$\begin{cases} y'(t) = 3y(t) + 2t^2 \text{ for } t > 0 \\ y(0) = 1; \end{cases}$$

compute two steps by the implicit Euler method, with $\Delta t = 2$, in order to approximate $y(4)$. Report the intermediate computations.

ADVANCED EXAM

3. Write the pseudocode of the backward substitution method used to solve linear systems where the matrix is upper triangular. Describe (with full justification) its computational cost. Show how it works when solving the system $Ax = b$, where

$$A = \begin{bmatrix} 2 & 2 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 1 \\ 4 \end{bmatrix}.$$

4. Describe the descent methods: the Gradient (steepest descent) and Conjugate Gradient methods

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Starting from $x^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, compute 2 iterations of the power method on the matrix

$$A = \begin{bmatrix} 2 & 2 \\ 2 & -1 \end{bmatrix}$$

returning the eigenvector and eigenvalue approximation and showing the intermediate steps.

2. Write the pseudocode of the Newton method. With initial guess $x_0 = 1$ apply one Newton iteration to find an approximate solution of the equation

$$(2x + 2)(x + 3) = 0$$

ADVANCED EXAM

3. Write the pseudo-code of the composite midpoint quadrature rule, then use the composite midpoint quadrature rule to compute an approximation of

$$\int_{-1}^2 (t^2 + 2t) dt$$

by splitting the integration interval $[-1, 2]$ into three subintervals. Report the intermediate computations.

4. Explain the *pagerank* algorithm and write its pseudocode.

- ☐ I want to take the BASIC EXAM
☐ I want to take the the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Compute the LU factrization (without pivoting) of the matrix

$$A = \begin{bmatrix} 3 & 3 & 3 \\ 6 & 8 & 8 \\ 9 & 13 & 14 \end{bmatrix}$$

showing the intermediate computations.

2. Write the pseudocode of the composite trapezoidal rule.

ADVANCED EXAM

3. Give the definition of the *least square method* for data fitting and write a pseudocode for the computation of the linear regression line. Then, compute the linear regression line that approximates the points:

$$(-2, 0), \quad (-1, 1), \quad (0, 1), \quad (1, -1), \quad (2, 0),$$

4. Give the statement of the convergence theorem for the Newton method for solving nonlinear equations. Prove that the order of convergence of the methods is 2.

- ☐ I want to take the BASIC EXAM (maximum grade is 24/30)
☐ I want to take the ADVANCED EXAM (maximum grade is 30/30 cum laude)

Total time is 1 hour.

BASIC EXAM

1. Given the Cauchy problem

$$\begin{cases} y'(t) = 2ty(t) & \text{for } t > 0 \\ y(0) = 3 \end{cases},$$

compute two steps of the Implicit Euler Method, with $\Delta t = 1$, in order to approximate $y(2)$. Report the intermediate computations.

2. Describe two possible criteria for stopping iterative solvers for linear systems.

Starting from $x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$, compute two iterations of the Jacobi method applied to the system $Ax = b$, where

$$A = \begin{bmatrix} 4 & 0 & -1 \\ 0 & 4 & 2 \\ -1 & 2 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} -9 \\ -18 \\ -15 \end{bmatrix}.$$

Report intermediate computations.

ADVANCED EXAM

3. Write the pseudo-code of the Gaussian Elimination Method (without pivoting) used to solve linear systems. Describe (with full justification) its computational cost. Show how the Gaussian Elimination Method (without pivoting) works when solving the system $Ax = b$, with

$$A = \begin{bmatrix} -4 & -2 & -3 \\ -16 & -10 & -13 \\ -4 & 0 & -4 \end{bmatrix}, \quad b = \begin{bmatrix} -2 \\ -14 \\ -4 \end{bmatrix}.$$

4. State and prove the theorem on the existence and uniqueness of the Lagrange interpolation polynomial of a given function. State, without a proof, an error estimate for the interpolation error.

Numerical Methods in Engineering Sciences
19/7/2024

First name: _____

Last name: _____

Student ID: _____

- ☐ I want to take the BASIC EXAM (maximum grade is 24/30)
☐ I want to take the ADVANCED EXAM (maximum grade is 30/30 cum laude)

Total time is 1 hour.

BASIC EXAM

1. Compute the regression line $r(x) = c_0 + c_1x$ for the following set of points:

$$(-4, -6), (-3, 11), (-1, 3), (0, 5), (1, 6), (2, 6)$$

2. Write the pseudo-code of the backward substitution method used to solve linear systems where the matrix is upper triangular. Describe (with full justification) its computational cost. Apply the backward substitution method to solve the system $Ux = b$, with

$$U = \begin{bmatrix} 1 & 2 & -2 \\ 0 & 3 & 5 \\ 0 & 0 & -2 \end{bmatrix}, \quad b = \begin{bmatrix} -6 \\ 7 \\ -4 \end{bmatrix}$$

ADVANCED EXAM

3. Write the pseudo-code of the power method for the computation of the dominant eigenvalue and eigenvector. State conditions that guarantee its convergence. Starting from $v^{(0)} = [-2, -3]^T$, compute two iterations of the power method on the matrix

$$A = \begin{bmatrix} 3 & -2 \\ -2 & 6 \end{bmatrix}.$$

4. Write the pseudo-code of the Newton method for solving a non-linear equation. State and prove its order of convergence.

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Starting from $x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$, compute 2 iterations of the Jacobi method to compute an approximate solution of the system $Ax = b$, where

$$A = \begin{bmatrix} 4 & 2 & 0 \\ 0 & 2 & 1 \\ 1 & -1 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}$$

2.

Introduce the power method for the computation of the dominant eigenvalue and eigenvector, with pseudocode. State conditions that guarantee its convergence

ADVANCED EXAM

3. Write the pseudo-code of the composite midpoint quadrature rule, then use the composite midpoint quadrature rule to compute an approximation of

$$\int_{-1}^2 (t^2 + 2t) dt$$

by splitting the integration interval $[-1, 2]$ into three subintervals. Report the intermediate computations.

4. State and prove the theorem on the existence and uniqueness of the Lagrange interpolant of a given function.

- ☐ I want to take the BASIC EXAM
☐ I want to take the the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Compute the LU factrization (without pivoting) of the matrix

$$A = \begin{bmatrix} 4 & 2 & 0 \\ 3 & 1 & 2 \\ 1 & 1 & -1 \end{bmatrix}$$

showing the intermediate computations.

2. Write the pseudocode of the composite midpoint quadrature rule.

ADVANCED EXAM

3. Describe the Crank-Nicolson scheme for the solution of an ordinary differential equation and explain its relation with the trapezoidal quadrature rule. Then, compute one step of the Crank-Nicolson scheme for the problem

$$\begin{cases} y'(t) = e^t y(t) \\ y(0) = 1 \end{cases}$$

selecting $\Delta t = 1$.

4. State and prove the theorem on the existence and uniqueness of the Lagrange interpolant of a given function.

- ☐ I want to take the BASIC EXAM
☐ I want to take the the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Given the function $f(x) = \sin(x)$ compute its Lagrange interpolant of degree 2 through the points $x_1 = -\pi$, $x_2 = 0$, $x_3 = \frac{\pi}{2}$.
2. Write the pseudocode of the Jacobi method to solve a linear system.

ADVANCED EXAM

3. Write the pseudocode of the forward substitution method used to solve linear systems where the matrix is upper triangular. Describe its computational cost. Show how it works when solving the system $Ax = b$, where

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 2 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

4. Describe the composite midpoint quadrature rule, write its pseudocode, state and prove an error bound.

- ☐ I want to take the BASIC EXAM (maximum grade is 24/30)
☐ I want to take the ADVANCED EXAM (maximum grade is 30/30 cum laude)

Total time is 1 hour.

BASIC EXAM

1. Apply the Gaussian elimination method, without pivoting, to solve the linear system $Ax = b$, where

$$A = \begin{bmatrix} -4 & 2 & 3 \\ 0 & -4 & -2 \\ 4 & -10 & -6 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$$

Report the intermediate steps.

2. Write the pseudo-code of the composite trapezoidal quadrature rule, then use the composite trapezoidal quadrature rule to compute an approximation of

$$\int_0^2 \frac{t}{t+1} dt$$

by splitting the integration interval $[0, 2]$ into four uniform subintervals. Report the intermediate computations.

ADVANCED EXAM

3. Describe the Crank-Nicolson scheme for the solution of an ODE and explain its relation with the trapezoidal quadrature rule. Then, compute two steps of the Crank-Nicolson scheme for the problem

$$\begin{cases} y'(t) = -2ty & \text{for } t > 0 \\ y(0) = 1 \end{cases}$$

selecting $\Delta t = 1/2$.

4. Write the pseudo-code of the Jacobi splitting method and describe its computational cost. Prove that if A is a diagonally dominant matrix, then the Jacobi method converges to the exact solution of the system $Ax = b$.

Numerical Methods in Engineering Sciences
25/1/2024

First name: _____

Last name: _____

Student ID: _____

- ☐ I want to take the BASIC EXAM (maximum grade is 24/30)
☐ I want to take the ADVANCED EXAM (maximum grade is 30/30 cum laude)

Total time is 1 hour.

BASIC EXAM

1. Apply the Gaussian elimination method, without pivoting, to solve the linear system $Ax = b$, where

$$A = \begin{bmatrix} -4 & -2 & 1 \\ -4 & 2 & -1 \\ -20 & 6 & -7 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 7 \\ 17 \end{bmatrix}$$

2. Write the pseudo-code of the Newton Method. Apply two Newton iterations to compute an approximate solution of the equation

$$x^3 + x - 2 = 0, \quad \text{starting with } x_0 = 0$$

ADVANCED EXAM

3. Write the pseudo-code of the composite trapezoidal quadrature rule, then use the composite trapezoidal quadrature rule to compute an approximation of

$$\int_0^2 (t^2 - t) dt$$

by splitting the integration interval $[0, 2]$ into four uniform subintervals. Report the intermediate computations.

4. Write the pseudo-code of the “power method” for computing the dominant eigenvalue and eigenvector and describe a possible stopping criterion. State and prove the theorem on the convergence of the method to the dominant eigenvalue (biggest in module).

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Given the function $f(x) = x^2 + x^3$ compute its Lagrange interpolant of degree 2 through the points $x_1 = -2$, $x_2 = 0$, $x_3 = 3$.
2. Write the pseudocode of the backward substitution method used to solve linear systems where the matrix is upper triangular. Describe (with full justification) its computational cost. Show how it works when solving the system $Ax = b$, where

$$A = \begin{bmatrix} 3 & 2 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ -1 \\ 2 \end{bmatrix}.$$

ADVANCED EXAM

3. Write the pseudocode of the Gaussian elimination method, without pivoting, and apply it to solve the linear system

$$\begin{bmatrix} 3 & -2 & 1 \\ 2 & 2 & 2 \\ 2 & 1 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \\ 21 \end{bmatrix}$$

showing the intermediate computations.

4. Give the statement of the convergence theorem for the Newton method used to solve nonlinear equations. Prove that the order of convergence of the methods is 2.

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Compute the linear regression $r(x) = c_0 + c_1x$ for the set of points

$$(-3, 0), (-2, 0), (-1, 3), (1, 1), (2, 1), (3, 2).$$

2. Describe the Crank-Nicolson scheme for the solution of an ODE and explain its relation with the trapezoidal quadrature rule. Report its pseudocode. Finally, compute one step of the Crank-Nicolson scheme for the problem

$$\begin{cases} y'(t) = e^{-t} y(t) \\ y(0) = 1 \end{cases}$$

with step-size $\Delta t = 1$.

ADVANCED EXAM

3. Describe the LU-based algorithm that computes the inverse of a matrix A . Give its pseudocode that should include the LU factorization part (it could be with or without pivoting, your choice) and give an estimate of its computational cost
4. Write the pseudocode of the bisection method. Apply two bisection iterations to the equation

$$x^3 + 1 = 0 \quad \text{in } [-2, 2].$$

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Write the LU factorization, without pivoting, of:

$$\begin{bmatrix} 2 & 7 & 5 \\ 14 & 50 & 36 \\ 2 & 8 & 8 \end{bmatrix}$$

showing the intermediate computations.

2. Write the pseudo-code of the composite Simpson quadrature rule, then use it to compute an approximation of

$$\int_{-1}^1 (1 + 2t + t^2) dt$$

by splitting the integration interval $[-1, 1]$ into two subintervals. Report the intermediate computations.

ADVANCED EXAM

3. Given the matrix

$$\begin{bmatrix} \alpha & 0 & 1 \\ 0 & \alpha & 1 \\ 1 & 1 & \alpha \end{bmatrix}, \quad \alpha \in \mathbb{R}^+,$$

determine for which values of α the Gauss-Seidel method applied to the system $Ax = b$ converges for every $b \in \mathbb{R}^3$ and for every initial guess $x^{(0)} \in \mathbb{R}^3$.

4. Give the statement of the convergence theorem of the Newton method for nonlinear equations. Prove that the order of convergence of the methods is 2.

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. With initial guess $x_0 = 0$ apply one Newton iteration to find an approximate solution of the equation

$$(x^2 + 6x + 4)(x + 2) = 0$$

2. Write the pseudocode of the Gaussian elimination method, without pivoting, and apply it to solve the linear system

$$\begin{bmatrix} 3 & -2 & 1 \\ 2 & 2 & 2 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 5 \end{bmatrix}$$

showing the intermediate computations.

ADVANCED EXAM

3. Write the pseudocode of the bisection method. Apply two bisection iterations to the equation

$$x^2 - 1 = 0 \quad \text{in } [0, 3].$$

4. Introduce the power method for the computation of the dominant eigenvalue and eigenvector, with pseudocode. State and prove its convergence properties. Then, motivate the inverse power method, reporting its pseudocode.

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Given the Cauchy problem:

$$\begin{cases} y'(t) = -2y(t) + t^2 \text{ for } t > 0 \\ y(0) = 2; \end{cases}$$

compute two steps by the implicit Euler method, with $\Delta t = 1/2$, in order to approximate $y(1)$. Report the intermediate computations.

2. Introduce the power method for the computation of the dominant eigenvalue and eigenvector, with pseudocode. State conditions that guarantee its convergence

ADVANCED EXAM

3. Describe the Crank-Nicolson scheme for the solution of an ODE and explain its relation with the trapezoidal quadrature rule. Then, compute one step of the Crank-Nicolson scheme for the problem

$$\begin{cases} y'(t) = (t+1)y(t) \\ y(0) = 1 \end{cases}$$

selecting $\Delta t = 2$.

4. State and prove the theorem on the existence and uniqueness of the Lagrange interpolant of a given function.

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Apply the Gaussian elimination method, without pivoting, to solve the linear system $Ax = b$, where

$$\begin{bmatrix} 3 & -2 & 4 \\ 2 & 2 & 2 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 5 \end{bmatrix}$$

showing the intermediate computations.

2. Write the pseudocode of the Newton method. Then, with initial guess $x_0 = 1$ apply one Newton iteration to the equation

$$(6x + 4)(x + 2) = 0$$

ADVANCED EXAM

3. Write the pseudo-code of the composite midpoint quadrature rule, then use the composite midpoint quadrature rule to compute an approximation of

$$\int_{-1}^2 (t + 2t^2) dt$$

by splitting the integration interval $[-1, 2]$ into three subintervals. Report the intermediate computations.

4. Write the pseudo-code of the Jacobi method (for solving linear systems) and prove its convergence for diagonally dominant matrices.

- ☐ I want to take the BASIC EXAM
☐ I want to take the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Starting from $x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$, compute 2 iterations of the Jacobi method to compute an approximate solution of the system $Ax = b$, where

$$A = \begin{bmatrix} 4 & 2 & 0 \\ 0 & 2 & 1 \\ 1 & -1 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}$$

2. Write the pseudocode of the Newton method. With initial guess $x_0 = 1$ apply one Newton iteration to find an approximate solution of the equation

$$(2x + 2)(x + 3) = 0$$

ADVANCED EXAM

3. Write the pseudo-code of the composite midpoint quadrature rule, then use the composite midpoint quadrature rule to compute an approximation of

$$\int_{-1}^2 (t^2 + 2t) dt$$

by splitting the integration interval $[-1, 2]$ into three subintervals. Report the intermediate computations.

4. Write the pseudocode of the Gaussian Elimination Method, without pivoting, and give an estimate of its computational cost. Give an example when the method does not work, that is, pivoting is necessary

- ☐ I want to take the BASIC EXAM
☐ I want to take the the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Given the function $f(x) = \sin(x)$ compute its Lagrange interpolant of degree 2 through the points $x_1 = -\frac{\pi}{2}$, $x_2 = 0$, $x_3 = \pi$.
2. Write the pseudocode of the Gauss-Seidel method to solve a linear system.

ADVANCED EXAM

3. Write the pseudocode of the backward substitution method used to solve linear systems where the matrix is upper triangular. Describe its computational cost. Show how it works when solving the system $Ax = b$, where

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 2 \\ 0 \end{bmatrix}.$$

4. Describe the composite midpoint quadrature rule, state and prove an error bound.

Numerical Methods in Engineering Sciences
9/2/2024

First name: _____

Last name: _____

Student ID: _____

- ☐ I want to take the BASIC EXAM (maximum grade is 24/30)
☐ I want to take the ADVANCED EXAM (maximum grade is 30/30 cum laude)

Total time is 1 hour.

BASIC EXAM

1. Compute the regression parabola $r(x) = c_0 + c_1x + c_2x^2$ for the following set of points

$$(-1, 3), (0, -5), (1, 10), (2, 3), (3, -1).$$

Report the intermediate computations.

2. Write the pseudo-code of the Implicit Euler Method. Then, given the Cauchy problem

$$\begin{cases} y'(t) = 3 - 2ty(t) & \text{for } t > 0 \\ y(0) = 1 \end{cases}$$

compute two steps of the Implicit Euler Method, with $\Delta t = 1/2$, in order to approximate $y(1)$. Report the intermediate computations.

ADVANCED EXAM

3. Write the pseudo-code of the backward substitution method used to solve linear systems where the matrix is upper triangular. Describe (with full justification) its computational cost. Show how the backward substitution works when solving the system $Ux = b$, with

$$U = \begin{bmatrix} -5 & 0 & 5 \\ 0 & 2 & 3 \\ 0 & 0 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 25 \\ 15 \end{bmatrix}$$

4. Describe the composite midpoint quadrature rule, write its pseudo-code, state and prove an error bound.

- ☐ I want to take the BASIC EXAM
☐ I want to take the the ADVANCED EXAM

Exam rules:

- Basic exam: the maximum grade is 24/30.
- Advanced exam: the maximum grade is 30/30 cum laude.

Total time is 1 hour. Students who get a positive grade in the written part (i.e., at least 18/30) *might* choose to take an oral exam. For students who choose the basic written exam, the maximum grade obtainable can never exceed 24/30.

BASIC EXAM

1. Apply the Gaussian elimination method (GEM) algorithm to solve

$$\begin{bmatrix} 3 & 6 & 3 \\ 1 & 4 & 3 \\ 2 & 6 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 6 \\ 0 \end{bmatrix}$$

showing the intermediate computations.

2. Write the pseudocode of the explicit Euler method to solve a Cauchy problem of the form :

$$\begin{cases} y'(t) = f(t, y(t)) \text{ for } t > 0 \\ y(0) = y_0. \end{cases}$$

Then, apply it to the case

$$\begin{cases} y'(t) = -t(y(t))^2 \text{ for } t > 0 \\ y(0) = 2. \end{cases}$$

and compute two steps of the method, with $\Delta t = 1/2$, in order to approximate $y(1)$. Report the intermediate computations.

ADVANCED EXAM

3. Explain what is the LU factorization of a matrix, without pivoting, and give its pseudocode. Then, compute the LU factorization, without pivoting, of

$$\begin{bmatrix} 3 & 6 & 3 \\ 1 & 4 & 3 \\ 2 & 6 & 7 \end{bmatrix}$$

Report the intermediate computations. Why pivoting is necessary in the general case?

4. Write the pseudocode of the Newton method and state the error estimate that expresses its convergence.